



Higher Institute of Technological Studies, Zaghounan



Department of Information Technology

Graduation Project

Project Title:

Academic Monitoring at the Higher Institute of Technological Studies of Zaghounan

Code PFE: DSI19

Host Institution: Higher Institute of Technological Studies, Zaghounan

Company Supervisor: Mr. Mohsen Othmani

ISSET Supervisor: Mr. Mohsen Othmani

Developed by: Mohamed amine Khalil

Period: From 03/02/2025 **To** 31/05/2025

Academic Year: 2024–2025

Dedication

To my dear parents ♡

No dedication can ever express the love, respect, dedication, and support that I have always had for them. No dedication can express what I owe them for their kindness, affection, and support.

Treasures of goodness, generosity, and tenderness, in testimony of my deep love and great gratitude, “May God keep them.”

To my dear brother

In testimony of my sincere gratitude for the efforts he has made in the accomplishment of my studies, I dedicate this modest work to him as a testament of my deep love and infinite gratitude.

To all my friends

for their encouragement and support during the preparation of this project. Their presence and words of motivation have been a constant source of strength and inspiration. Their friendship means the world to me, and I am profoundly grateful for each one of them.

Acknowledgments

I would like to express my sincere gratitude to ISET for offering me the opportunity to carry out my final year project within their institution. Allowed me to apply my academic knowledge in a real-world context and gain hands-on experience in software development. This project has significantly enriched my skills, broadened my understanding of system design and implementation, and helped shape my future career path.

I am especially thankful to my supervisor for the invaluable time, guidance, and effort he devoted throughout the internship. His continuous support, insightful advice, and close supervision were instrumental in overcoming challenges and achieving the project's goals. His willingness to share knowledge, provide constructive feedback, and encourage independent thinking greatly enhanced my learning. I am deeply appreciative of his dedication and mentorship, which made a lasting impact on both the success of this project and my professional growth.

Furthermore, we would like to express our heartfelt thanks to everyone who contributed, in any capacity, to the production of this report and the realization of this project. We hope it meets their expectations and justifies their confidence in us.

Table of Contents

General Introduction	1
Chapter 1: Project Framework and Methodological Choices	3
1. Introduction	3
2. Presentation of the Higher Institute of Technological Studies of Zaghoul	3
2.1. Higher Institute of Technological Studies of Zaghoul (ISETZ)	3
2.2. Activities	3
3. Study of the existing	3
3.1. Overview of the existing process	3
3.2. Existing solutions	4
4. Proposed solution	4
5. Agile methodology	5
5.1. SCRUM Framework	5
5.2. Scrum Events	5
5.3. SCRUM Team	6
6. Conclusion	6
Chapter 2: Project Framework and Methodological Choices	7
1. Introduction	7
2. Specifications of Requirements	7
2.1. Identification of Actors	7
2.2. Functional Requirements	7
2.3. Global Use Case Diagram	9
2.4. Non-functional Requirements	9
3. Technical Specification	12
3.1. Hardware Environment	12
3.2. Technological Environment	12
3.3. Software Environment	14
4. Project Architecture	15
4.1. Logical architecture	15
4.2. Physical Architecture	15
5. Class diagram	17
6. Project Management with SCRUM	18
6.1. Product Backlog	18
6.2. Sprint planning	21
7. Conclusion	21
Chapter 3: Business Intelligence Concepts and Implementation	22
1. Introduction	22
2. Understanding Business Intelligence	22

3.	Data Marts	22
3.1.	Definition	22
3.2.	Fact Tables	23
3.3.	Dimension Tables	23
4.	Data Modelling in BI	23
4.1.	Star Schema	24
4.2.	Snowflake Schema	24
5.	ETL (Extract, Transform, Load)	25
5.1.	Extract	25
5.2.	Transform	25
5.3.	Load	25
6.	Conclusion	25
Chapter 4: Sprint 1 User management, Role management, authentication, web design and navigation foundations		26
1.	Introduction	26
2.	Specification of requirements	26
2.1.	Sprint Backlog	26
2.2.	Use case diagram	27
3.	Design	28
3.1.	Class diagram	28
3.2.	Sequence diagrams and textual description of use cases	28
3.2.1.	Authentication and Logout use case	28
3.2.2.	Reset password use case	30
3.2.3.	User management use case	31
3.2.4.	Role management use case	34
3.2.5.	Account management use case	37
4.	Implementation	39
5.	Conclusion	43
Chapter 5: Sprint 2 Core data flows: timetables, semesters & attendance assignment		44
1.	Introduction	44
2.	Specification of Requirements	44
2.1.	Sprint Backlog	44
2.2.	Use case diagram	45
3.	Design	46
3.1.	Class diagram	46
3.2.	Sequence diagrams and textual description of use cases	47
3.2.1.	Timetable management use case	47
3.2.2.	Semester management user case	49
3.2.3.	Assign attendance	49
3.2.3.	Notifications	50

4. Implementation	51
5. Conclusion	55
Chapter 6: Sprint 3 Reporting & analytics: dashboards	56
1. Introduction	56
2. Specification of Requirements	56
2.1. Sprint Backlog	56
2.2. Use cases	57
3. Connecting to the Data Source	58
4. Data Preparation (Power Query)	59
5. Data Modelling	60
6. Metrics	61
7. Report Design & Visualization	62
8. Security & Role Configuration	66
9. Publishing & Sharing	66
10. Conclusion	67
Chapter 7: Sprint 4 Pdf Reports, admin settings management	68
1. Introduction	68
2. Specification of Requirements	68
2.1. Sprint Backlog	68
2.2. Use cases	69
3. Sequence diagrams and textual description of use cases	69
3.1. Generate and export report pdf use case	69
3.2. Class group create use case	70
3.3. Class group update use case	71
3.4. Class group delete use case	72
4. Implementation	74
	76
5. Conclusion	77

List of Tables

TABLE 1 EXISTING SOLUTIONS	4
TABLE 2 CHARACTERISTICS OF PC	12
TABLE 3 USED TECHNOLOGIES	13
TABLE 4 USED SOFTWARE	14
TABLE 5 PRODUCT BACKLOG	18
TABLE 6 SPRINT PLANNING	21
TABLE 7 SPRINT 1 BACKLOG	26
TABLE 8 ANALYSIS AUTHENTICATION AND LOGOUT USE CASES	29
TABLE 9 ANALYSIS RESET PASSWORD USE CASE	31
TABLE 10 ANALYSIS CREATES USER USE CASE	32
TABLE 11 ANALYSIS UPDATE USER USE CASE	33
TABLE 12 ANALYSIS DELETES USER USE CASE	34
TABLE 13 ANALYSIS CREATE ROLE USE CASE	35
TABLE 14 ANALYSIS UPDATE ROLE USE CASE	36
TABLE 15 ANALYSIS DELETE ROLE USE CASE	37
TABLE 16 ANALYSIS MANAGES ACCOUNT USE CASE	38
TABLE 17 SPRINT 2 BACKLOG	44
TABLE 18 ANALYSIS IMPORT EXCEL FILE TIMETABLE USE CASE	47
TABLE 19 ANALYSIS RESET TIMETABLES USE CASE	48
TABLE 20 ANALYSIS CREATE SEMESTER USE CASE	49
TABLE 21 ANALYSIS ASSIGN ATTENDANCE USE CASE	50
TABLE 22 ANALYSIS USE CASE	51
TABLE 23 SPRINT 3 BACKLOG	56
TABLE 24 SPRINT 4 BACKLOG	68
TABLE 25 ANALYSIS CREATES CLASS GROUP USE CASE	71
TABLE 26 ANALYSIS UPDATE CLASS GROUP USE CASE	72
TABLE 27 ANALYSIS DELETES CLASS GROUP USE CASE	73

List of Figures

FIGURE 1: SEATS ONE.....	4
FIGURE 2: ONETAP	4
FIGURE 3: ATTENDANCE RADAR.....	4
FIGURE 4: SCRUM FRAMEWORK	6
FIGURE 5: SCRUM TEAM.....	6
FIGURE 6: USER USE CASE.....	9
FIGURE 7: SUPERVISOR USE CASE	10
FIGURE 8: HEAD OF DEPARTMENT USE CASE	10
FIGURE 9: PROFESSOR USE CASE.....	10
FIGURE 10: ADMINISTRATOR USE CASE.....	11
FIGURE 11: ADMINISTRATOR AGENT	12
FIGURE 12: ANGULAR LOGO	13
FIGURE 13: FLUTTER LOGO	13
FIGURE 14: DRF LOGO	13
FIGURE 15: POSTGRES LOGO.....	13
FIGURE 16: DOCKER LOGO	13
FIGURE 17: REDIS LOGO	13
FIGURE 18: CELERY LOGO.....	13
FIGURE 19: FIREBASE.....	13
FIGURE 20: POWERBI LOGO	13
FIGURE 21: VS CODE LOGO	14
FIGURE 22: ANDROID STUDIO LOGO	14
FIGURE 23: POSTMAN LOGO.....	14
FIGURE 24: PGADMIN 4 LOGO	14
FIGURE 25: DRAW IO LOGO.....	14
FIGURE 26: POWERBI DESKTOP LOGO	14
FIGURE 27: DOCKER DESKTOP LOGO.....	14
FIGURE 28: TRELLO LOGO.....	14
FIGURE 29: GITHUB.....	14
FIGURE 30: LOGICAL ARCHITECTURE OF THE SYSTEM	15
FIGURE 31: DEPLOYMENT DIAGRAM.....	16
FIGURE 32: CLASS DIAGRAM	17
FIGURE 33: DATA MART PRESENTATION.....	22
FIGURE 34: THE BASICS OF DATA MODELLING IN BUSINESS INTELLIGENCE	23
FIGURE 35: STAR SCHEMA MODEL	24
FIGURE 36: SNOWFLAKES SCHEMA MODEL.....	24
FIGURE 37: ETL PROCESS	25
FIGURE 38: ADMINISTRATOR SPRINT 1 USE CASE	27
FIGURE 39: USER SPRINT 1 USE CASE	27
FIGURE 40: CLASS DIAGRAM SPRINT 1	28
FIGURE 41: AUTHENTICATION SEQUENCE DIAGRAM.....	28
FIGURE 42: LOGOUT SEQUENCE DIAGRAM.....	29
FIGURE 43: CONFIRMS RESET NEW PASSWORD SEQUENCE DIAGRAM	30
FIGURE 44: REQUEST RESET LINK SEQUENCE DIAGRAM.....	30
FIGURE 45: CREATE USER SEQUENCE DIAGRAM	32
FIGURE 46: UPDATE USER SEQUENCE DIAGRAM.....	33
FIGURE 47: DELETE USER SEQUENCE DIAGRAM	34
FIGURE 48: CREATE ROLE SEQUENCE DIAGRAM.....	35
FIGURE 49: UPDATE ROLE SEQUENCE DIAGRAM	36
FIGURE 50: DELETE ROLE SEQUENCE DIAGRAM.....	37
FIGURE 51: ACCOUNT UPDATE SEQUENCE DIAGRAM.....	38
FIGURE 52: LOGIN PAGE	39
FIGURE 53: RESET PASSWORD PAGE	39

FIGURE 54: MESSAGE CONFIRM RESET PASSWORD REQUEST WAS SENT	40
FIGURE 55: EMAIL RECEIVED WITH RESET LINK	40
FIGURE 56: MESSAGE DISPLAYED IN CASE THE TOKEN IN INVALID OR EXPIRED	40
FIGURE 57: ACCOUNT SETTINGS PAGE	41
FIGURE 58: NEW PASSWORD PAGE	41
FIGURE 59: DELETE USER.....	42
FIGURE 60: ADD USER FORM	42
FIGURE 61: ADD ROLE FORM WITH VERIFICATION OF EXISTING ONES	43
FIGURE 62: SUCCESS MESSAGE	43
FIGURE 63: SPRINT 2 ADMINISTRATOR AGENT USE CASES	45
FIGURE 64: SPRINT 2 ADMINISTRATOR USE CASES	45
FIGURE 65: SPRINT 2 CLASS DIAGRAM.....	46
FIGURE 66: IMPORT EXCEL FILE TIMETABLE SEQUENCE DIAGRAM	47
FIGURE 67: RESET TIMETABLES SEQUENCE DIAGRAM.....	48
FIGURE 68: CREATE SEMESTER SEQUENCE DIAGRAM.....	49
FIGURE 69: ASSIGN ATTENDANCE SEQUENCE DIAGRAM	50
FIGURE 70: NOTIFICATION SEQUENCE DIAGRAM	51
FIGURE 71: ADD NEW SEMESTER	52
FIGURE 72: RESULT OF IMPORT EXCEL FILE	52
FIGURE 73: DOCKER CONTAINER.....	53
FIGURE 74: VIEWING TIMETABLE AFTER FILTERING	53
FIGURE 75: CURRENT SESSION SCREEN	54
FIGURE 76: DOCKER CONSOLE NOTIFICATION SUCCESS	54
FIGURE 77: NOTIFICATION	54
FIGURE 78: ASSIGNING ATTENDANCE	54
FIGURE 79: SUPERVISOR USE CASE	57
FIGURE 80: HEAD OF DEPARTMENT USE CASE.....	57
FIGURE 81: PROFESSOR USE CASE.....	57
FIGURE 82: CHOOSING SOURCE POSTGRESSQL	58
FIGURE 83: CONNECTING TO THE DATABASE.....	58
FIGURE 84: ADDING COLUM.....	59
FIGURE 85: POWER QUERY POWER BI SCREEN	59
FIGURE 86: OUR SNOWFLAKE SCHEMA	60
FIGURE 87: MODELLING AND RELATIONSHIP FROM POWER BI.....	60
FIGURE 88: METRICS FROM TABLE FACTTIMETABLE	61
FIGURE 89: METRICS PRESENT COUNT USING DAX	62
FIGURE 90: METRICS ATTENDANCE RATE % USING DAX.....	62
FIGURE 91: FIRST REPORT “OVERALL ATTENDANCE”	62
FIGURE 92: THIRD REPORT “PROFESSOR REPORT”	63
FIGURE 93: SECOND REPORT “COURSE REPORT”	63
FIGURE 94: FOURTH REPORT “GROUP REPORT”	64
FIGURE 95: FIFTH REPORT: MONTHLY ALERTS	64
FIGURE 96: SEVENTH REPORT: UNASSIGNED REPORT	65
FIGURE 97: SIXTH REPORT: CLASSROOMS REPORT.....	65
FIGURE 98: POWER BI MANAGE ROLES	66
FIGURE 99: POWER BI SERVICE.....	66
FIGURE 100: ADMINISTRATOR AGENT USE CASE SPRINT 4	69
FIGURE 101: ADMINISTRATOR SPRINT 4 USE CASE	69
FIGURE 102: EXPORT PDF SEQUENCE DIAGRAM.....	70
FIGURE 103: CREATE CLASS GROUP SEQUENCE DIAGRAM	71
FIGURE 104: UPDATE CLASS GROUP SEQUENCE DIAGRAM	72
FIGURE 105: DELETE CLASS GROUP SEQUENCE DIAGRAM	73
FIGURE 106:DEPARTMENT MANAGEMENT SCREEN.....	74
FIGURE 107: CLASS GROUP MANAGEMENT SCREEN	74
FIGURE 108: SESSION MANAGEMENT SCREEN.....	75
FIGURE 109: CLASSROOM MANAGEMENT SCREEN.....	75
FIGURE 110; COURSE MANAGEMENT SCREEN	76

FIGURE 111: REPORT SCREEN 76

FIGURE 112: PDF REPORT DOWNLOADED ON DEVICE 76

General Introduction

In today's digital world, the management, collection, and analysis of data have become central pillars of efficient operations across industries, particularly within the educational sector. Institutions today organize vast amounts of operational data, from class schedules to teacher attendance records, utilizing specialized systems designed for this purpose. The growing volume of data presents challenges that go beyond simple storage, institutions must ensure that data is accessible, structured, secure, and available in a format that can be analysed to support decision-making and continuous improvement.

In this context, academic monitoring systems are powerful tools for educational institutions, offering better organization, and deeper insights into academic activities. By digitizing and centralizing essential processes such as attendance tracking, class scheduling, and performance analysis these systems enhance both the administrative workflow and the educational experience. Furthermore, utilizing data analytics transforms raw operational data into strategic insights, helping institutions optimize resource usage, detect emerging trends, helps ensure the smooth progression of courses and facilitates coordination among professors and plan future academic activities more effectively.

As part of our end-of-studies program this project was conceived and developed within the Higher Institute of Technological Studies of Zaghuan. Its primary objective is to design and implement a comprehensive academic monitoring system that addresses critical academic management needs. The proposed system is composed of three core components:

1. Web platform for importing and organizing academic schedules such as study days timetables into structured database.
2. Mobile application for facilitating real-time attendance management
3. Data analytics module for producing detailed academic reports.

The core problem this project seeks to solve is digitizing manual processes to unlock their full potential. Many institutions still depend on paper-based systems for attendance, scheduling, and performance tracking, which leads to reporting delays and poor visibility into daily. This inefficiency not only wastes resources but also restricts decision-making, lower both operational effectiveness and educational quality. By closing these gaps, the project will streamline workflows, enhance data accuracy, and foster seamless collaboration among academic staff.

To achieve these goals, we utilized modern and reliable technologies. The mobile application was developed using Flutter, offering a lightweight and intuitive interface for attendance tracking. The web platform was developed using Angular and Tailwind CSS, providing an efficient and responsive environment for managing academic schedules and managing system settings. PostgreSQL was chosen for robust and scalable data storage, while Django was employed for server-side management and API development. For the analytical component, Power BI was used to generate dynamic reports and visualizations based on collected academic data, offering valuable insights into attendance patterns, course progress, professors engagement, and classroom usage throughout the semester.

The present report is composed of seven chapters structured as follows:

- The first chapter, titled “Project Framework and Methodological Choices”, introduces the broader context of digitalization in academic institutions, outlines the initial challenges identified at the Higher Institute of Technological Studies of Zaghuan, and describes the methodology adopted for this project.
- The second chapter, titled “Analysis and Specification of Requirements”, details the functional and non-functional requirements of the system and presents the chosen system architecture.
- The third chapter titled “Business Intelligence Concepts and Implementation” provides a general overview of Business Intelligence (BI), exploring its key concepts, components, and goals. It also introduces common BI implementation approaches used.
- The following four chapters chronicle the project’s implementation across a series of development sprints: “Sprint 1”, “Sprint 2”, “Sprint 3”, and “Sprint 4”. Each sprint chapter describes its goals, analyses the work completed, presents the design and development stages, and concludes with reflections on the progress made and challenges encountered.
- Finally, the report concludes with a “General conclusion”, summarizing the results achieved, reflecting on the project's objectives, and discussing possible future improvements to the academic monitoring system.

Chapter 1: Project Framework and Methodological Choices

1. Introduction

In this chapter, we begin by presenting an overview of the higher institute of Technological Studies of Zaghoun and its core functions. We then examine the current state of academic monitoring within the institution, outline the proposed solution tailored to the identified challenges, and detail the adopted project management methodology namely the Agile framework, with a particular focus on the scrum approach

2. Presentation of the Higher Institute of Technological Studies of Zaghoun

2.1. Higher Institute of Technological Studies of Zaghoun (ISETZ)

ISET (Institut Supérieur des Études Technologiques) is a network of 25 public higher-education institutes in Tunisia. They operate under the supervision of the Ministry of Higher Education and Scientific Research and are managed by the Directorate-General of Technological Studies. These institutes offer a range of programs, including applied undergraduate degrees, continuing education aligned with industry needs, technology resource centres, skills hubs, and business incubators [1].

2.2. Activities

- **Initial degree programmes** in fields like IT, engineering, business, etc.
- **Continuing-education courses** tailored to evolving industry needs.
- **Technology resource centres** providing access to labs, equipment and prototyping facilities.
- **Competency hubs** specialising in key technical domains.
- **Business incubators** supporting student and graduate startups.

3. Study of the existing

In this section, we will look at the current process for managing attendance and monitoring teaching activities within our hosting university, as well as review similar apps or systems available in the market, highlighting their strengths and weaknesses.

3.1. Overview of the existing process

The current existing process, an administrator agent prints out multiple PDF timetables one for each group of different departments and physically to mark lectures attendance on paper during every session. This manual approach has several clear drawbacks:

- **Time consuming searches:** Sifting through dozens of unorganized timetables just to find the right classroom or lecture.
- **No filtering:** Paper records can't be sorted by building, session, or professor to facilitate the search and the attendance assignment.
- **Frequent errors:** Hand-written entries lead to typos, missed marks, and lost sheets.
- **Slow reporting:** reporting daily or monthly attendance statistics from scattered papers takes hours even days.

- **Wasteful use of resources:** Constantly printing and storing reams of paper.
- **Unusable data:** Attendance data or any other data available during this process can't be useful into other apps or analytics tools.
- **Scaling challenges:** As stacks of paper grow, managing and organizing them becomes impossible.
- **Poor archiving:** Old records are hard to file, retrieve, or analyse over time.

3.2. Existing solutions

Table 1 Existing Solutions

Solutions	Advantages	Disadvantages
 Figure 1: SEAtS ONE	[2] • Full campus solution (attendance, timetables, analytics) • Can import Excel timetables. • Mobile app for live attendance. • Powerful built-in dashboards and alerts.	• Very expensive for small schools • Setup is complex needs training and IT support • Heavy system if you only need simple features
 Figure 2: OneTap	[3] • Very simple to use • Can import Excel lists • Cheap/free options • Mobile/tablet ready (QR code attendance)	• Attendance based on students scanning (not agent-driven) • Timetable management is limited • You often need to export data to analyse more deeply)
 Figure 3: Attendance Radar	[4] • Super-fast Bluetooth-based attendance • Very easy setup (small app) • Allows Excel imports • Good for quick sessions and mass attendance	• Limited to Bluetooth/GPS students need devices with them • No deep analytics/dashboards inside app • More suited to single-class sessions than complex campus-wide operations

4. Proposed solution

Based on a detailed analysis of the current process, available resources, and a review of similar market solutions, we propose the development of a system with a user-friendly interface. This project aims to build a Web and Mobile application to support the monitoring of teaching activities at ISET Zaghuan. The app will provide advanced features for managing timetables, tracking attendance, and generating statistical reports to support better management decisions and improve the quality of education which provides the following functionalities:

- **Automate timetable imports:** organize schedules straight from Excel into your database no manual entry required.
- **Track teaching sessions & absences:** Record both individual professors absences and Collective absences affecting entire student classes.
- **Threshold alerts:** Get flagged when course completion falls below a set percentage.
- **Progress comparison:** Monitor the progress of professors teaching the same course by comparing each group's advancement side-by-side, ensuring alignment and consistent coverage of content before examinations.

- **Room-usage report:** Generate stats on how often and how fully each classroom is used.
- **Visual dashboard:** Show key metrics at a glance with charts progress curves, absence histograms, and more.
- **Secure, role-based login:** Keep data safe with different access levels for teachers, department heads, and admins.
- **Easy report exports:** Download any report as PDF or Excel for sharing or archiving.
- **Mid-term & end-of-term summaries**

5. Agile methodology

Agile [5] Software Development is a methodology used in software production that emphasizes flexible, collaborative working. It promotes adaptive planning, early delivery, and continuous improvement, along with the ability to respond quickly to changes. Rather than a linear approach, this process is iterative, allowing teams to manage their work in small, usable increments.

Agile is based on four core values. The values of the Agile methodology emphasize:

- Individuals and interactions over processes and tools.
- Working on software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

5.1. SCRUM Framework

Scrum [6] is an empirical process, where decisions are based on observation, experience and experimentation. Scrum has three pillars: transparency, inspection and adaptation. This supports the concept of working iteratively. Think of Empiricism as working through small experiments, learning from that work and adapting both what you are doing and how you are doing it as needed.

5.2. Scrum Events

Scrum events facilitate the adaptation of certain aspects of the process, product, progress, or relationships:

- **Sprint Planning:** The goal is to define what will be done in the sprint and how it will be done. This meeting is held at the beginning of each sprint and defines how the project will be approached based on the steps and deadlines of the Product Backlog
- **Daily/Weekly Meeting:** During the sprint, team members hold a 15-minute daily meeting (or one hour weekly) to answer three questions individually: What did I do yesterday? What will I do today? What help do I need?
- **Sprint Review:** Held at the end of the sprint, the development team presents the completed features and gathers feedback from user representatives. This is also when planning for the next sprints can be anticipated.
- **Sprint Retrospective:** The team reviews the sprint goals, notes positive and negative points, and aims to avoid repeating mistakes.

As indicated in the figure below, the Scrum method follows an iterative process to obtain a product that closely meets client needs by considering their evolution and thus maximizing the value of the delivered product. If a problem arises during a sprint, responsibility is shared between the Product Owner, the Scrum Master, and the Scrum Team.

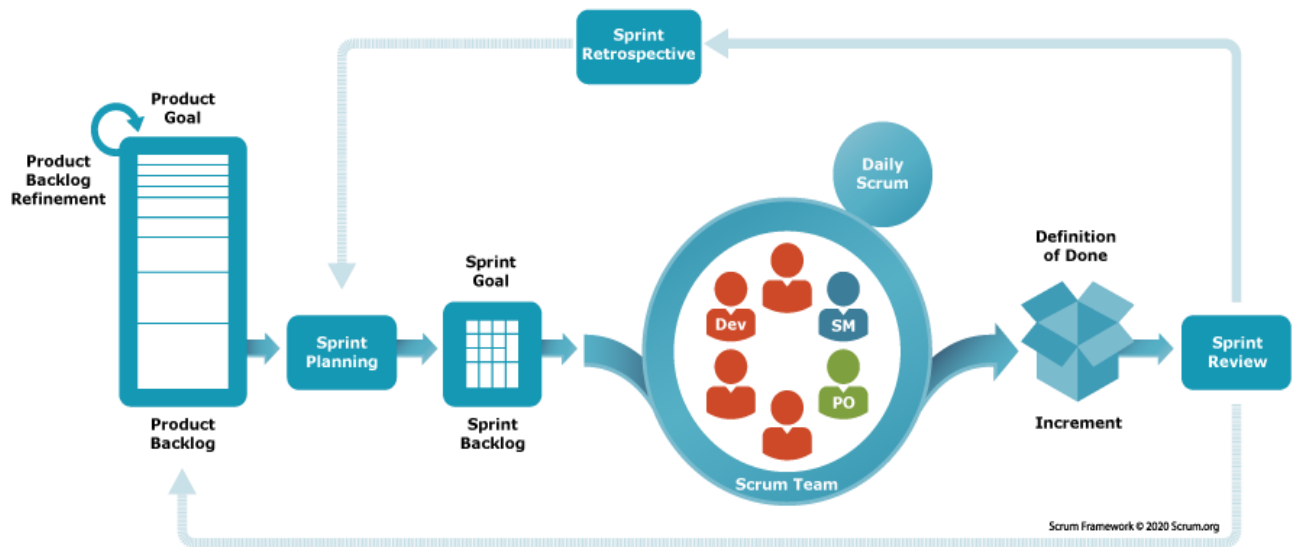


Figure 4: Scrum Framework

5.3. SCRUM Team

The fundamental unit of Scrum is a small team of people, a Scrum Team. The Scrum Team consists of one Scrum Master, one Product Owner, and Developers. Within a Scrum Team, there are no sub-teams or hierarchies. It is a cohesive unit of professionals focused on one objective at a time, the Product Goal.

- **Product Owner:** is accountable for maximizing the value of the product resulting from the work of the Scrum Team. How this is done may vary widely across organizations, Scrum Teams and individuals.
- **Scrum Master:** has many responsibilities within an organization and is ultimately accountable for the success of the Scrum Team.
- **Development Team:** are the people on the Scrum Team that are committed to creating any aspect of a usable Increment each sprint.

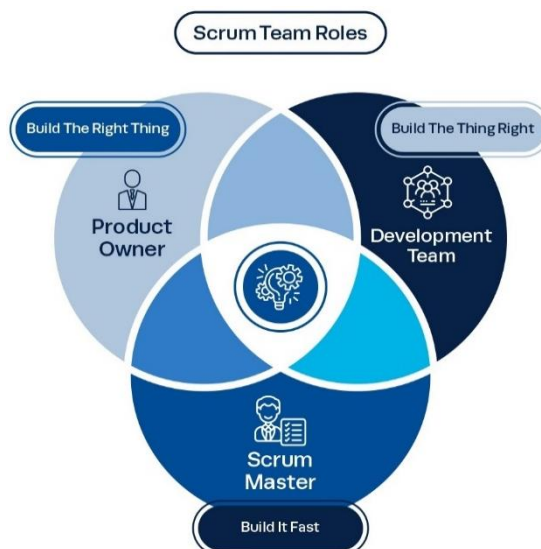


Figure 5: Scrum Team

6. Conclusion

In this chapter, we introduced our project by outlining the proposed solution and the Agile/Scrum methodology adopted for its development. In chapter 2 we will explore the key theoretical foundations and concepts that underpin and inform the design and implementation of our solution.

Chapter 2: Project Framework and Methodological Choices

1. Introduction

This chapter delves into the detailed requirements for the Academic Monitoring at the Higher Institute of Technological Studies of Zaghouan project. It includes the identification of key actors involved in the system, outlines the functional and non-functional requirements, and specifies the technical aspects necessary for its implementation. Furthermore, it presents the overall project architecture and describes the use of the SCRUM methodology to manage development tasks, with an emphasis on the product backlog and sprint planning to track progress and ensure timely delivery.

2. Specifications of Requirements

2.1. Identification of Actors

Based on the project specification and our objectives, we have identified five primary user roles who will interact with and benefit from the system:

- **Administrator:** Responsible for end-to-end management of the application. This includes configuring user accounts, defining semesters and courses, managing classrooms, and maintaining up-to-date timetables by importing Excel-based schedules into the database.
- **Administrator Agent:** The frontline user who conducts attendance checks in real time. The mobile app will present only the current session based on date and time enabling the agent to record attendance efficiently and accurately.
- **Supervisor:** Has read-only access to comprehensive Power BI dashboards. These reports visualize key metrics such as course progress, attendance trends, and overall performance so that supervisors can monitor project health and intervene proactively.
- **Head of Department:** Views department-specific analytics via the Power BI interface. This role focuses on high-level insight into course outcomes, resource utilization, and attendance patterns within their department.
- **Professor:** Accesses individual performance metrics and course-level data. In addition to reviewing their own attendance records and session statistics, professors can compare progress across all sections of the same course viewing peers' metrics to ensure consistent pacing and facilitate collaborative exam creation thereby maintaining a coherent academic environment.

2.2. Functional Requirements

Functional requirements specify the concrete capabilities the system must provide to satisfy each user will interact or benefits from the system:

System must allow Any Authenticated User to:

- **Login:** Users must be able to authenticate using email or username plus password.
- **Persistent Session:** Users remain logged in across browser or app sessions until they explicitly log out.
- **Password Reset:** Users can initiate a secure “forgot password” flow and receive a reset link via email.

- **Account Update:** Users can modify their profile information (name, email) and change their password.
- System must allow Administrator to:
- **Manage Semesters:** Create, edit, delete semesters, and toggle their active/inactive status.
- **Manage Timetables:** Import Excel-based schedules for the current semester; reset or replace future session entries when changes occur.
- **Generate Reports:** Export PDF reports of unassigned or pending attendance records.
- **Configure System Settings:** Define and maintain core entities (users, administrator agents, sessions, courses, class groups, classrooms) and application parameters.

System must allow Administrator Agent to:

- **View Current Sessions:** Display all sessions matching the current date and time window.
- **Assign Attendance:** Mark students absent, professor absent or present with optional observations, and submit in real time.
- **Filter Sessions:** Quickly narrow lessons by building/block and by assignment status (unassigned vs. assigned) to minimize errors.
- **Edit Recent Assignments:** Within the same session period, revise attendance entries; afterward, only administrators may modify them.
- **Generate Attendance Reports:** Export attendance records for a selected date or range as PDF.

System must allow Supervisor to:

- **View University-Wide Metrics:** Dashboard of overall attendance rates, courses below progress targets, professor and class-group performance, and daily trends.
- **Periodic Analysis:** Filter metrics by custom date ranges (weekly, monthly, semester).
- **Department & Course Drill-Down and Drill-Through:** Inspect attendance and progress statistics for individual departments, specific courses, professors, or class groups.
- **Monthly Alerts:** Highlight professors or classes that fall below defined attendance thresholds, filtered by month.
- **Classroom Utilization:** Analyse room usage rates.
- **Attendance assignment Reports:** Displays unassigned attendance entries and summary statistics related to assigned records.

System must allow Head of Department to:

- **Department Overview:** Access aggregated attendance and progress statistics for all courses in their department.
- **Section Comparison:** Compare performance across different sections of the same course to ensure consistency.

System must allow Professors to:

- **Personal Analytics:** View one's own attendance records, session completion rates, and academic indicators.
- **Course Benchmarking:** Compare attendance and progress metrics across all sections of courses they teach, alongside peers' data to align pacing.

2.3. Global Use Case Diagram

The general use case diagrams of our system, as illustrated in Figures 6 through 10, encapsulate the core functionalities and interactions between various user roles within the application. These diagrams serve as foundational blueprints, providing a clear representation of how each actor namely, the Administrator, Administrator agent, supervisor, Head of Department and Professor interact with the system's features. They ensure a comprehensive understanding of user requirements and system capabilities, facilitating effective development and integration of all components.

2.4. Non-functional Requirements

- **API Responsiveness:** All API endpoints should respond within 2 seconds under normal load conditions.
- **Report Generation:** Generating reports (e.g., PDF, Excel) should not exceed 5 seconds.
- **Data Volume:** Efficiently handle growing volumes of attendance records and timetable data.
- **Access Tokens:** Short-lived JWTs for API access, minimizing exposure if compromised.
- **Refresh Tokens:** Long-lived, securely stored tokens to obtain new access tokens without re-login; revocable on logout or suspicious activity.
- **API Integration:** Provide APIs for data synchronization and interoperability with other organizational tools and platforms.
- **Row-Level Security (RLS) in Power BI:** Defines roles/filters so that Supervisors, HODs, and Professors only see the rows (departments, courses, groups) they're authorized to view.
- **Intuitive UI:** Clear workflows and responsive layouts for both web (admin) and mobile (administrator agents).
- **Modular Codebase:** Django REST Framework divided into apps; clear separation of concerns.
- **Mobile Platforms:** Full support for Android 13+.
- **Browser Support:** Web admin app functional on Chrome, Firefox, Edge, Safari (latest releases).

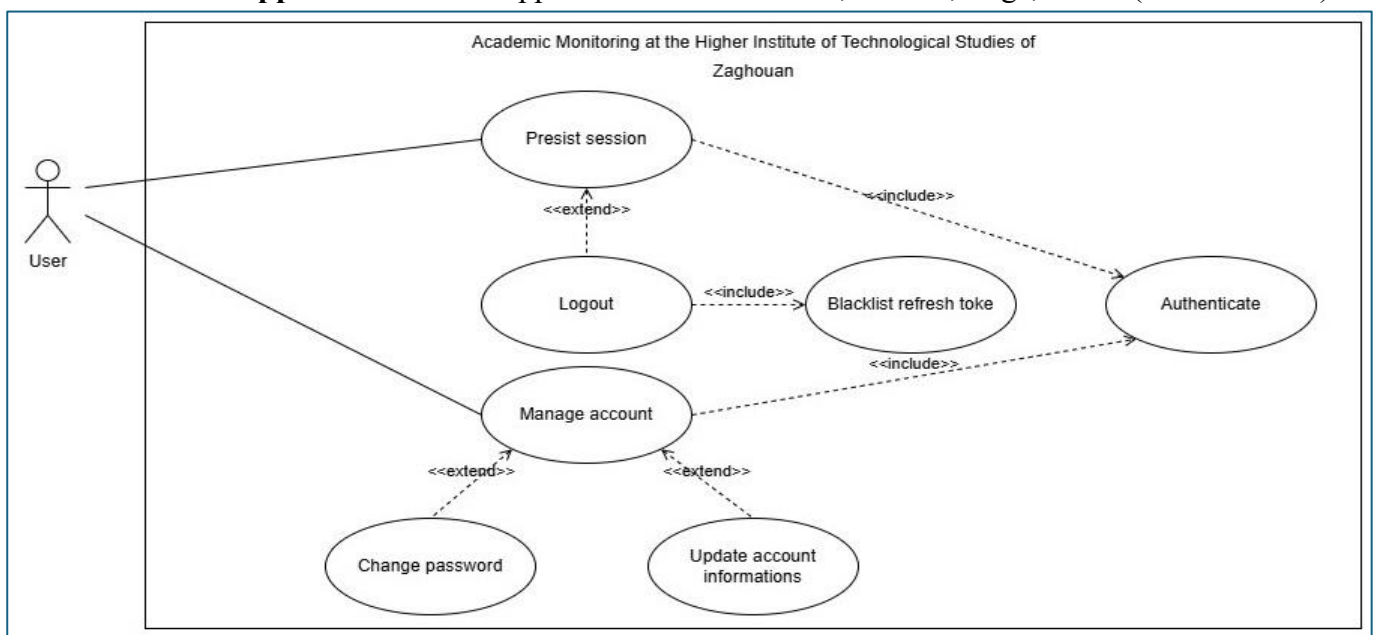


Figure 6: User use case

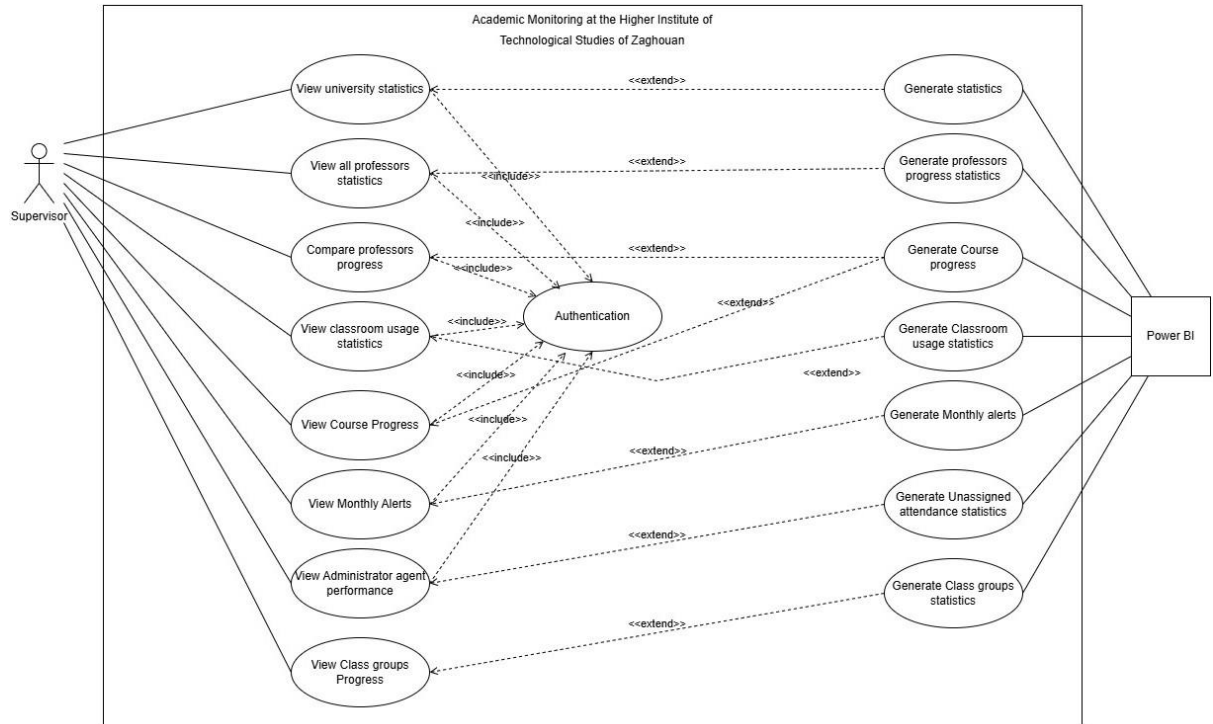


Figure 7: Supervisor use case

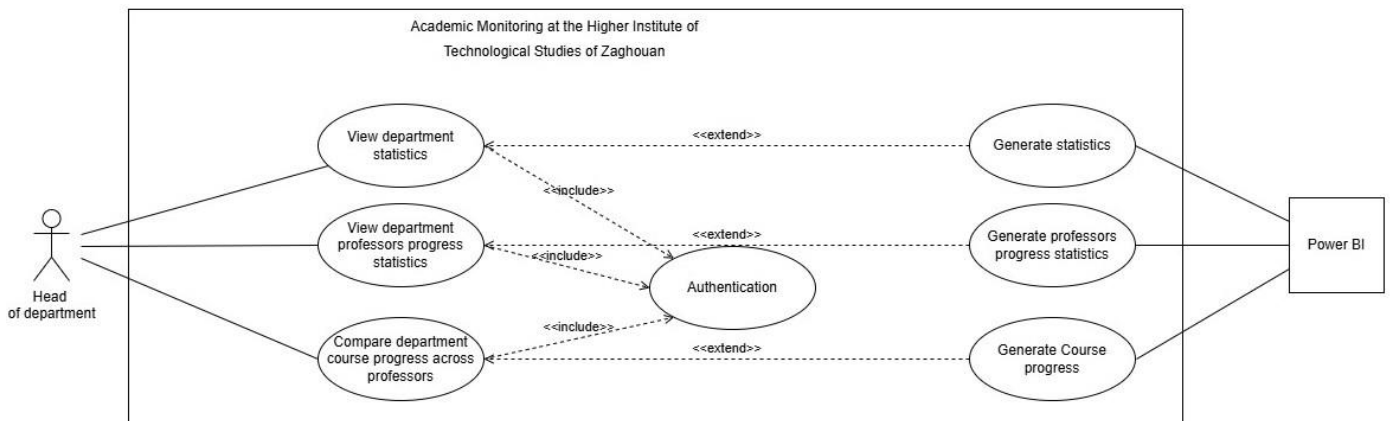


Figure 8: Head of department use case

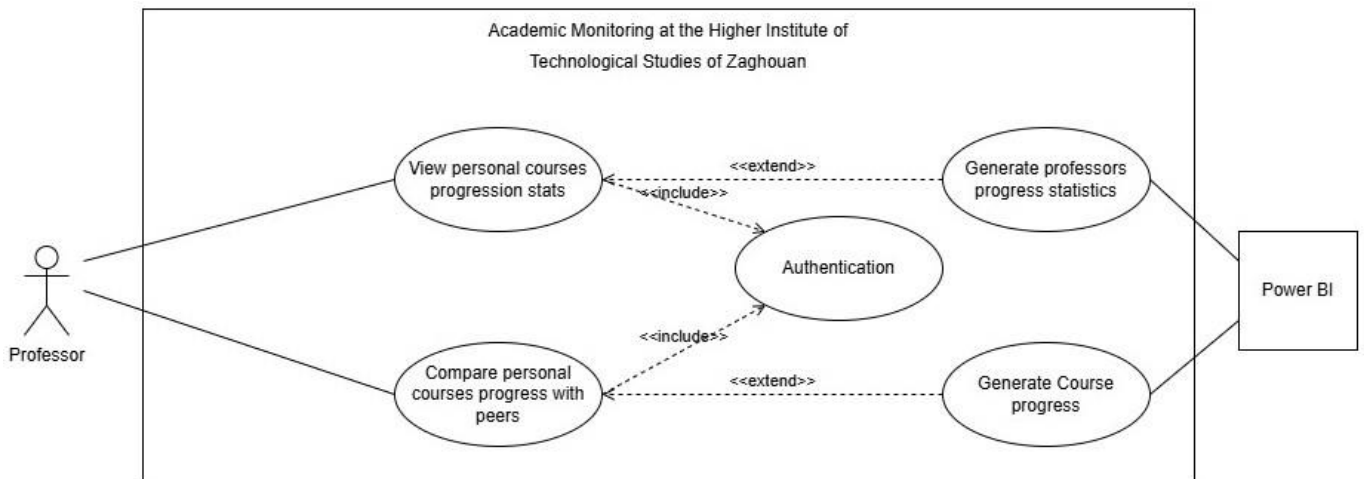


Figure 9: Professor use case

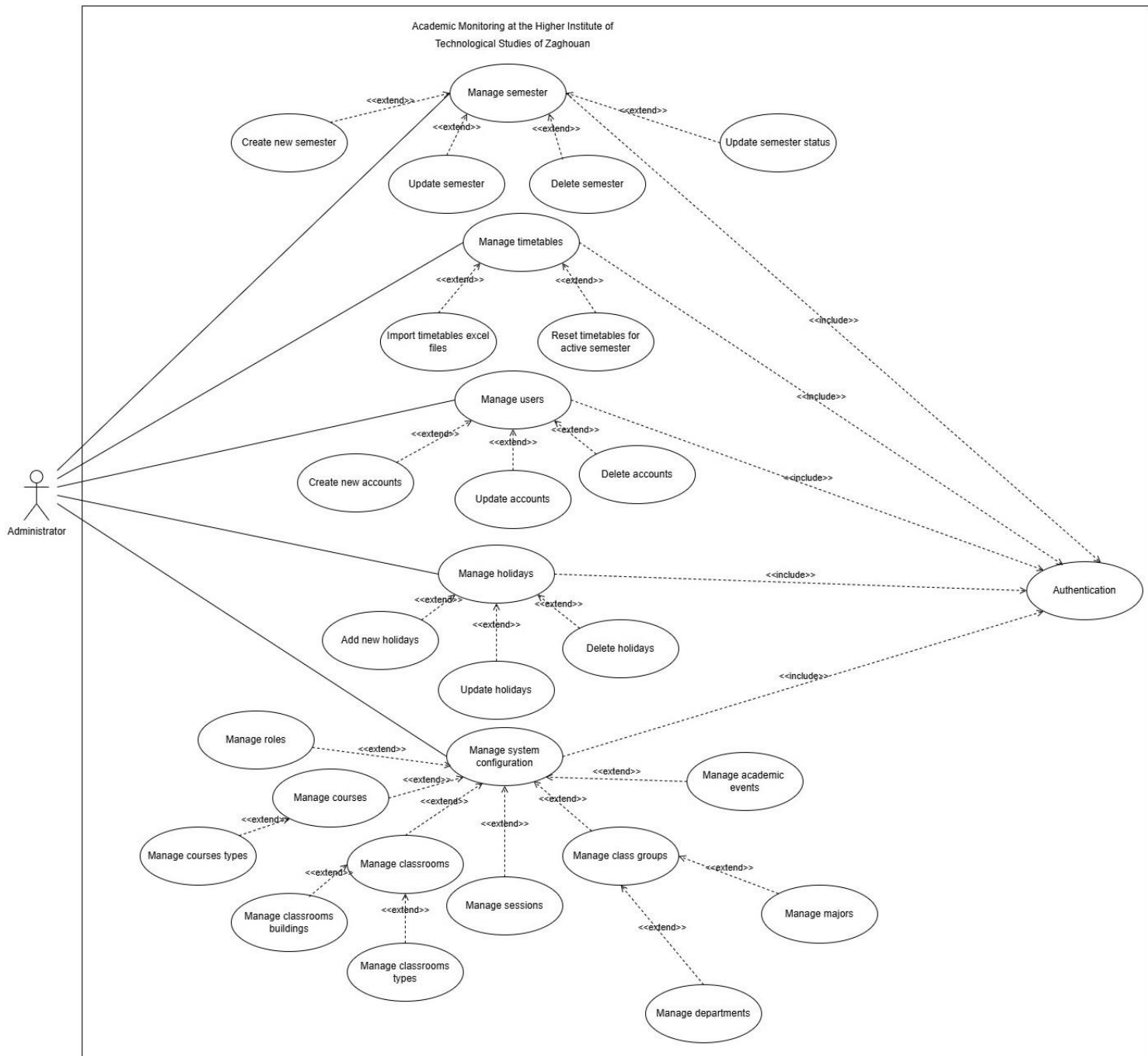


Figure 10: Administrator use case

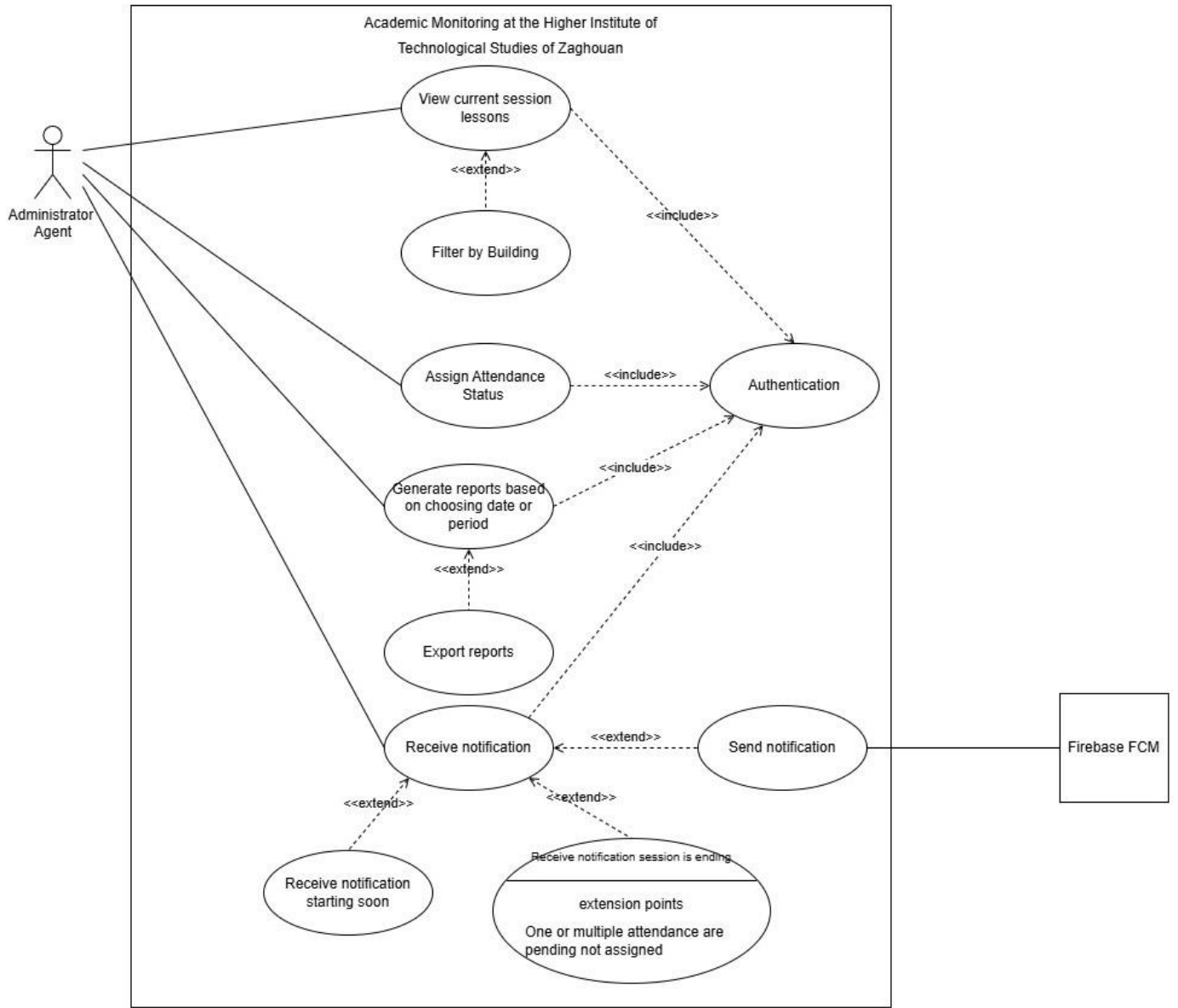


Figure 11: Administrator agent

3. Technical Specification

3.1. Hardware Environment

For the completion of our project, we used PC with the characteristics listed in Tables 2.

Table 2 Characteristics of PC

Computer	Lenovo IdeaPad 110
Processor	i7-8550U CPU @ 1.80GHz 2.00 GHz
Graphic card	Nvidia MX110
Hard disk	512GB
RAM	8GB
Operating system	Windows 11

3.2. Technological Environment

This section will focus on the study of various languages and frameworks to make an informed choice for the execution of our project. Table 3 presents the different technologies that will be used

Table 3 Used technologies

Technology	Description
 Figure 12: Angular logo	Angular [7] is an application-design framework and development platform for creating efficient and sophisticated single-page apps.
 Figure 13: Flutter logo	Flutter [8] is an open-source framework for building beautiful, natively compiled, multi-platform applications from a single codebase.
 Figure 14: DRF LOGO	Django Rest Framework (DRF) [9] is a widely-used, full-featured API framework designed for building RESTful APIs with Django.
 Figure 15: Postgres logo	PostgreSQL [10] is a powerful, open-source object-relational database system that uses and extends the SQL language combined with many features that safely store and scale the most complicated data workloads.
 Figure 16: Docker logo	Docker [11] an open platform for developing, shipping, and running applications, enabling you to separate your applications from your infrastructure so you can deliver software quickly.
 Figure 17: Redis logo	Redis [12] providing cloud and on-prem solutions for caching. Redis's role not just as a cache or data store, but specifically as a reliable, low-latency intermediary for passing messages between services.
 Figure 18: Celery logo	Celery [13] is a simple, flexible, and reliable distributed system to process vast amounts of messages. It's a task queue with focus on real-time processing, while also supporting task scheduling.
 Figure 19: Firebase	Firestore Cloud Messaging (FCM) [14] you can notify a client app; you can send notification messages to drive user re-engagement and retention.
 Figure 20: PowerBI logo	Power BI [15] is a suite of business analytics tools to analyse data and share insights. Monitor your business and get answers quickly with rich dashboards available on every device.

3.3. Software Environment

Table 4 Used software

Software	Description
 Figure 21: vs code logo	Visual Studio Code [16] a lightweight but powerful source code editor which runs on your desktop and is available for Windows, macOS, and Linux.
 Figure 22: Android studio logo	Android Studio [17] is the official Integrated Development Environment (IDE) for Android app development.
 Figure 23: Postman logo	Postman [18] is the single platform for designing, building, and scaling APIs together.
 Figure 24: pgAdmin 4 logo	pgAdmin [19] is the most popular and feature rich Open-Source administration and development platform for PostgreSQL, the most advanced Open-Source database in the world.
 Figure 25: Draw io logo	Draw.io [20] Collaborate with shared cursors in real-time. draw.io has everything you expect from a professional diagramming tool.
 Figure 26: PowerBI desktop logo	Power BI [14] Create rich, interactive reports with visual analytics at your fingertips
 Figure 27: Docker Desktop logo	Docker Desktop [21] enhances your development experience by offering a powerful, user-friendly platform for container management.
 Figure 28: Trello logo	Trello [22] manages the big milestones and the day-to-day tasks of working together and getting things done.
 Figure 29: GitHub	GitHub [23] is a collaborative platform for version control and open-source development.

4. Project Architecture

This section provides a detailed overview of our attendance monitoring system's architecture, covering both its physical deployment and logical layering.

4.1. Logical architecture

Logical concerns are separated into presentation, business, and data layers. This architecture supports a hybrid setup involving web, mobile, and analytics components presenter on figure 30. We will use deployment and class diagrams to illustrate how each component is instantiated and how they communicate.

- **The presentation layer** consists of three user-facing interfaces: a web app (Angular), a mobile app (Flutter), and Power BI. The web and mobile apps interact with the system via HTTP requests, while Power BI connects directly to the database to generate analytical reports.
- **The business layer** is represented by the Django REST API, which processes client requests, enforces business rules, and coordinates background tasks.
- **The data layer** is composed of a central database that handles all persistent storage needs and supports both transactional queries from the API and analytical imports from Power BI.

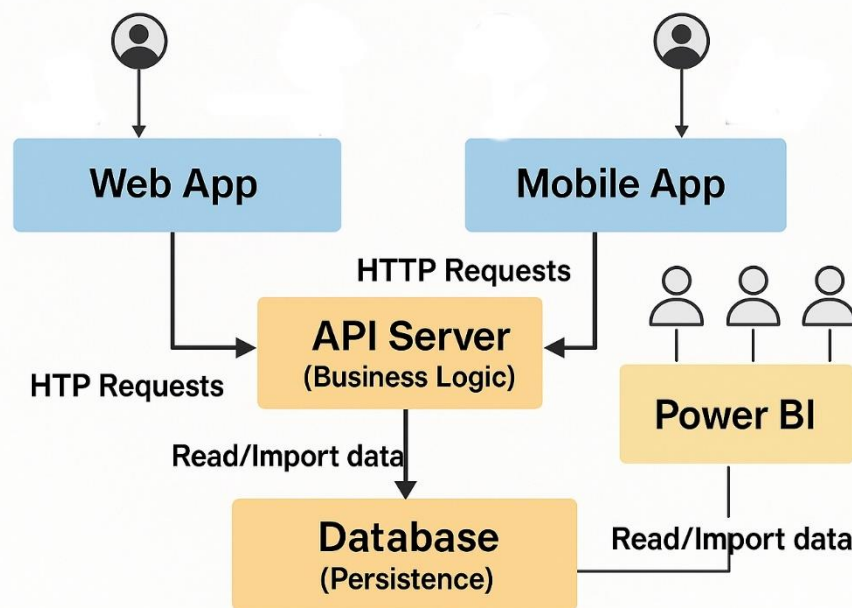


Figure 30: Logical Architecture of the system

4.2. Physical Architecture

The physical architecture describes how our system's components are deployed onto real infrastructure and interconnected over the network. Client devices (the Angular web app and Flutter mobile app) communicate via HTTPS with a dedicated Django API server, which in turn connects to a standalone PostgreSQL DBMS on its own host. Asynchronous workload is isolated on a Docker host running Redis, Celery Beat, and Celery Worker containers. Beat schedules jobs into Redis, Worker dequeues them, then issues HTTP calls back to the API and sends notifications via SMTP or Firebase Cloud Messaging. Reporting is handled by the Power BI service, which directly imports data from the PostgreSQL database. The following deployment diagram illustrates the physical distribution of these components, their hosting nodes, and all key network links.

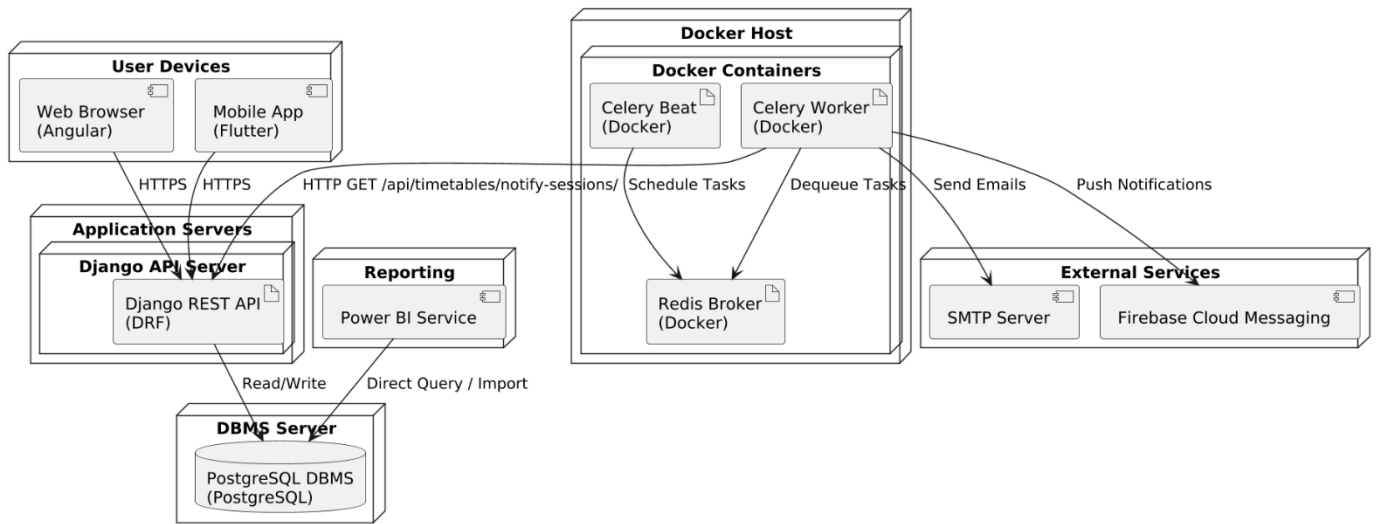


Figure 31: Deployment diagram

5. Class diagram

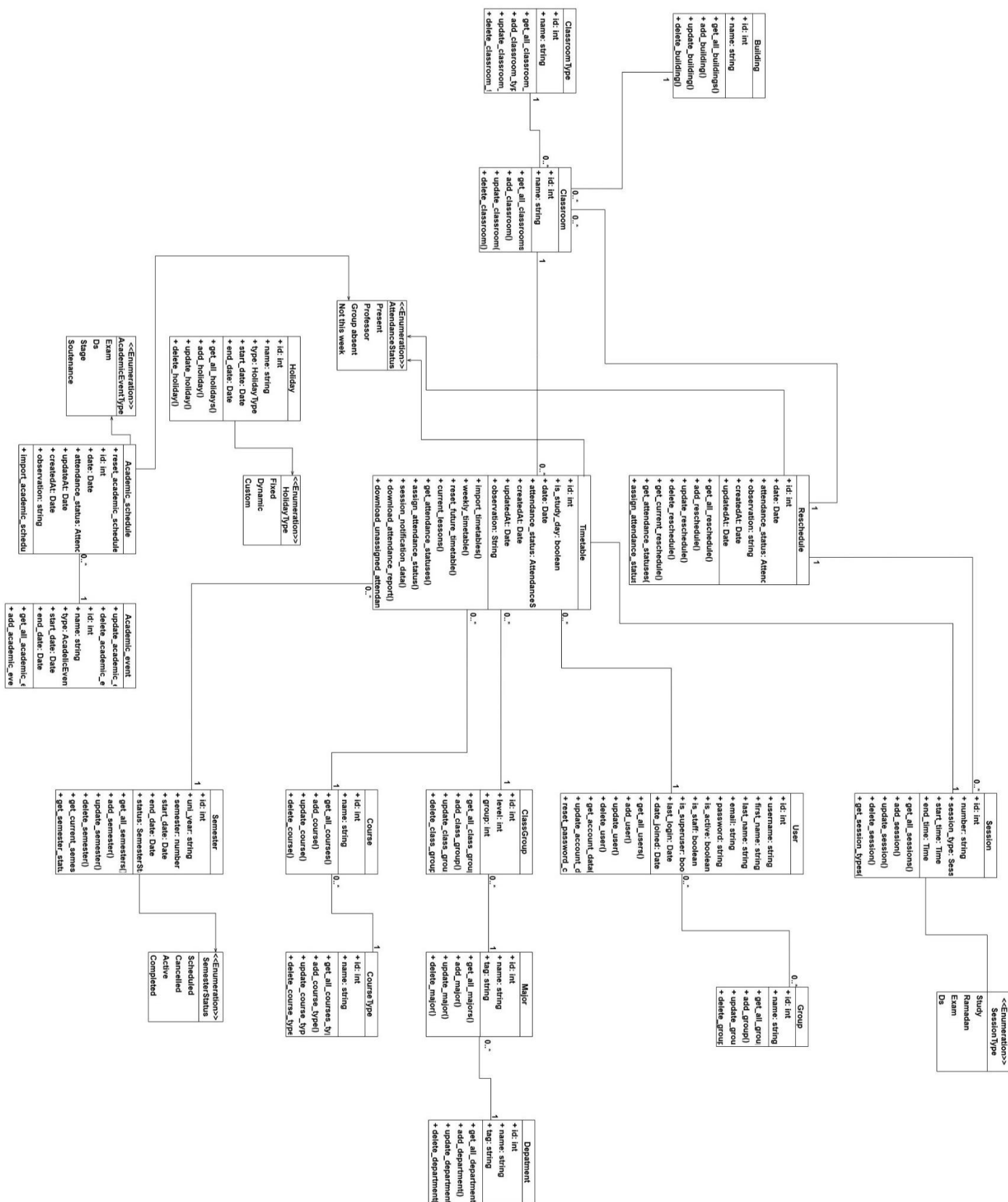


Figure 32: Class diagram

6. Project Management with SCRUM

6.1. Product Backlog

The Product Backlog serves as the single source of truth for all requirements in our Scrum project and drives sprint planning. Table 5 below defines each backlog item using the following attributes:

- **ID:** A unique, auto-incremented identifier for each backlog entry.
- **Feature:** The high-level capability or module to which the user story belongs.
- **USID:** A distinct identifier for each individual user story.
- **User Story:** A concise description of an actor's need or goal.
- **Priority:** The relative importance or urgency for delivering the user story.
- **Complexity:** An effort estimate reflecting the relative difficulty of implementation.

Table 5 Product Backlog

ID	Feature	USID	User Story	Priority	Complexity
U1	Login	US1	As a user, I want to log in to my account so that I view and operate resources that I'm authorized to access based on my role.	High	5
		US1.2	As a user, I want to stay logged in to my account so that I don't have to login again.	High	3
U2	Logout	US2	As a user, I want to log out from my account so that no one else can access my information.	High	1
UM3	Users' management	US3.1	As an administrator, I want to create new users.	High	5
		US3.2	As an admin, I want to update user.	Low	2
		US3.3	As an admin, I want to delete user.	Low	2
AM	Own account management	US4.1	As a user, I want to update my password, email and information so that I can maintain secure and up-to-date access to my account.	Low	1
	Forget password	US4.2	As a user, I want to reset my password if I forget it so that I can regain access to my account.	Low	3
TT1	Timetable management	US5.1	As an admin, I want to upload an Excel file containing timetables for professors or classes so that the data is automatically imported into the database.	High	8
		US5.2	As an admin, I want to update timetable so that I can keep schedule data accurate.	Medium	3
		US5.3	As an admin, I want to reset timetable so that I can keep schedule data accurate	Medium	3
AA1	Assign attendance status	US6	As an administrator agent, I want to assign attendance statuses so that I can record which lessons or	High	5

			academic events have been attended or missed via mobile app.		
AA2	Generate daily attendance report	US7	As an administrator agent, I want to generate attendance reports based on a specific date or period so that I can provide reports later via mobile app.	High	8
AA3	Attendance status management	US8	As an admin, I want to view the history of all attendance statuses and updates so that I can correct any mistakes if they occur.	Medium	2
DB	Dashboard viewing	US9	As a supervisor, I want to view statistics on the university's courses progress status.	High	3
		US10	As a head of department, I want to view statistics on my department's progress.	Medium	2
		US11	As a professor, I want to view my personal progress statistics.	Medium	1
SM1	Semester management	US12.1	As an admin, I want to add a semester so that it can be included in the system.	High	8
		US12.2	As an admin, I want to update the semester period dates so that any necessary changes can be reflected.	Medium	5
		US12.3	As an admin, I want to delete a semester so that mistakenly created semesters can be removed.	Low	2
		US12.4	As an admin, I want to update the semester status from active to completed so that I can create a new semester when the previous one ends.	High	3
SM2	Semester history viewing	US13	As a supervisor, I want to view historical statistics from previous semesters.	Low	8
CS	Compare statistics	US14	As a supervisor, I want to compare the progress of two professors' courses so that I can maintain harmony before exams and ensure alignment between classes.	Medium	2
		US14.2	As a supervisor, I want to compare statistics between semesters so that I can evaluate whether the current semester has improved or declined compared to previous ones.	Medium	3
		US15	As a head of department, I want to compare the progress of two	Medium	2

			professors in my department so that I can assess performance and alignment.		
		US16	As a professor, I want to compare my personal progress with that of my classes and courses so that I can stay aligned with my colleagues.	Medium	2
DM	Departments management	US17	As an admin, I want to manage departments (view, add, delete, update) so that department information remains accurate.	Low	2
CM	Classes management	US18	As an admin, I want to manage classes (view, add, delete, update).	Low	2
CRM1	Classrooms management	US19	As an admin, I want to manage classrooms (view, add, delete, update) so that classroom data remains accurate.	Low	2
CRM2	Classroom statistics viewing	US20	As a supervisor, I want to view statistics about classroom usage and overuse so that I can make informed decisions about classroom allocation.	Medium	3
COM	Courses management	US21	As an admin, I want to manage courses (view, add, delete, update).	Low	2
HM	Holiday management	US22	As an admin, I want to manage holiday periods and dates (view, add, update, delete).	Medium	8
AE1	Academic events management	US23	As an admin, I want to manage academic events (view, add, update, delete) and set their dates.	High	8
NOT	Notifications	US24	As an administrator agent, I want to receive a notification before session starts.	Low	3
		US25	As an administrator agent, I want to receive a notification when session is about to end and some attendance still pending needs to be assigned.	Low	3
NAV	Navigation	US26	As a user, I want to view the sidebar with my accessible options so that I can navigate the things I can use on the website.	High	2
S	Session management	US27	As an admin, I want to manage session (view, add, delete, update) so that sessions data remains accurate.	Low	2
R	Role management	US28.1	As admin, I want to view all roles, so that I know which roles we have	High	3

		US28.2	As admin, I want to add new role, so that I keep add new roles keep roles updated all the time.	High	3
		US28.3	As admin, I want to delete role, so that I can delete unwanted roles	High	3
		US28.4	As admin, I want to update role, so that I can update roles.	Hight	3

6.2. Sprint planning

By estimating each user story with Fibonacci numbers [24], we ensured a balanced distribution of work across three three-week sprints. Grouping stories by their cumulative complexity scores kept each sprint focused and achievable:

- **Sprint 1 (36 points):** Established the foundational infrastructure, including user management, authentication and authorization mechanisms and core navigation features
- **Sprint 2 (45 points):** Develop key functionality import and update timetables, manage semesters, and enable attendance assignment in the mobile app.
- **Sprint 3 (26 points):** Deliver analytics and final features integrate Power BI dashboards, comparison tools.
- **Sprint 4 (37 points):** which is finalizing all missing features such as design for app administrative settings, PDF reporting exports, and rescheduling workflows.

This three-sprint roadmap establishes clear milestones across the development timeline, allocating sufficient time for implementation, testing, and review while ensuring a consistent, predictable delivery pace.

Table 6 Sprint planning

Sprint	Focus Area	User Story ID	Complexity Points
1	User management, auth & web design and nav foundations	US1, US1.2, US2, US3.1, US3.2, US3.3, US4.1, US4.2, US26, US29.1, US29.2, US29.3, US29.4	36
2	Core data flows: timetables, semesters & attendance assignment	US5.1, US5.2, US5.3, US12.1, US12.2, US12.3, US12.4, US6, US8, US24, US25	45
3	Reporting & analytics: dashboards,	US9, US10, US11, US13, US14, US14.2, US15, US16, US20	26
4	Pdf Reports, rescheduling, admin settings management	US17, US18, US19, US21, US22, US23, US7, US27	37

7. Conclusion

A thorough analysis and clear specification of both functional and non-functional requirements have provided a robust foundation for our academic monitoring system. By defining and mapping out the hybrid deployment architecture, we have established a coherent blueprint that will drive the detailed design and implementation phases. This disciplined approach ensures that each component integrates smoothly, meets performance and security targets, and supports scalable, maintainable growth as we deliver the final solution.

Chapter 3: Business Intelligence Concepts and Implementation

1. Introduction

This chapter provides an overview of the BI concepts integrated into our system. It outlines the fundamental principles of data modelling, including common schema designs, and discusses data integration methodologies such as ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform). Additionally, it highlights the role of BI tools in enhancing data analysis and reporting capabilities within the system.

2. Understanding Business Intelligence

Business intelligence [25] analysts transform raw data into meaningful insights that drive strategic decision-making within an organization. BI tools enable business users to access different types of data, historical and current, third-party and in-house, as well as semi structured data and unstructured data such as social media. Users can analyse this information to gain insights into how the business is performing and what it should do next.

3. Data Marts

3.1. Definition

Data Marts [26] are a subset of a data warehouse that is tailored to the needs of a particular business unit or department. Data marts often contain data important to a specific business area, such as sales, marketing, or finance. Data marts are frequently used to improve decision-making inside an organisation.

Reasons for creating a data mart

- Easy access to often needed data
- Easy to implement
- Lower cost than establishing a full data warehouse
- Potential customers are better defined than in a full data warehouse.
- It is less crowded and contains only necessary business data.

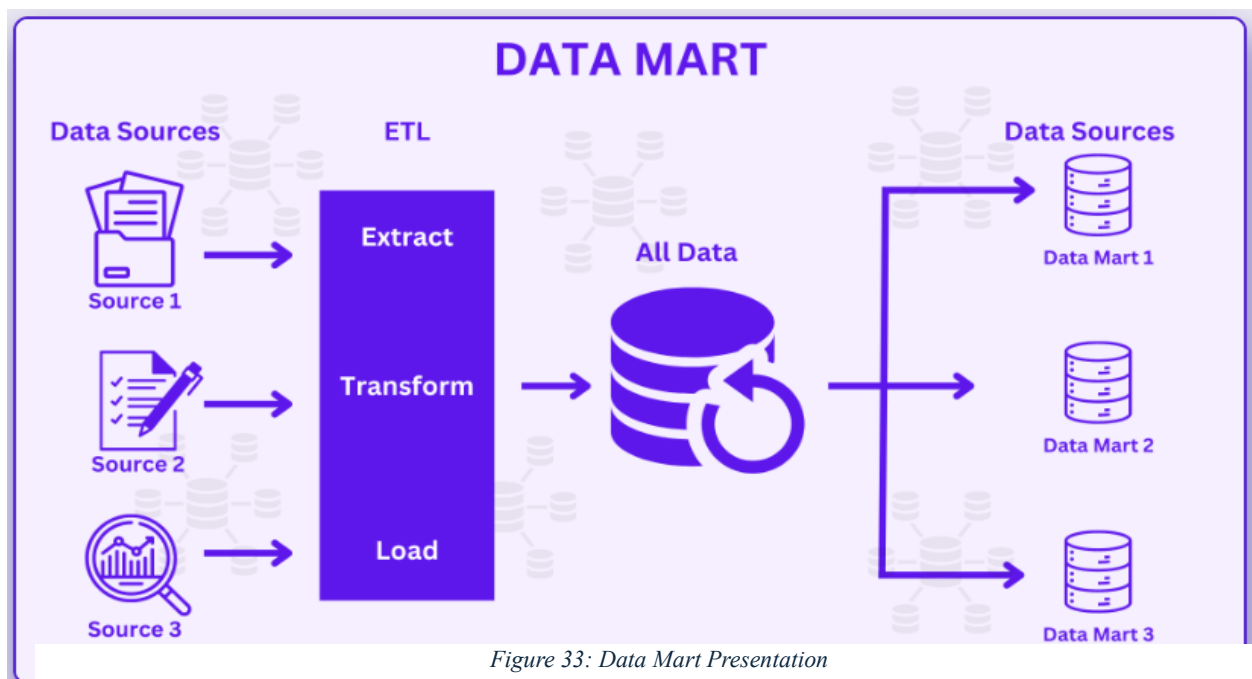


Figure 33: Data Mart Presentation

There are mainly two approaches of designing a data mart. These approaches are:

Dependent Data Marts: Dependent data marts are logical subsets of physical subsets of a larger data warehouse. The data marts are considered as subsets of a data warehouse in this technique. This technique begins with the creation of a data warehouse, from which other data marts can be produced. These data marts rely on the data warehouse and retrieve the necessary records from it. There is no requirement for data mart integration in this technique because the data warehouse builds the data mart. It is also referred to as a **top-down strategy**.

Independent Data Marts: The second method is independent data marts (IDM). In this case, independent data marts are built initially, followed by the construction of a data warehouse based on these independent multiple data marts. Because each data mart is developed independently in this method, data mart integration is essential. It is also known as a **bottom-up strategy** since data marts are combined to create a data warehouse.

3.2. Fact Tables

Fact tables store quantitative data about business processes or events, such as sales transactions or customer interactions. They serve as the core repository of business performance data, structured to support aggregation and numerical analysis.

3.3. Dimension Tables

Dimension tables provide descriptive attributes related to the facts, such as customer demographics, product details, or time periods. These tables offer context for the measures recorded in the fact table, enabling users to analyse different aspects of their business.

4. Data Modelling in BI

Effective data modelling is crucial for organizing data in a manner that supports efficient analysis and reporting. Two common schema designs used in data warehousing are the Star Schema and the Snowflake Schema.

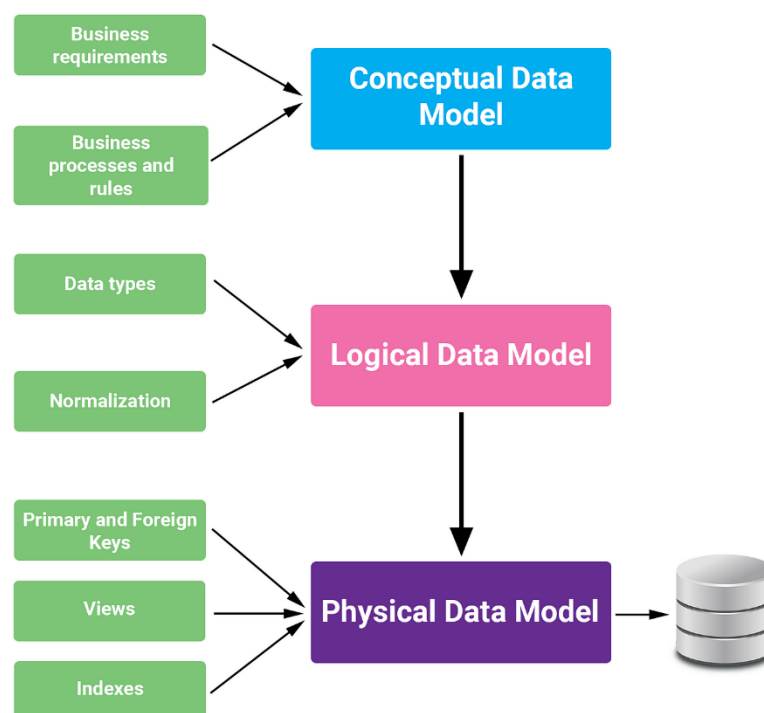


Figure 34: The basics of data modelling in business intelligence

4.1. Star Schema

The Star Schema is characterized by a central fact table that contains quantitative data, surrounded by dimension tables that provide descriptive attributes related to the facts. This design simplifies complex queries and enhances performance, making it suitable for straightforward analytical needs.

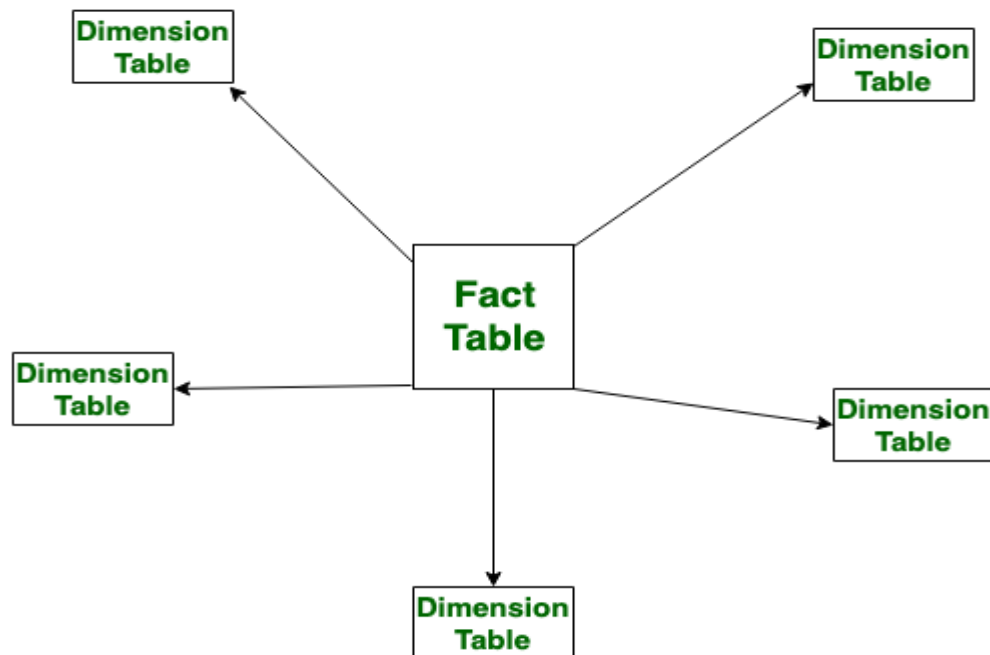


Figure 35: Star schema model

4.2. Snowflake Schema

The Snowflake Schema is a more normalized form of the Star Schema, where dimension tables are further divided into sub-dimensions. This design reduces data redundancy and can lead to improved data integrity. However, it may introduce complexity in query design and performance.

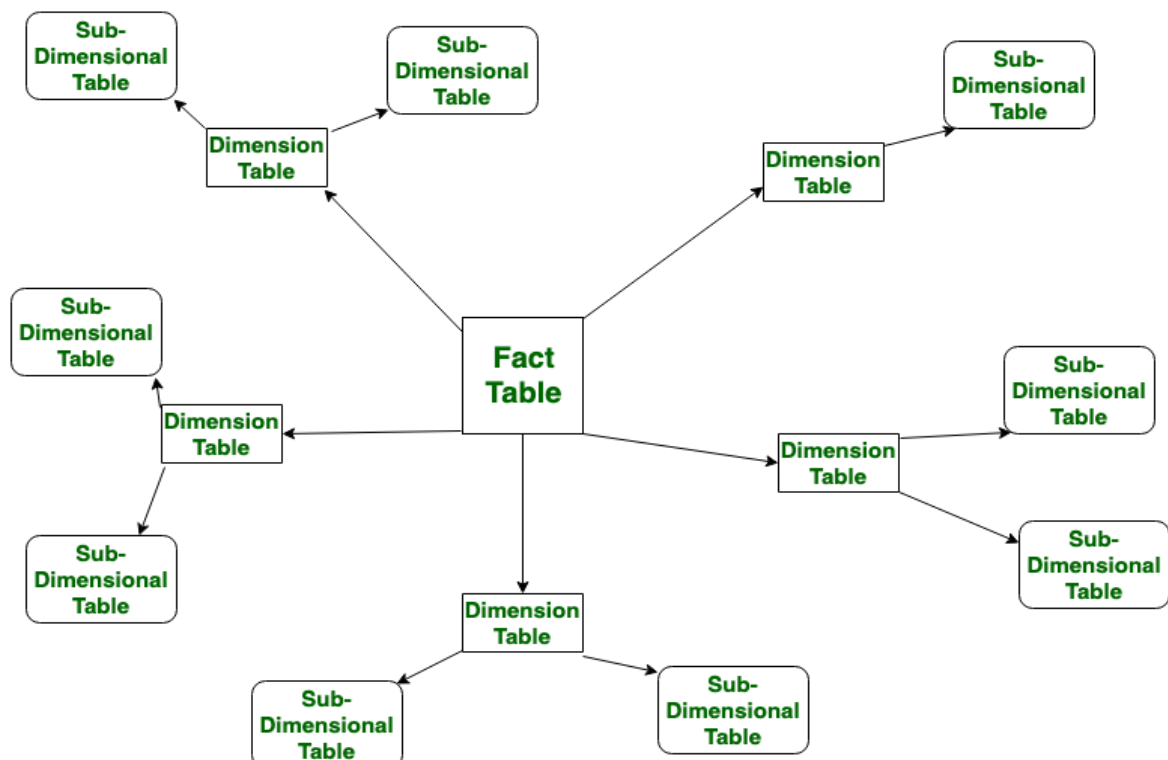


Figure 36: Snowflakes schema model

5. ETL (Extract, Transform, Load)

Extract, transform, and load (ETL) [27] is the process of combining data from multiple sources into a large, central repository called a data warehouse. ETL uses a set of business rules to clean and organize raw data and prepare it for storage, data analytics, and machine learning (ML).

5.1. Extract

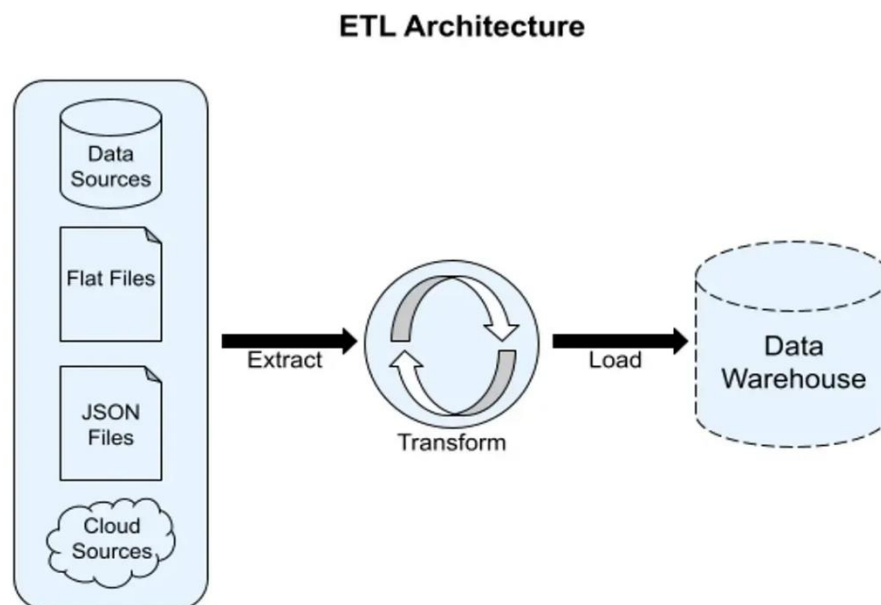
In the extraction phase, data is retrieved from diverse source systems, which may include relational databases, cloud-based applications, flat files, APIs, and more. The objective is to collect all relevant data, regardless of its format or structure, and bring it into a staging area for further processing. This step ensures that data from multiple sources is available for transformation and analysis.

5.2. Transform

The transformation phase involves converting the extracted data into a consistent and usable format. This process includes tasks such as data cleansing (removing inaccuracies and inconsistencies), data standardization (ensuring uniform formats), and data enrichment (enhancing data with additional information). Transformations may also involve applying business rules, aggregating data, and resolving data quality issues to ensure the data meets the requirements of the target system.

5.3. Load

In the loading phase, the transformed data is transferred into the target system, such as a data warehouse, data lake, or another data repository. This step ensures that the data is structured and stored in a manner that supports efficient querying and analysis. Loading can be performed in batches or in real-time, depending on the organization's needs and the capabilities of the target system.



6. Conclusion

Integrating Business Intelligence into our system establishes a foundation for data-driven decision-making. By employing effective data modelling techniques and robust data integration processes, we ensure that stakeholders have access to accurate and timely information. The use of BI tools further empowers users to explore data, uncover insights, and drive strategic initiatives. Subsequent sections will delve into the specific implementation details and outcomes of our BI integration efforts.

Chapter 4: Sprint 1 User management, Role management, authentication, web design and navigation foundations

1. Introduction

This chapter outlines the foundational elements developed during Sprint 1, focusing on establishing user management and authentication mechanisms, as well as laying the groundwork for web design and navigation. These components are crucial for ensuring secure user interactions and an intuitive user experience.

2. Specification of requirements

In this section, we will present our sprint backlog as well as the use case diagram and the presentation of descriptions of some user stories.

2.1. Sprint Backlog

The sprint 1 Backlog will encompass the user stories defined in Table 7. Each user story is composed of four steps:

1. Design the interface.
2. Implement the functionality in the backend.
3. Integrate the frontend with the backend.
4. Test the feature.

In each sprint backlog, the tasks of each user story will not be detailed. Instead, only the user stories for the sprint and their complexities will be listed.

Table 7 Sprint 1 backlog

ID	User Story	Priority	Complexity
1	As a user, I want to log in and, based on my role, see different views and access different resource ensuring that only authorized users can view or modify specific content.	High	5
1.2	As a user, I want to stay logged in to my account so that I don't have to login again.	High	3
2	As a user, I want to log out from my account so that no one else can access my information.	High	1
3.1	As an admin, I want to create new users account so that academic staff and professors can login.	High	5
3.2	As an admin, I want to update user account.	Low	2
3.3	As an admin, I want to delete user account.	Low	2
4.1	As a user, I want to update my password and email so that I can maintain secure and up-to-date access to my account.	Low	1
4.2	As a user, I want to reset my password if I forget it so that I can regain access to my account.	Low	3
26	As a user, I want to view the sidebar with my accessible options so that I can navigate the things I can use on the website.	High	2
29.1	As admin, I want to view all roles, so that I know which roles we have in the system.	High	3

US29.2	As admin, I want to add new role, so that I keep add new roles keep roles updated all the time.	High	3
US29.3	As admin, I want to delete role, so that I can delete unwanted roles.	High	3
US29.4	As admin, I want to update role, so that I can update roles.	High	3

2.2. Use case diagram

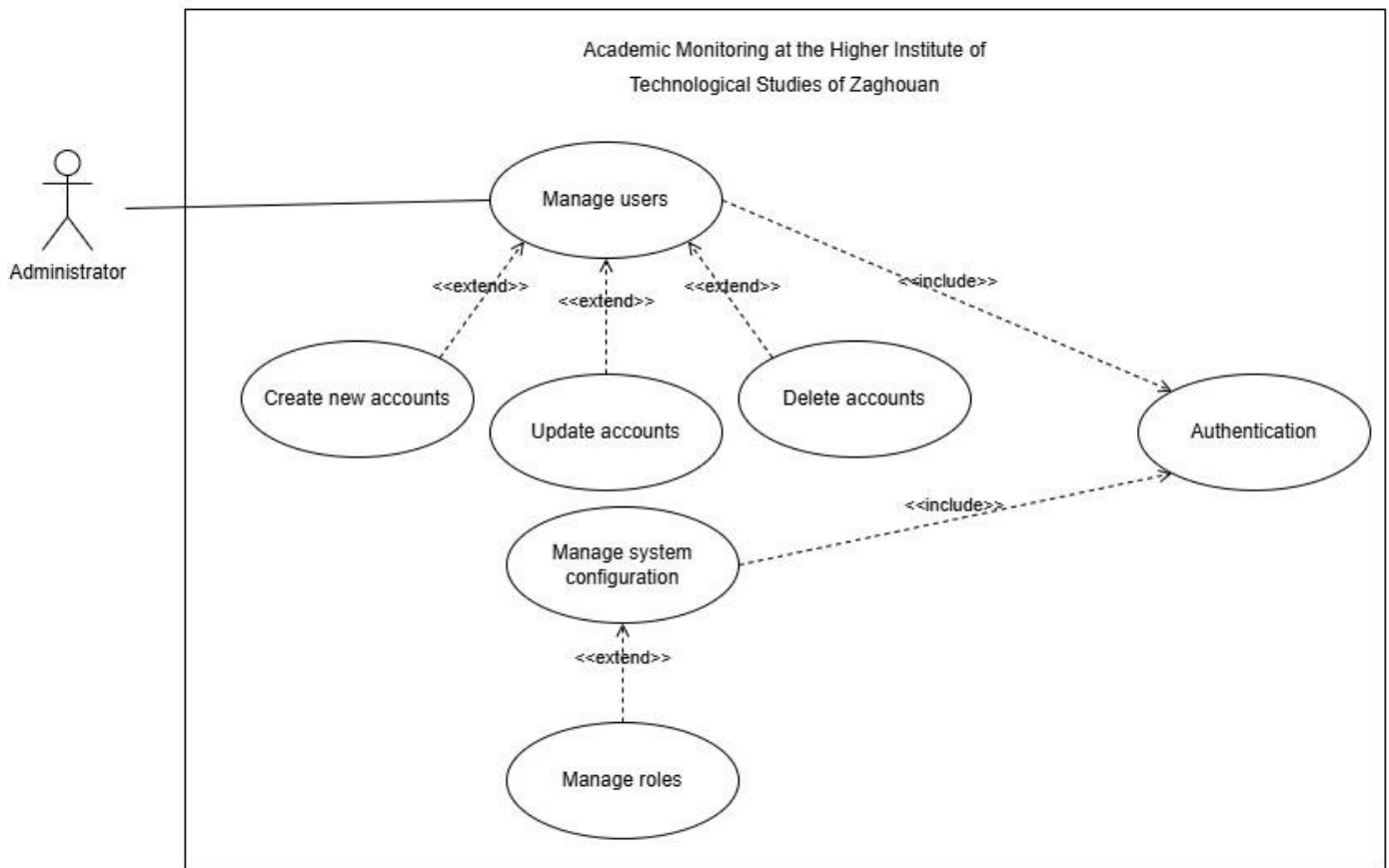


Figure 38: Administrator sprint 1 use case

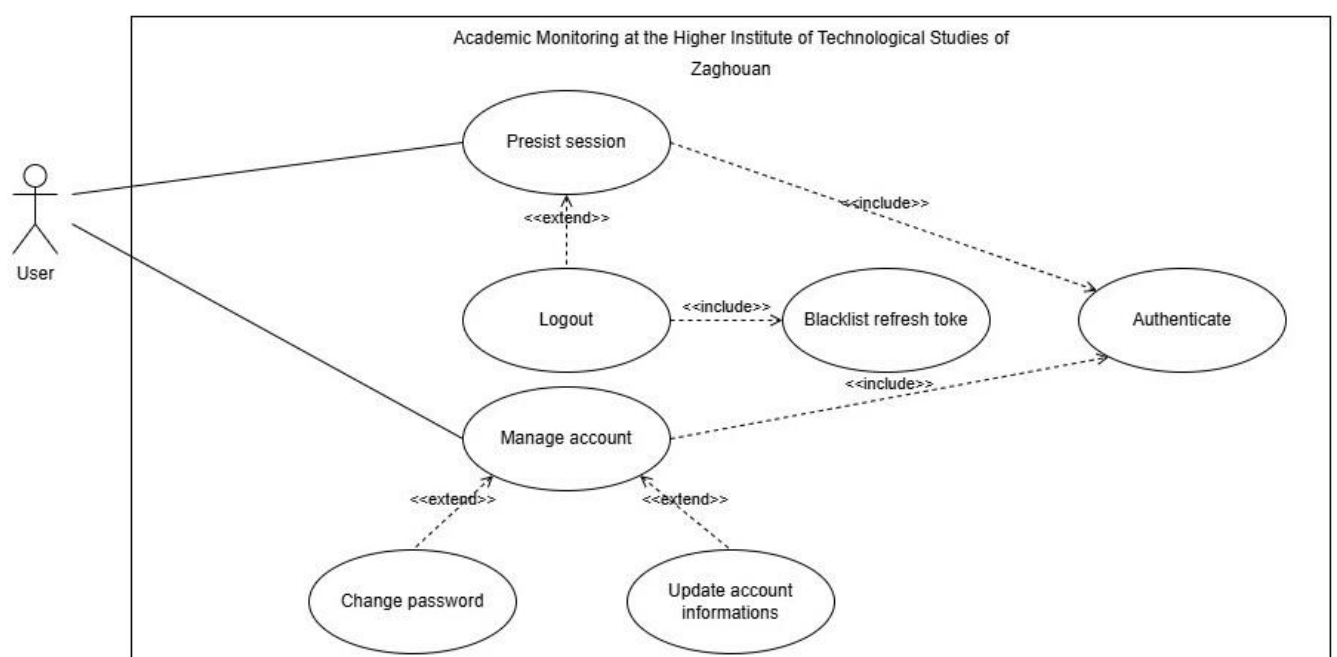


Figure 39: User sprint 1 use case

3. Design

3.1. Class diagram

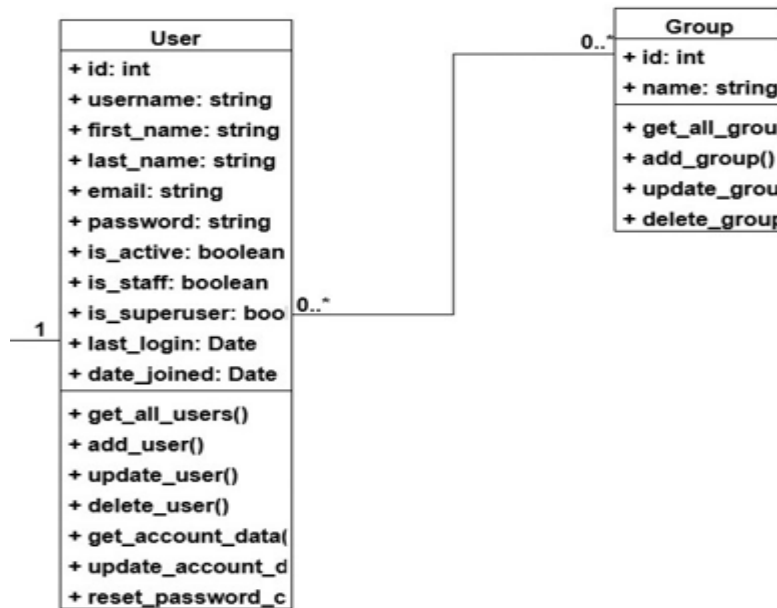


Figure 40: Class diagram sprint 1

3.2. Sequence diagrams and textual description of use cases

3.2.1. Authentication and Logout use case

Figure 39 and Figure 40 illustrates the “Authenticate” and “Logout” sequence architecture. When a user submits their credentials via the web or mobile frontend, the frontend sends a POST `api/auth/login` request to the Django REST API. The API verifies credentials against the database using Django’s ORM. If validation succeeds, SimpleJWT generates an access token and a refresh token, which the API returns (200 OK) to the frontend. The frontend then stores both tokens and redirects the user to the homepage. If credential validation fails, the API responds with 401 Unauthorized, and the frontend displays an error message and if user wants to logout the refresh token will be blacklisted for securities measurements.

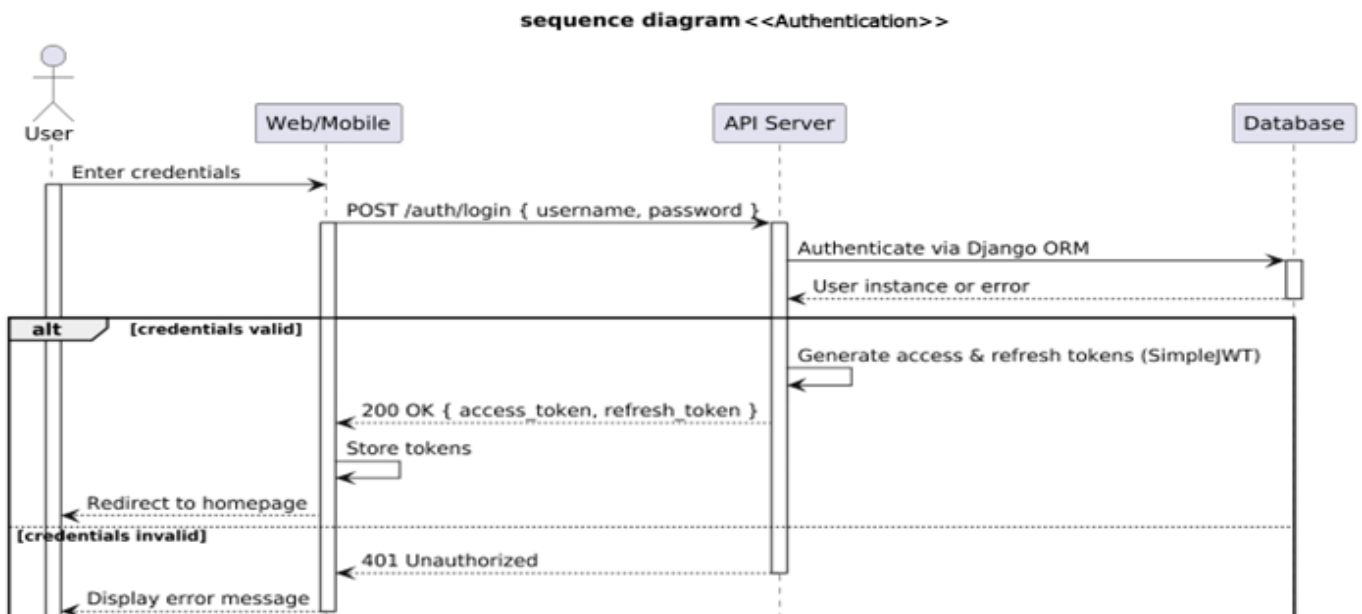


Figure 41: Authentication sequence diagram

sequence diagram «logout»

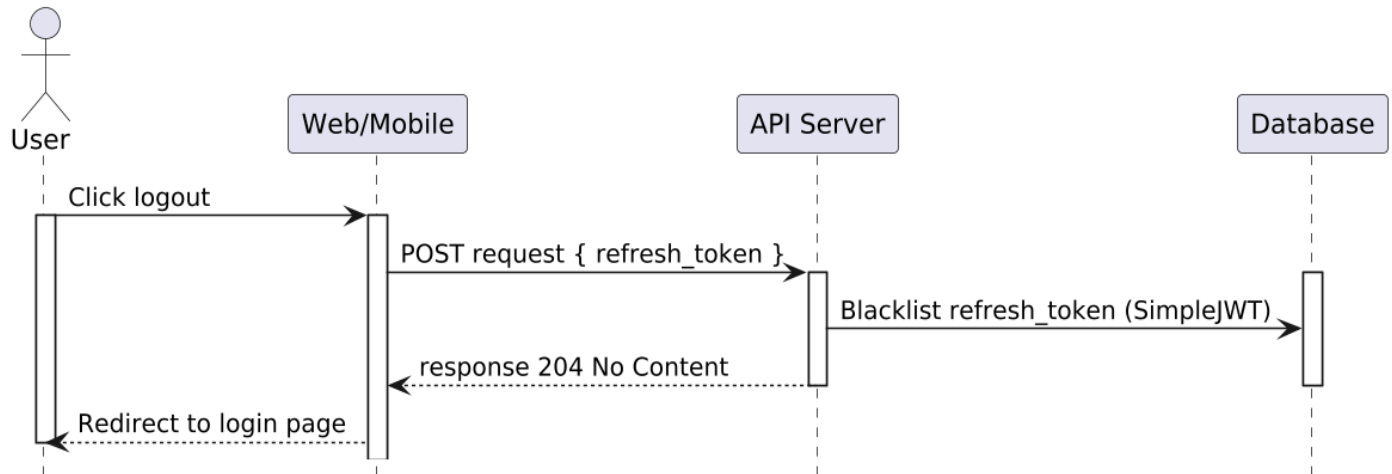


Figure 42: Logout sequence diagram

Table 8 Analysis authentication and logout use cases

Use Case: Authentication	
Title	Authentication, logout
Actor	User
Objective	Allow users to securely log into the system and initiate a session.
Description	Enable authorized users to access web/mobile platform via authentication and logout.
Pre-condition	User has an account and is on the login interface or in homepage.
Post-condition	User is successfully logged in and granted access to their account or logged out.
Nominal Scenario	
1. User navigates to the login page 2. User enters a valid email and password. 3. System receives the credentials 4. System validates the credentials 5. System returns an access token with refresh token. 6. Frontend either mobile or web stores tokens. 7. Frontend redirect user to homepage. 8. User wants to logout 9. Frontend send request to blacklist the refresh token 10. Backend blacklisted (simpleJWT) the token 11. User redirected to login page	
Alternative Scenario	
A1: Invalid credentials 2. System detects invalid credentials. 3. Api returns an error message. 4. View displays an authentication error to the user.	

3.2.2. Reset password use case

Figure 41 and 42 summarize the “Forgot Password → Reset Password” flow in our hybrid architecture. When a user requests a reset, the frontend posts their email to POST /auth/password-reset-request. The API uses Django’s ORM to check for an account but always enqueues a password_reset_task onto Redis (even if no account exists) to prevent revealing user data. A Celery worker processes this task, sending a reset link by email if the address is valid; otherwise, it quietly drops the task. The API immediately returns “200 OK” so the user cannot infer whether the email was found. When the user clicks the reset link, the frontend POSTs the reset_token to POST /auth/verify-reset-token. The API validates the token against the database—200 OK if valid (display the new-password form) or 400 if invalid (show “Token invalid or expired”). Finally, submitting the new password (POST /auth/password-reset) prompts

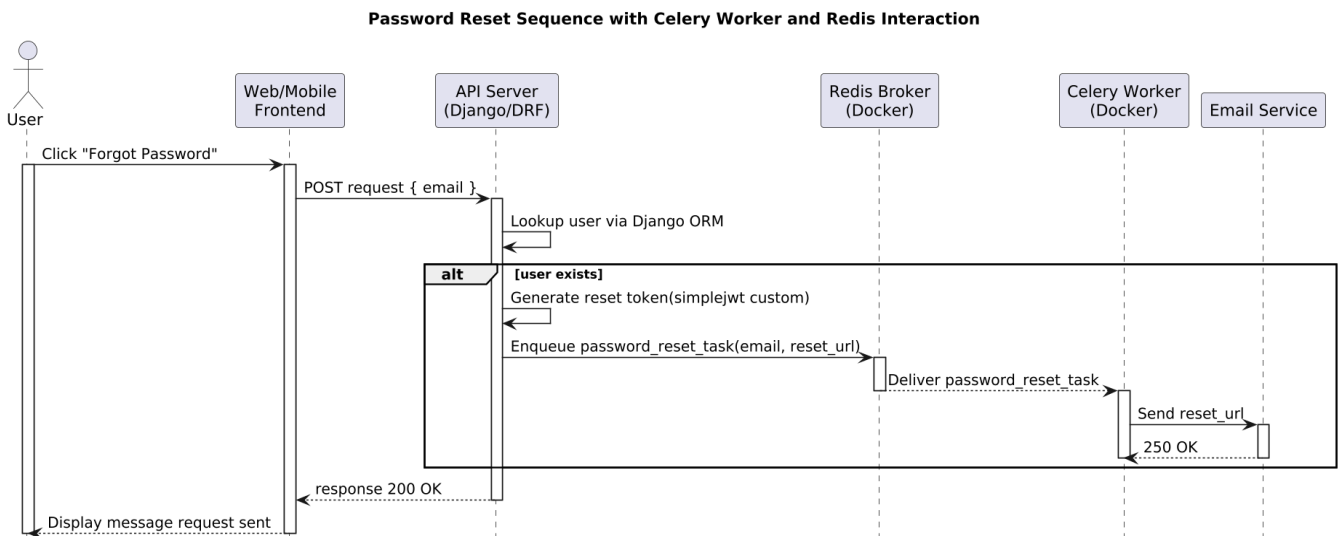


Figure 44: Request reset link sequence diagram

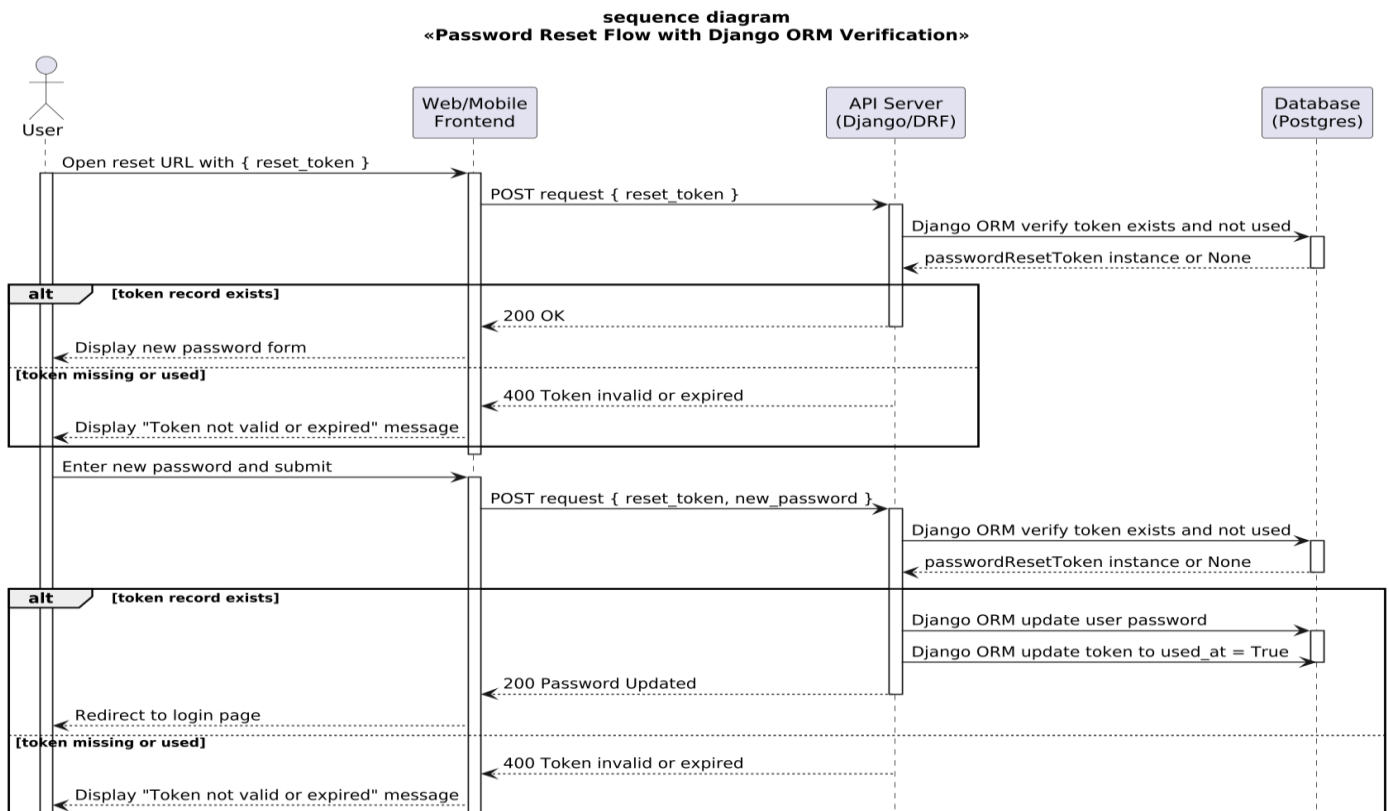


Figure 43: confirms reset new password sequence diagram

the API to re-verify the token, update the password via Django ORM, mark the token as used, and return 200 OK. On success, the frontend redirects to login.

Table 9 Analysis reset password use case

Use Case: Reset password	
Title	Reset password
Actor	User
Objective	Allow users to regain access to their accounts in case they forget password
Description	Enable update account password using their email identifier and receive reset link.
Pre-condition	User without access since he forgot his password.
Post-condition	User is successfully logged in and granted access to their account using new password.
Nominal Scenario	
1. User navigates to the forgot password form 2. User enter their email. 3. System receives the email 4. System validates the email belongs to an existing account 5. System returns success message that request is sent and link will be sent soon to his email. 6. User receives email with reset link. 7. Reset link will redirect user to new password form. 8. User fill form and confirm new password. 9. System updates user password. 10. User receive message success 11. User redirected to login page to logged in 12. User able to login to his account using new password	
Alternative Scenario	
A1: token is invalid or expired 2. System detects invalid token. 3. Api returns an error message that token is invalid.	

3.2.3. User management use case

User management involves securely handling the creation, update and deletion of user accounts within a system. When creating a new user, the system first checks whether the provided email address already exists in the database to prevent duplicate accounts. This validation ensures that each user has a unique identifier and helps maintain data integrity. Once verified, the user is created and stored in the database with their associated details. On the other hand, when deleting a user, the system must confirm that the user is not referenced elsewhere in the database as a foreign key. If the user is not linked to any other critical records they can be safely removed.

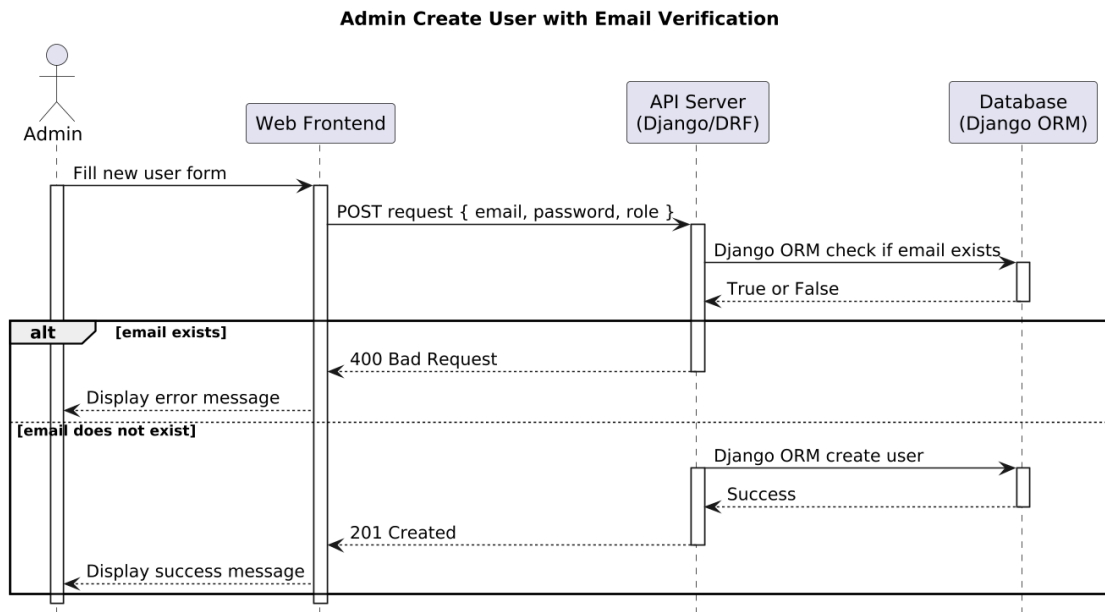


Figure 45: Create user sequence diagram

Table 10 Analysis creates user use case

Use Case: Manage user	
Title	Create user
Actor	Admin
Objective	Allow admin to create users and assign roles to them
Description	Enable admin to create users mostly for academic event account and professors to be used in timetables later
Pre-condition	No users
Post-condition	As many users created need it
Nominal Scenario	
1. Admin navigate to user management interface 2. Click add user 3. Fill form to add user 4. Click add receive success message	
Alternative Scenario	
A1: email already exist for another user	

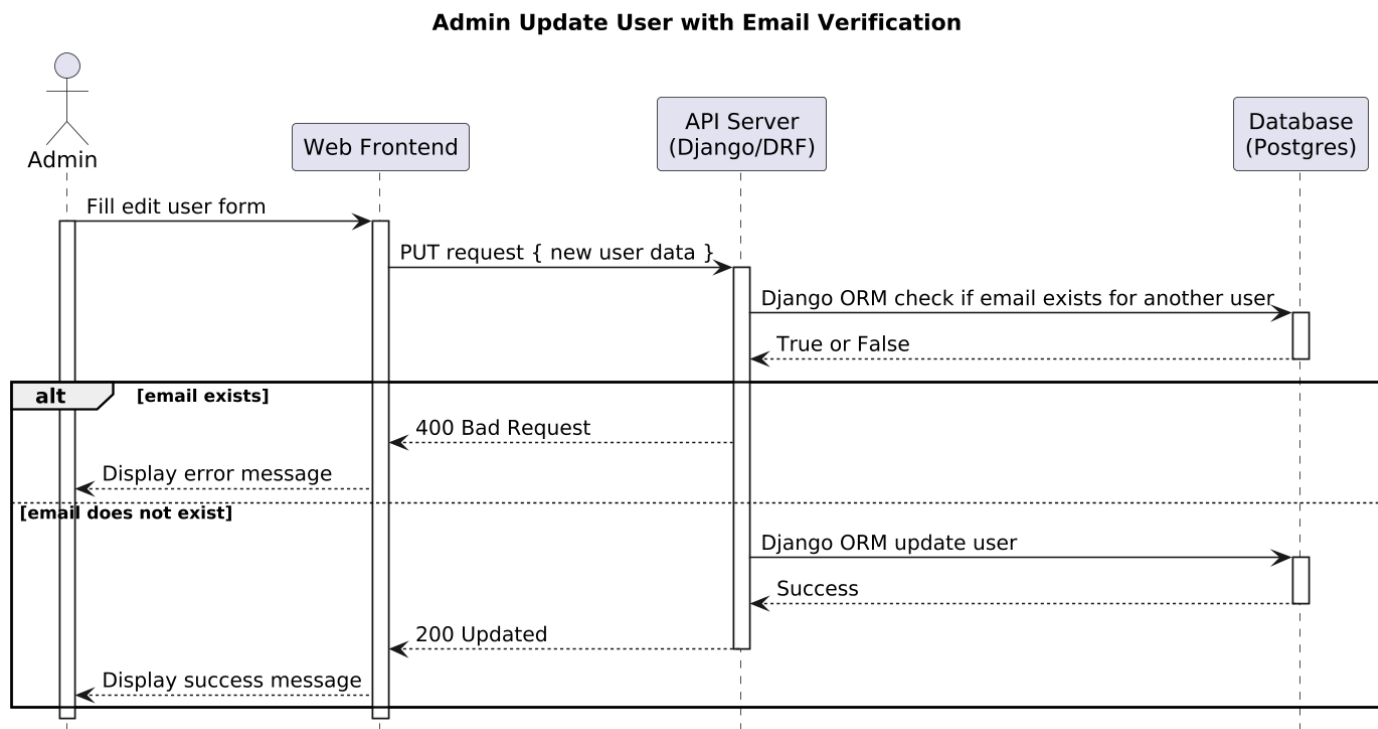


Figure 46: Update user sequence diagram

Table 11 Analysis update user use case

Use Case: Manage user	
Title	update user
Actor	Admin
Objective	Allow admin to update users
Description	Enable admin to update users to keep database up to date
Pre-condition	Users need updates
Post-condition	Users updated
Nominal Scenario	
- Admin navigate to user management interface - Click edit user - Fill form to edit user - Click edit receive success message	
Alternative Scenario	
A1: email already exist for another user	

sequence diagram admin delete user

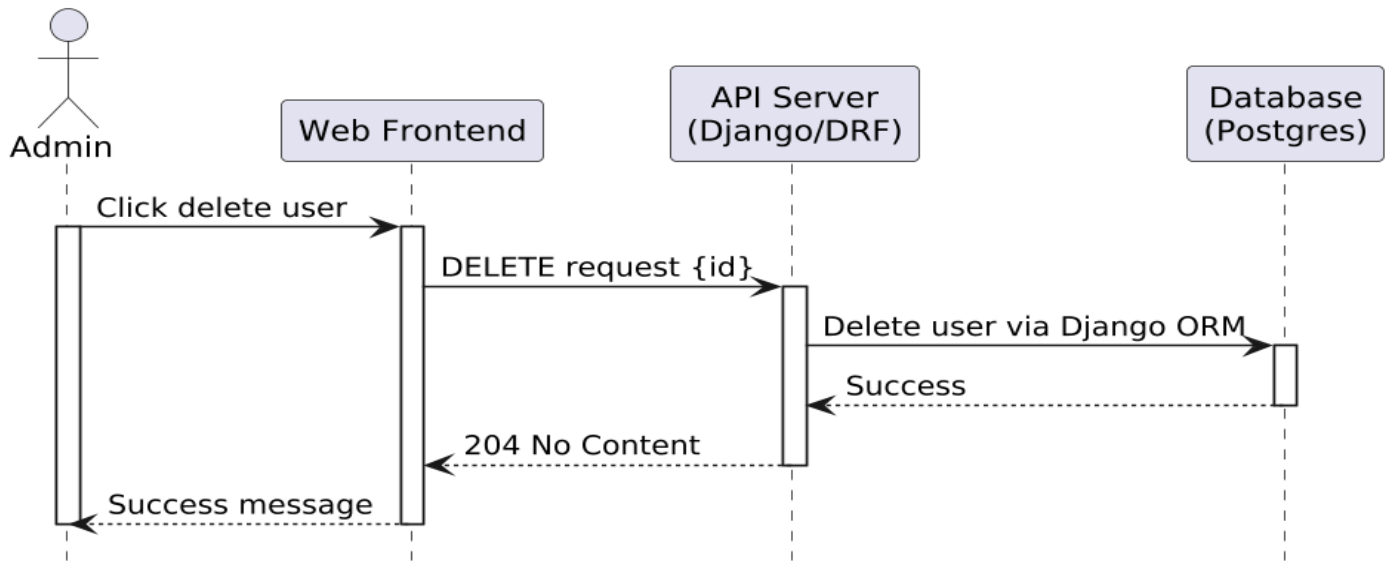


Figure 47: Delete user sequence diagram

Table 12 Analysis deletes user use case

Use Case: Manage user	
Title	delete user
Actor	Admin
Objective	Allow admin to delete users
Description	Enable admin to delete unwanted users to keep database.
Pre-condition	Unwanted user
Post-condition	Users deleted
Nominal Scenario	
1. Admin navigate to user management interface 2. Click delete user 3. Confirm delete 4. receive success message	
Alternative Scenario	
A1: Cannot delete user: protected by foreign key constraints.	

3.2.4. Role management use case

Role management ensures that each role within the system is unique and serves a specific purpose related to user permissions and access control. When creating a new role, the system first checks that the role name does not already exist in the database to prevent duplicates and maintain clarity in permission assignments. This uniqueness check helps avoid confusion and ensures that each role has a distinct identity and function. When deleting a role, the system verifies that it is not currently assigned to any user or

referenced elsewhere in the database. Only roles that are not in use can be safely deleted, ensuring that user access control remains consistent and that no orphaned references compromise the system’s integrity.

Admin Create Role with Existence Check

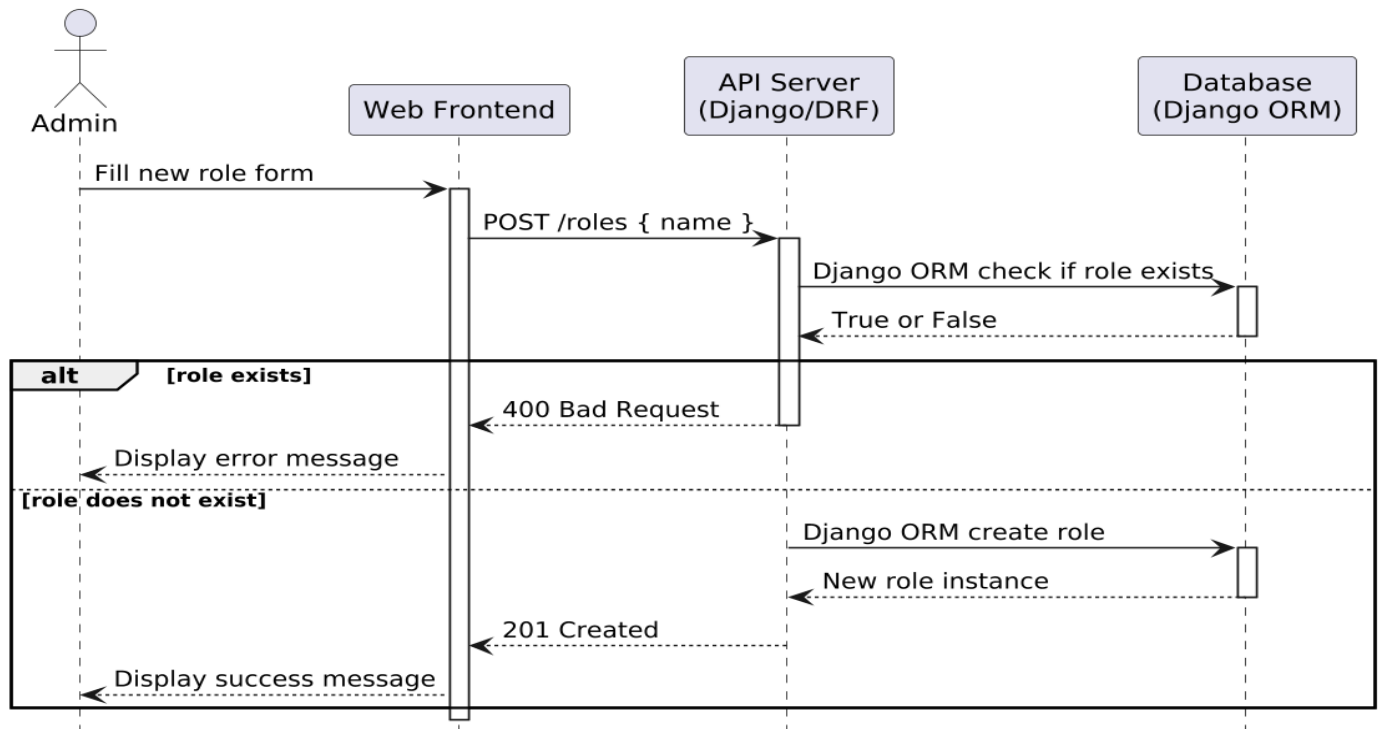


Figure 48: Create role sequence diagram

Table 13 Analysis create role use case

Use Case: Manage role	
Title	Create role
Actor	Admin
Objective	Allow admin to create roles.
Description	Enable admin to create roles to be able to categories users.
Pre-condition	No roles
Post-condition	As many roles created need it
Nominal Scenario	
1. Admin navigate to role management interface 2. Click add role 3. Fill form to add role 4. Click add receive success message	
Alternative Scenario	
A1: role already exists	

Admin Update Role with Existence Check

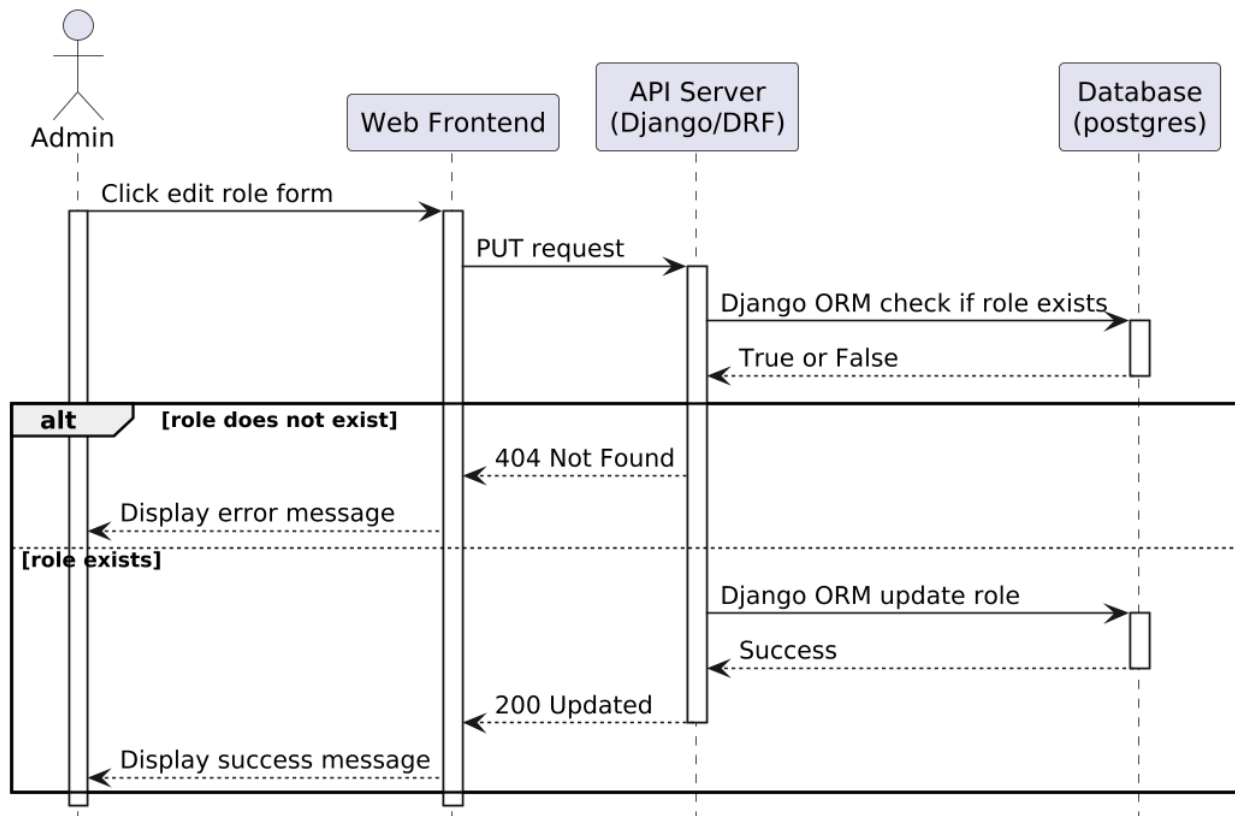


Figure 49: update role sequence diagram

Table 14 Analysis update role use case

Use Case: Manage role	
Title	Update role
Actor	Admin
Objective	Allow admin to update roles.
Description	Enable admin to update roles to be able to correct roles in case of mistake or changing of the naming.
Pre-condition	Role needs to be updated
Post-condition	Role updated
Nominal Scenario	
1. Admin navigate to role management interface 2. Click edit role 3. Fill form to edit role 4. Click edit receive success message	
Alternative Scenario	
A1: role already exists	

Admin Delete Role with Existence Check

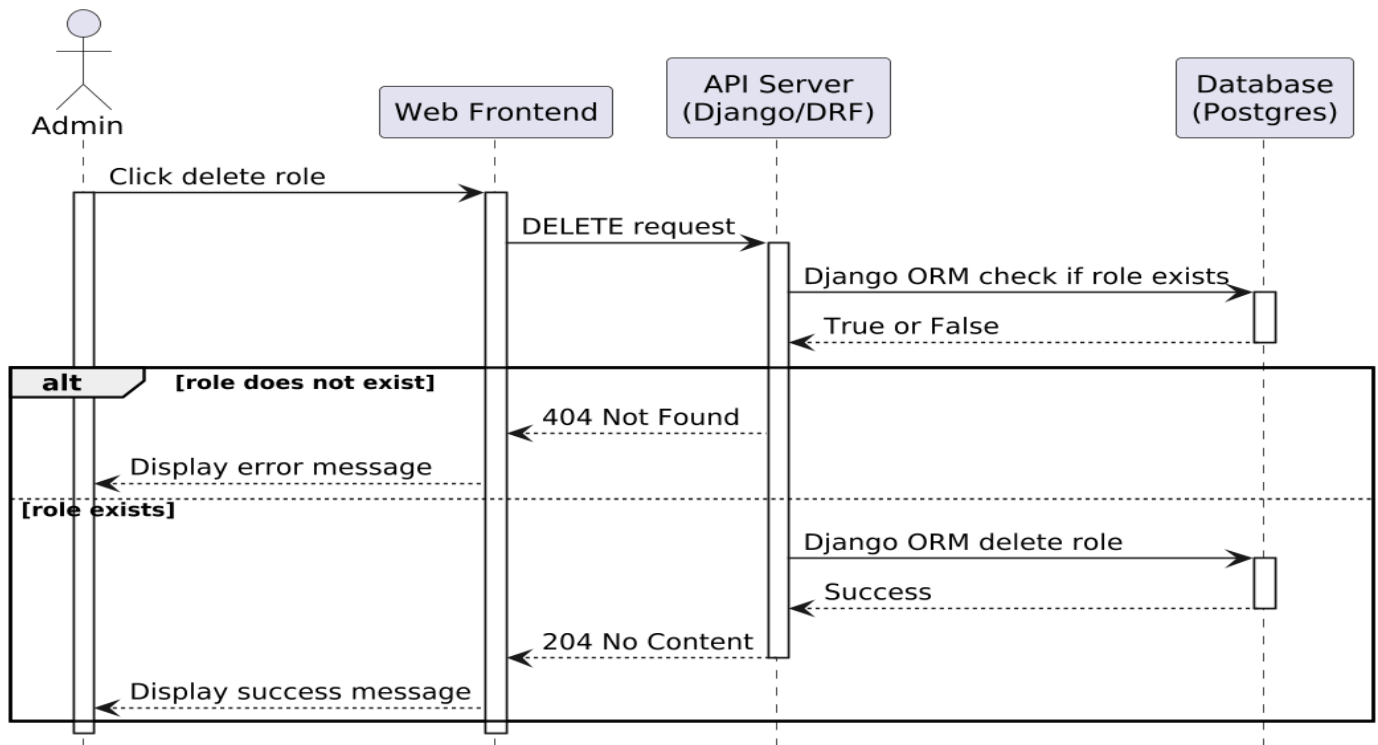


Figure 50: delete role sequence diagram

Table 15 Analysis delete role use case

Use Case: Manage role	
Title	delete role
Actor	Admin
Objective	Allow admin to delete roles
Description	Enable admin to delete unwanted roles to keep database.
Pre-condition	Unwanted role
Post-condition	role deleted
Nominal Scenario	
1. Admin navigate to role management interface 2. Click delete role 3. Confirm delete 4. receive success message	
Alternative Scenario	
A1: Cannot delete role: protected by foreign key constraints.	

3.2.5. Account management use case

Account management allows users to maintain and update their personal information securely. A user can modify key account details such as email, username, password, first name, and last name. When updating the email or username, the system performs a uniqueness check to ensure that the new values do not already exist in the database, maintaining data integrity and avoiding conflicts. Password updates

require verification of the current password to ensure security and prevent unauthorized changes. Changes to the first and last name are straightforward but may be logged for auditing purposes. This functionality empowers users to keep their account information accurate and up to date while preserving system security and consistency.

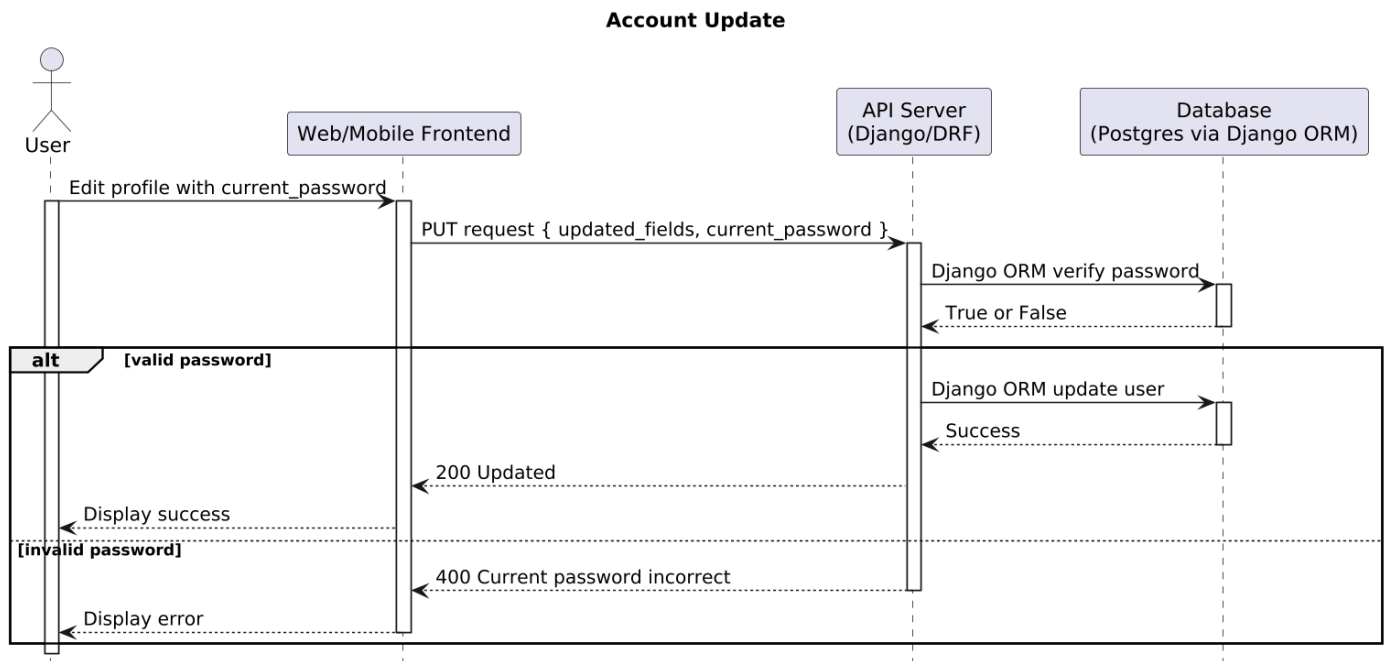


Figure 51: Account update sequence diagram

Table 16 Analysis manages account use case

Use Case: Manage account	
Title	update account
Actor	User
Objective	Allow user to update users
Description	Enable admin to update users to keep database up to date
Pre-condition	User needs updates to update his account data
Post-condition	User account updated
Nominal Scenario	
1. User navigate to account settings 2. Fill form with new changes 3. Fill his password 4. Click confirm 5. Receive success message	
Alternative Scenario	
A1: password incorrect 2: display error message	

4. Implementation

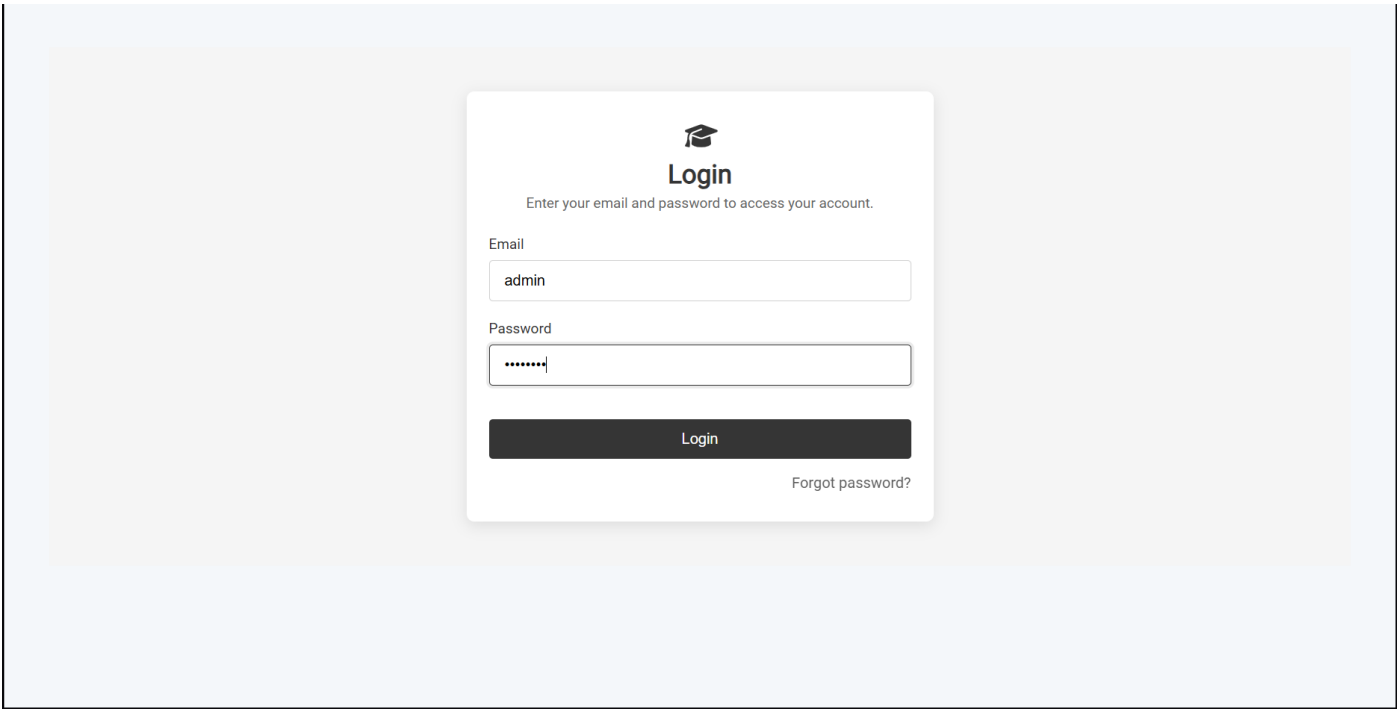


Figure 52: Login page

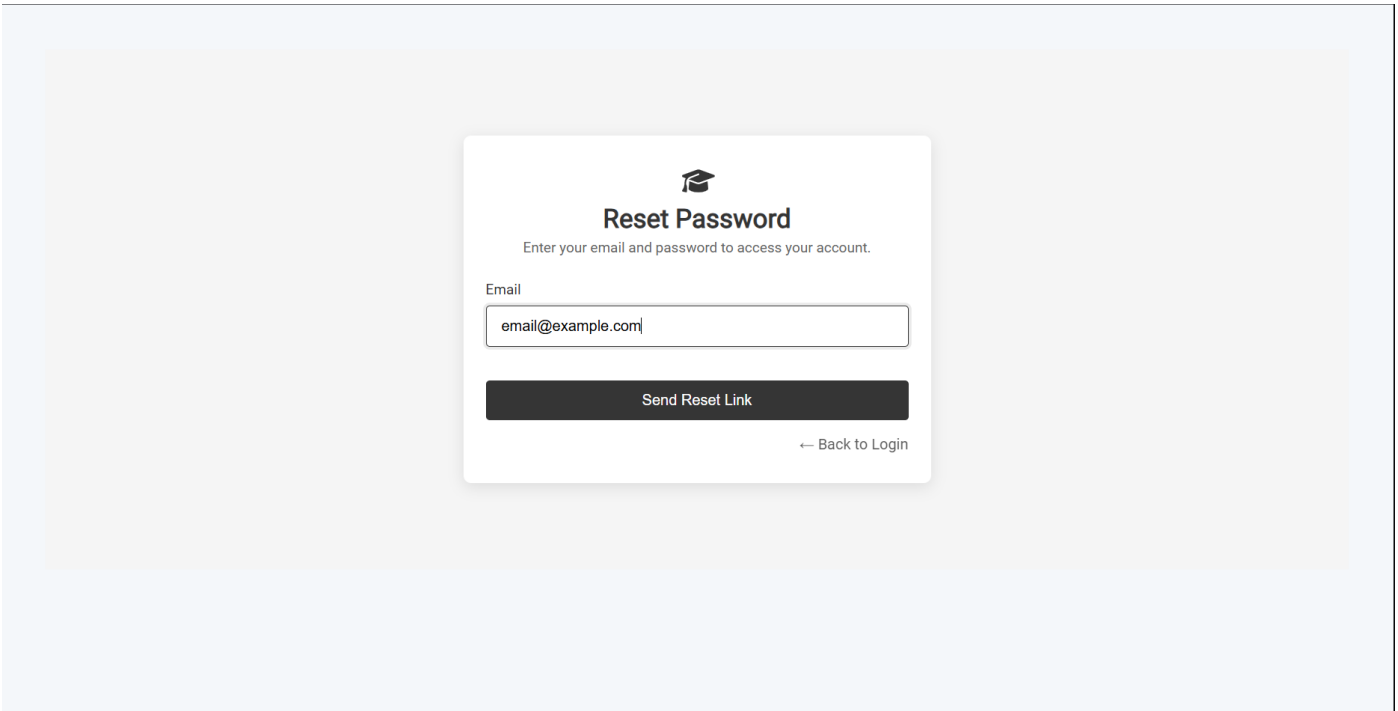


Figure 53: Reset password page

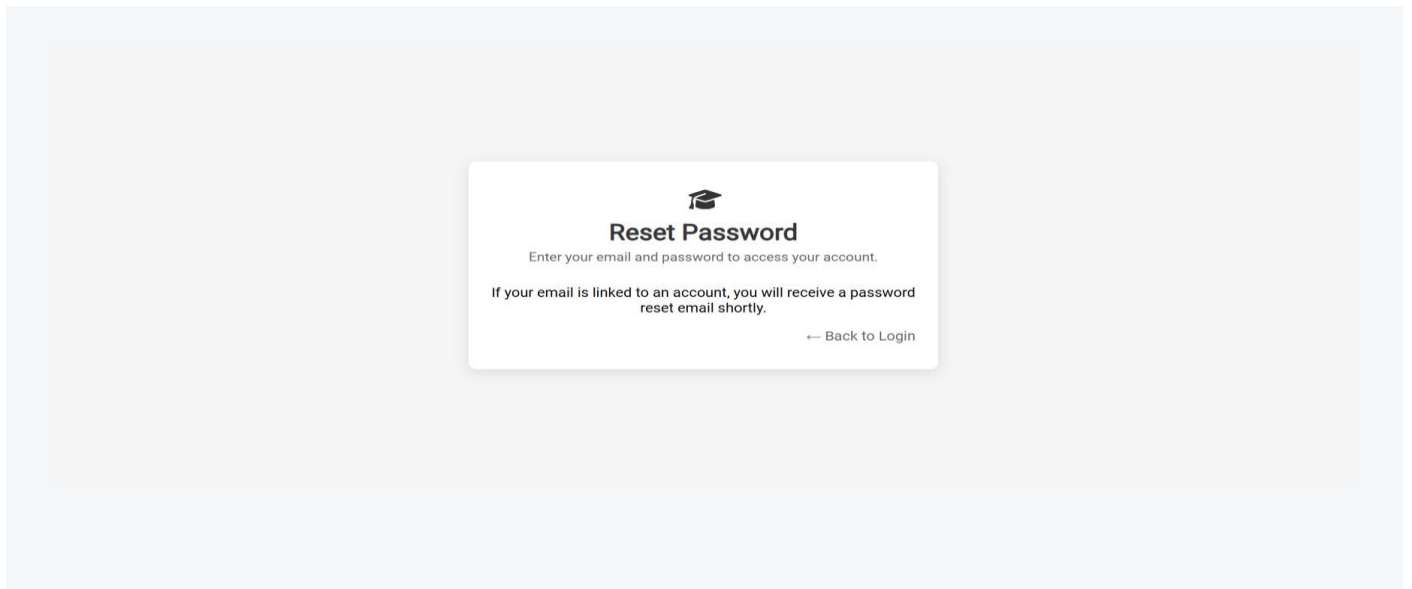


Figure 54: Message confirm reset password request was sent

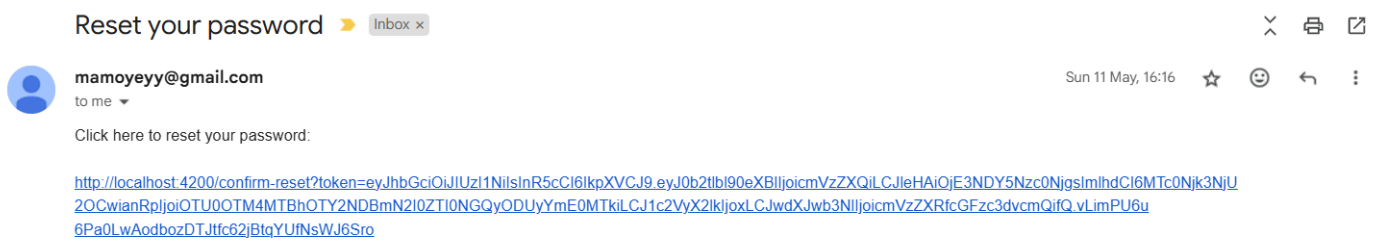


Figure 55: Email received with reset link

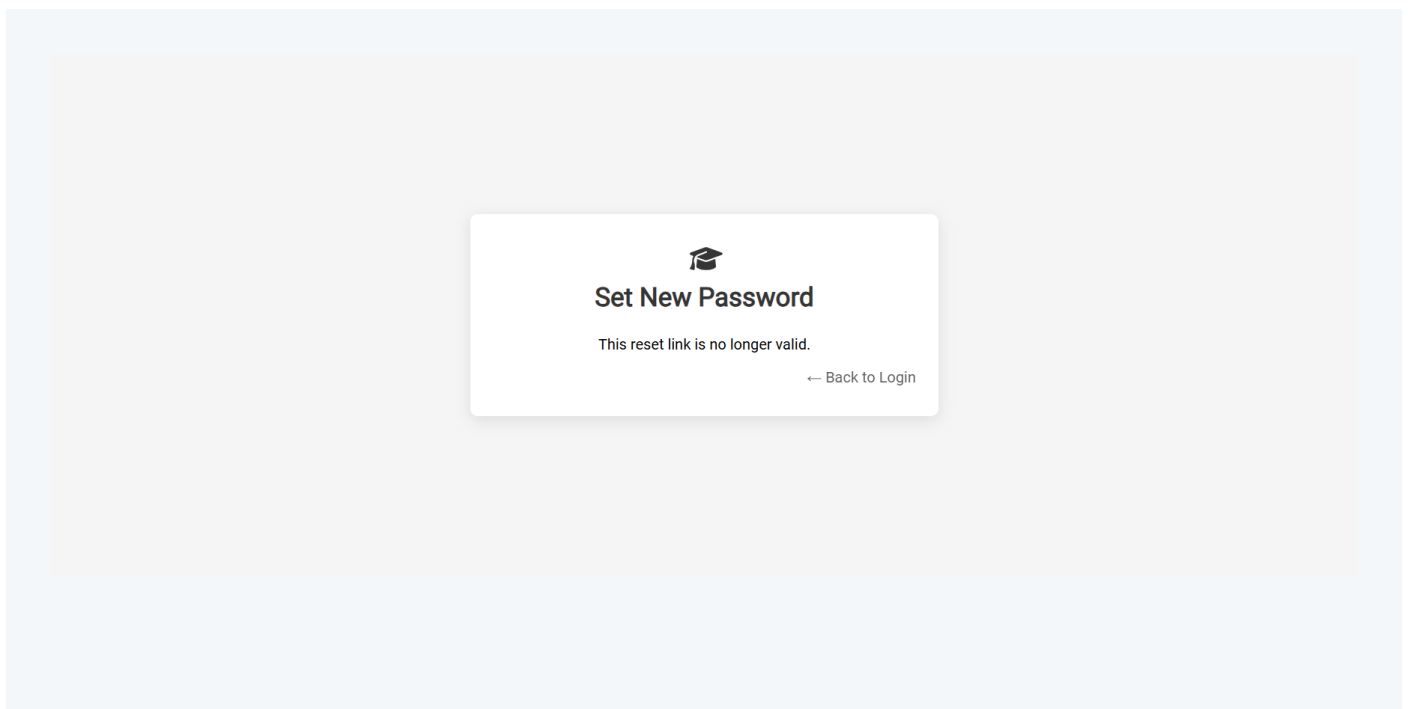
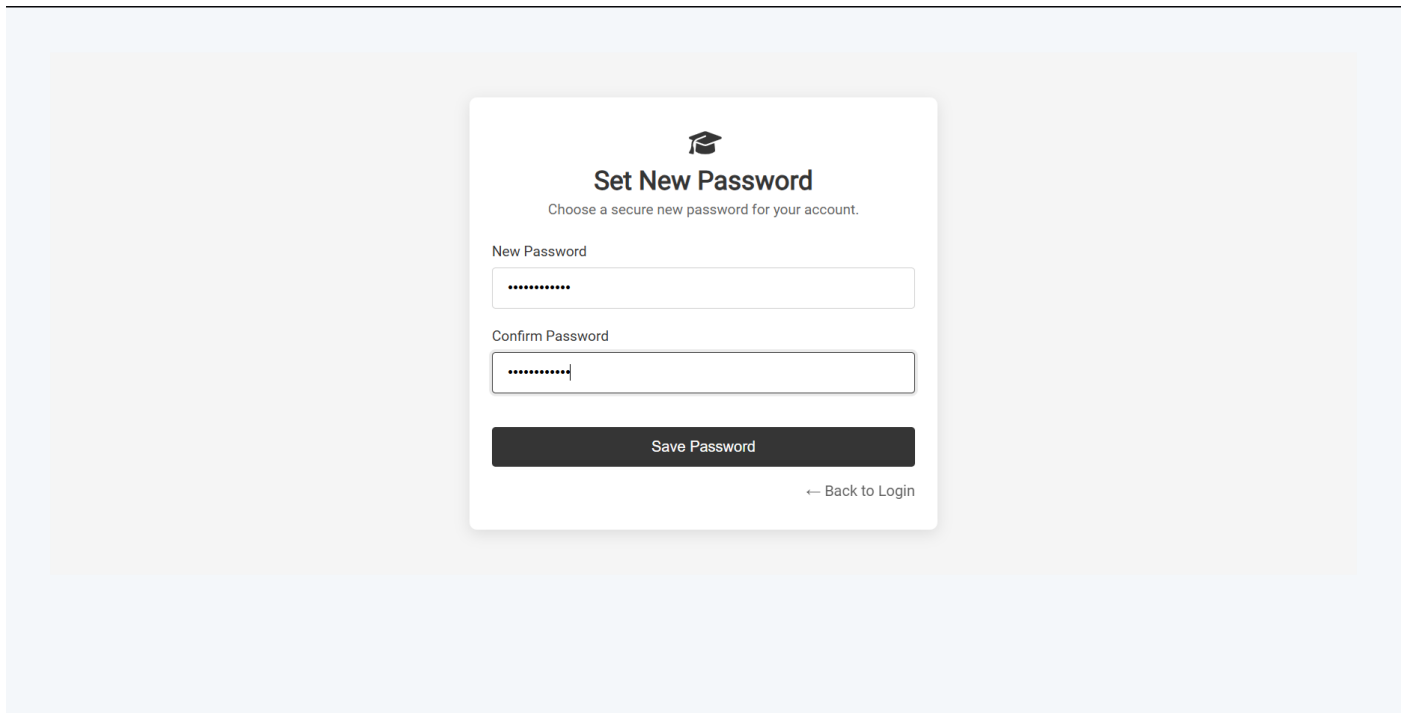


Figure 56: Message displayed in case the token is invalid or expired



The image shows a 'Set New Password' page. At the top, there is a graduation cap icon. Below it, the title 'Set New Password' is displayed, followed by the instruction 'Choose a secure new password for your account.' The form consists of two input fields: 'New Password' and 'Confirm Password', both containing masked characters (dots). Below these fields is a dark 'Save Password' button. At the bottom right, there is a link that says '← Back to Login'.

Set New Password
Choose a secure new password for your account.

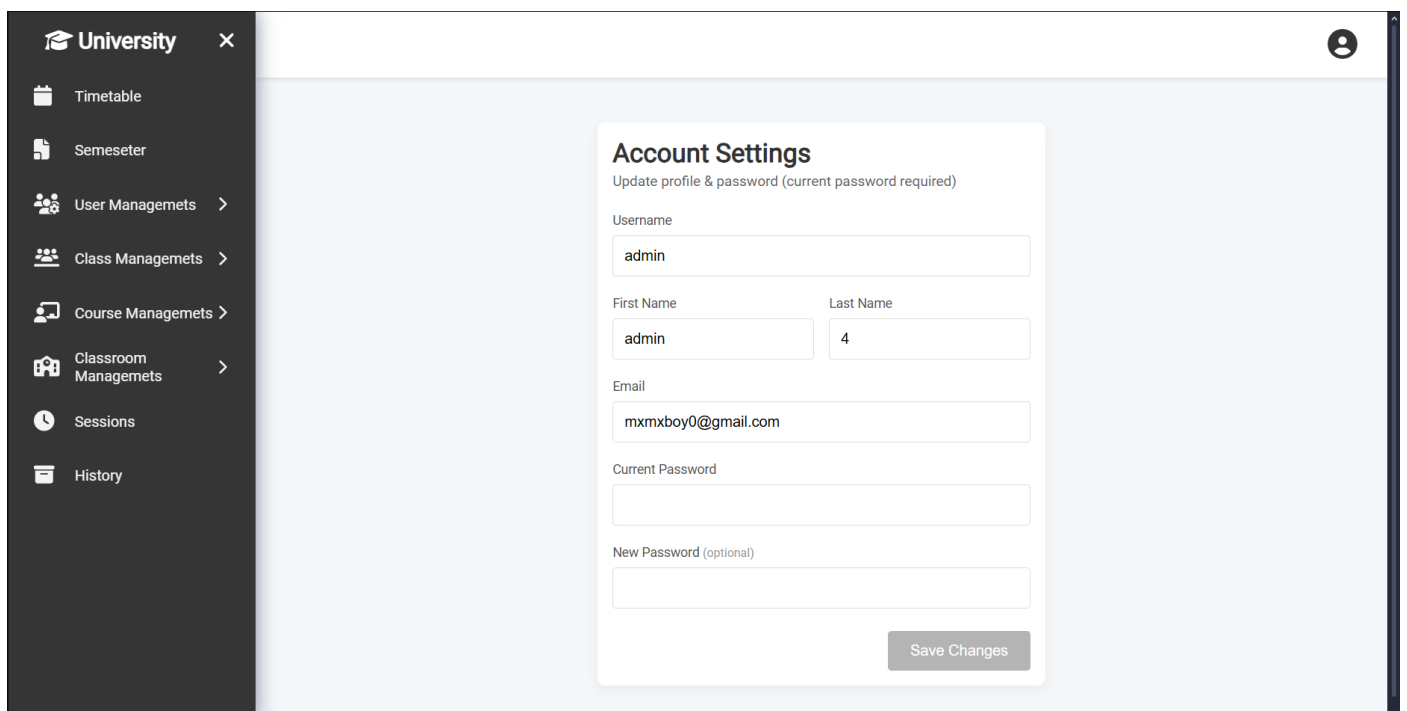
New Password
.....

Confirm Password
.....

Save Password

[← Back to Login](#)

Figure 58: New password page



The image shows an 'Account Settings' page. On the left is a dark sidebar with the 'University' logo and a list of menu items: Timetable, Semester, User Managemets, Class Managemets, Course Managemets, Classroom Managemets, Sessions, and History. The main content area has a title 'Account Settings' and a subtitle 'Update profile & password (current password required)'. The form includes fields for 'Username' (admin), 'First Name' (admin), 'Last Name' (4), 'Email' (mxmxboy0@gmail.com), 'Current Password', and 'New Password (optional)'. A 'Save Changes' button is at the bottom right.

University ×

- Timetable
- Semester
- User Managemets >
- Class Managemets >
- Course Managemets >
- Classroom Managemets >
- Sessions
- History

Account Settings
Update profile & password (current password required)

Username
admin

First Name
admin

Last Name
4

Email
mxmxboy0@gmail.com

Current Password
.....

New Password (optional)
.....

Save Changes

Figure 57: Account settings page

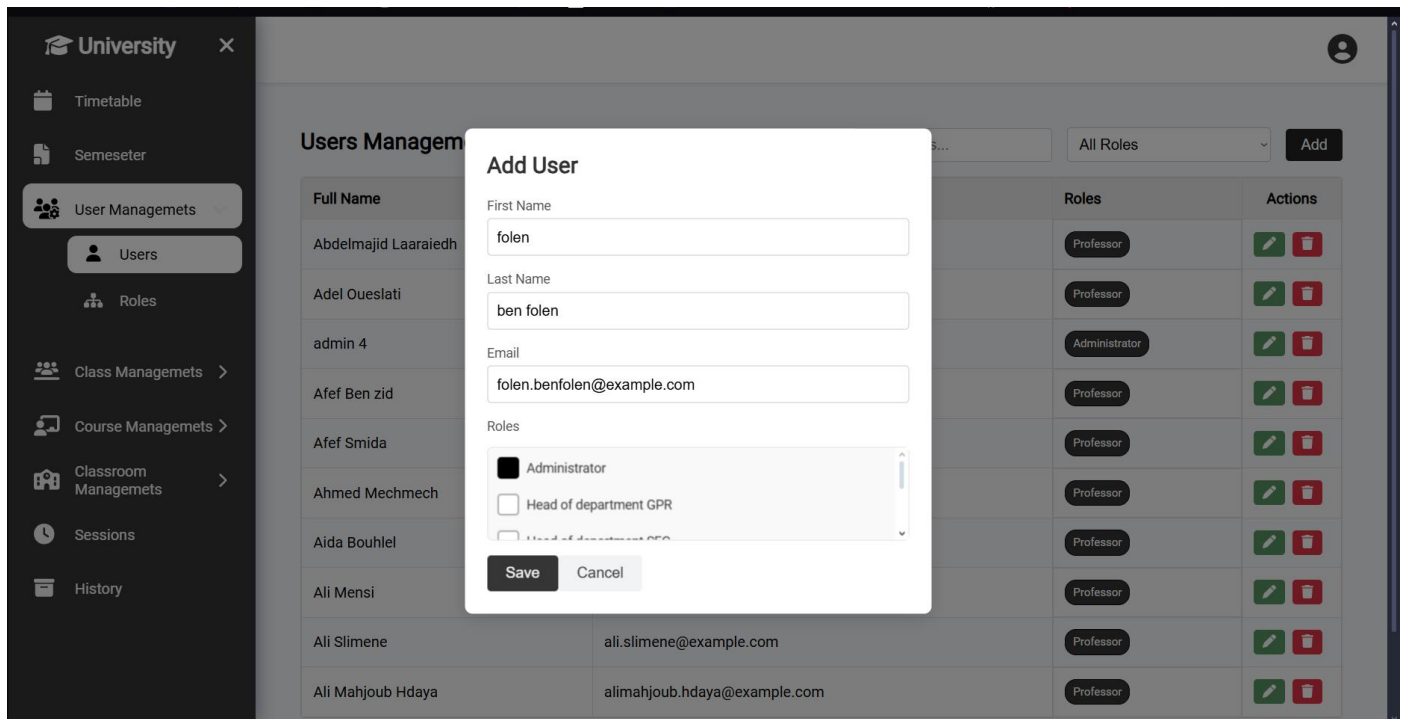


Figure 60: Add user form

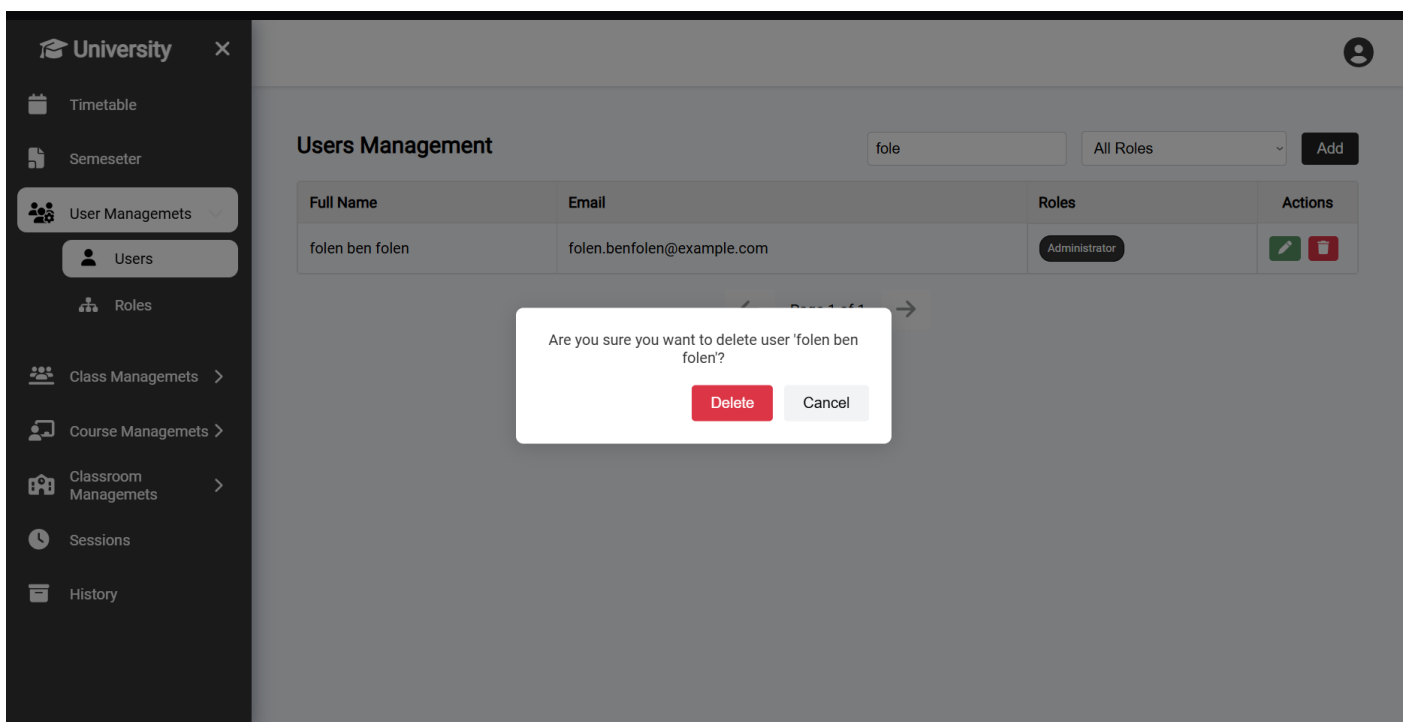


Figure 59: delete user

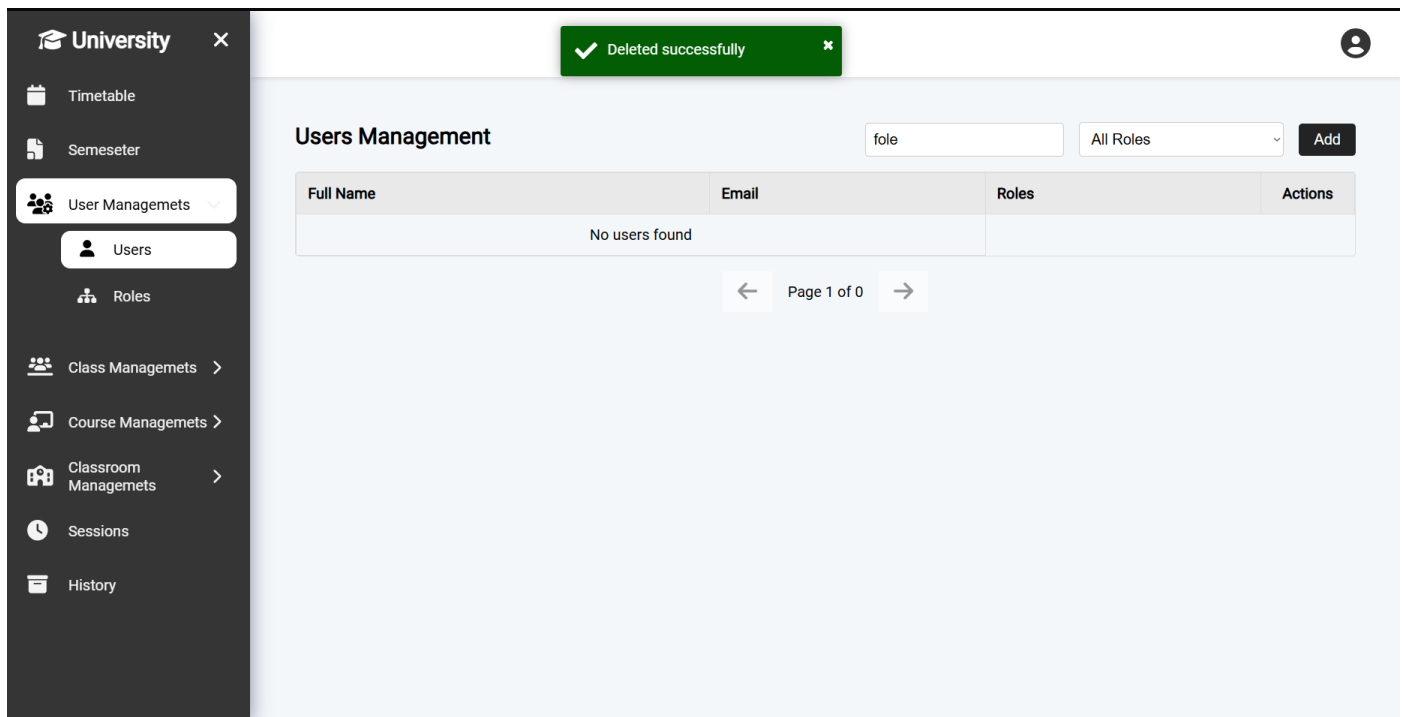


Figure 62: Success message

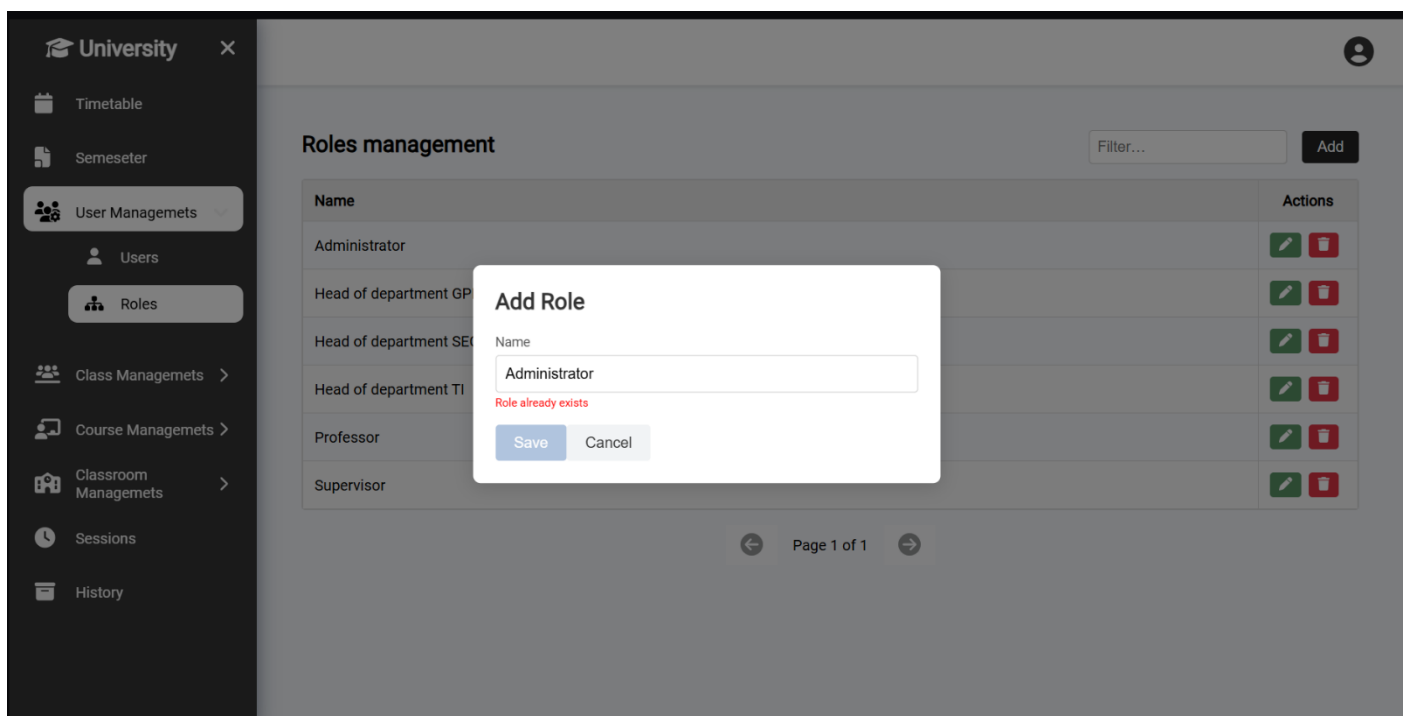


Figure 61: Add role form with verification of existing ones

5. Conclusion

Sprint 1 delivers the core foundation by implementing login, logout, account settings, and token refresh for persistent sessions, along with user and role management, and basic design and navigation elements. These components establish the essential groundwork for all subsequent features.

Chapter 5: Sprint 2 Core data flows: timetables, semesters & attendance assignment

1. Introduction

This chapter details the core data flows established during Sprint 2, focusing on the management of timetables, semesters, and attendance assignment. These features ensure the availability of crucial academic data needed for future reporting, analytics, and decision-making.

2. Specification of Requirements

This section outlines the Sprint 2 backlog, alongside the relevant use case diagram and selected user story descriptions.

2.1. Sprint Backlog

The Sprint 2 backlog includes user stories related to timetable management, semester configuration, and attendance assignment via the mobile interface. Each user story generally follows these four steps:

1. Design the interface (mobile or web).
2. Implement backend logic and models.
3. Integrate frontend with backend services.
4. Test and validate the feature.

Only user stories and their estimated complexities are listed, without detailing individual tasks.

Table 17 Sprint 2 backlog

ID	User Story	Priority	Complexity
5.1	As an admin, I want to upload an Excel file containing timetables for professors or classes so that the data is automatically imported into the database.	High	8
5.2	As an admin, I want to update timetable so that I can keep schedule data accurate.	Medium	3
5.3	As an admin, I want to delete timetable so that I can keep schedule data accurate	Medium	3
12.1	As an admin, I want to add a semester so that it can be included in the system.	High	8
12.2	As an admin, I want to update the semester period dates so that any necessary changes can be reflected.	Medium	5
12.3	As an admin, I want to delete a semester so that mistakenly created semesters can be removed.	Low	2
12.4	As an admin, I want to update the semester status from active to completed so that I can create a new semester when the previous one ends.	High	3
6	As an administrator agent, I want to assign attendance statuses so that I can record which	High	5

	lessons or academic events have been attended or missed via mobile app.		
8	As an admin, I want to view the history of all attendance statuses and updates so that I can correct any mistakes if they occur.	Medium	2
24	As a user, I want to view the sidebar with my accessible options so that I can navigate the things I can use on the website.	Low	3
25	As an administrator agent, I want to receive a notification when session is about to end and some attendance still pending needs to be assigned.	Low	3

2.2. Use case diagram

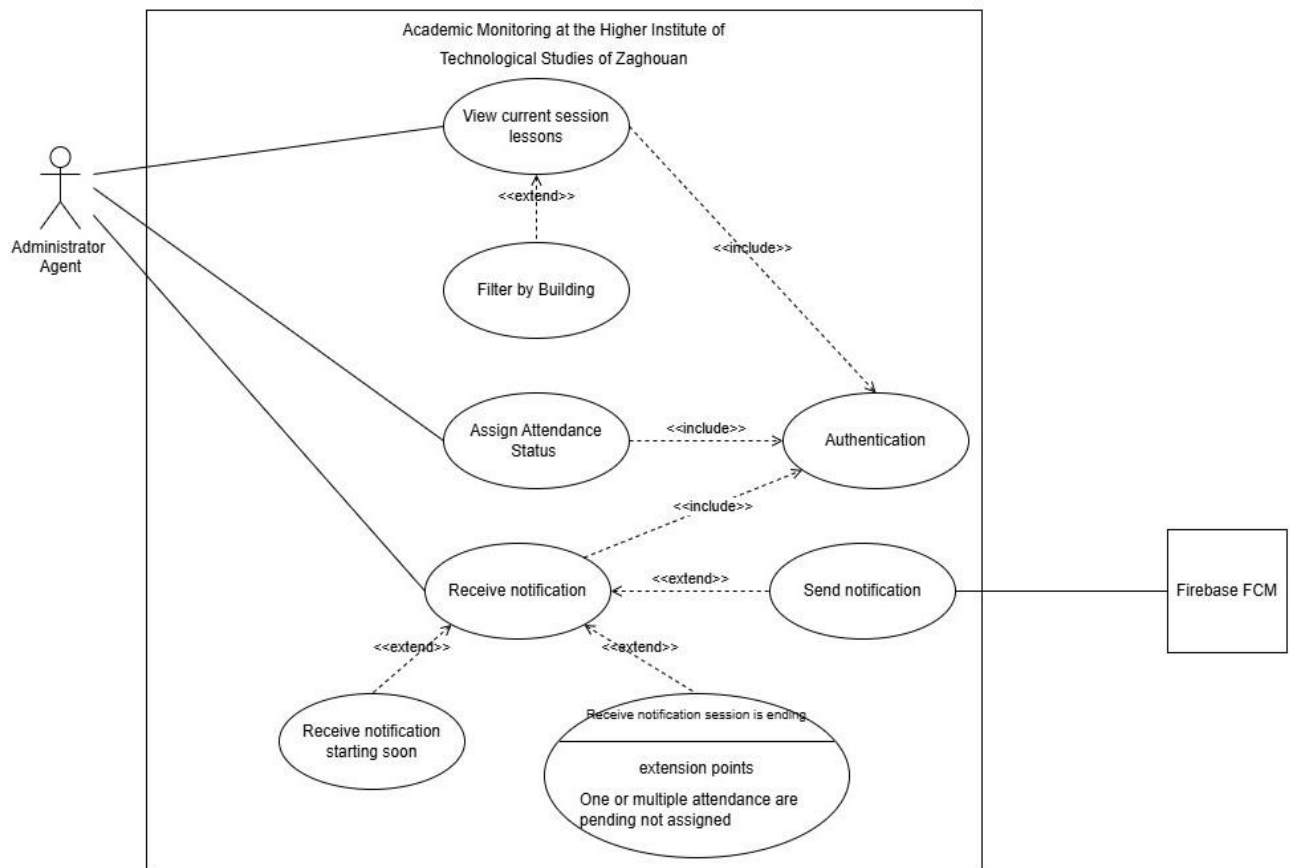


Figure 63: Sprint 2 Administrator agent use cases

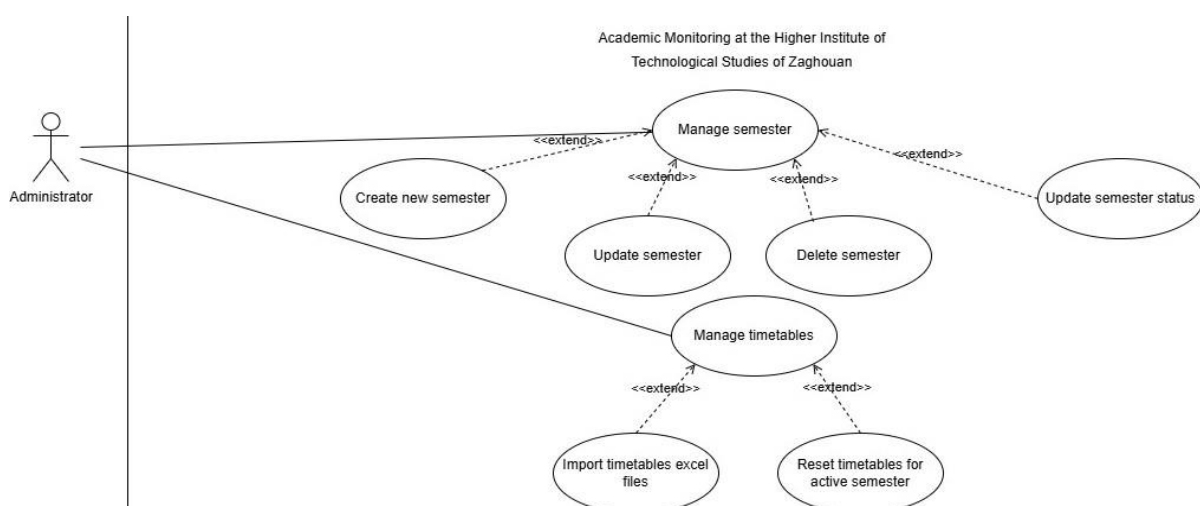


Figure 64: Sprint 2 Administrator use cases

3. Design

3.1. Class diagram

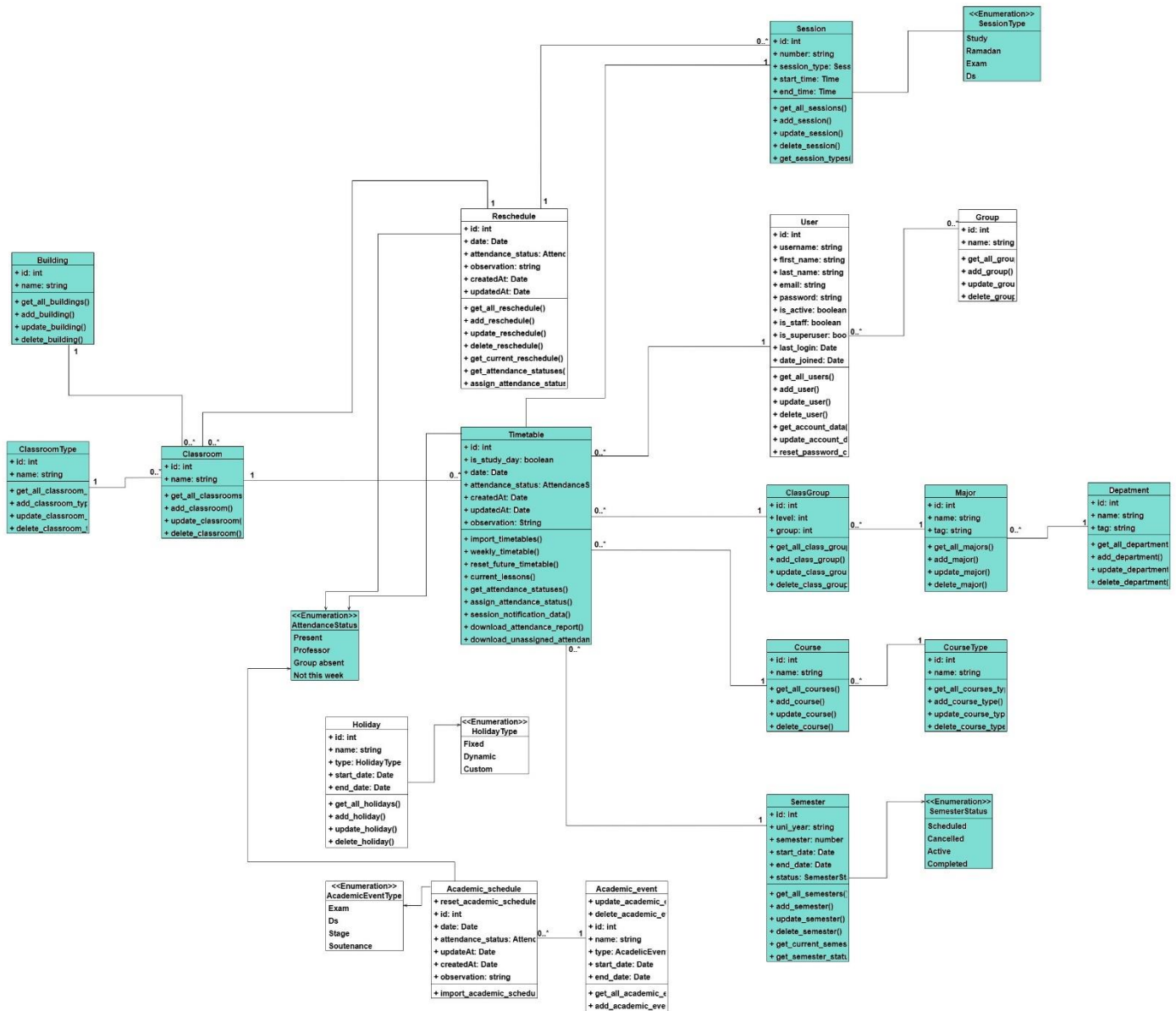


Figure 65: Sprint 2 class diagram

3.2. Sequence diagrams and textual description of use cases

3.2.1. Timetable management use case

When an administrator uploads an Excel file containing timetable data, the frontend sends the file to the Django API, which first retrieves the active semester's start and end dates via ORM. The API parses each row of the spreadsheet into timetable entries and iterates weekly from the semester's start date through its end date, matching the original row's weekday. For each computed date, the backend checks against holidays and academic events to determine the `is_study_day` flag before saving the entry via Django ORM. Once all rows and their weekly occurrences are processed, the API compiles import logs—detailing successes, errors, or missing data—and returns them to the frontend, where the administrator

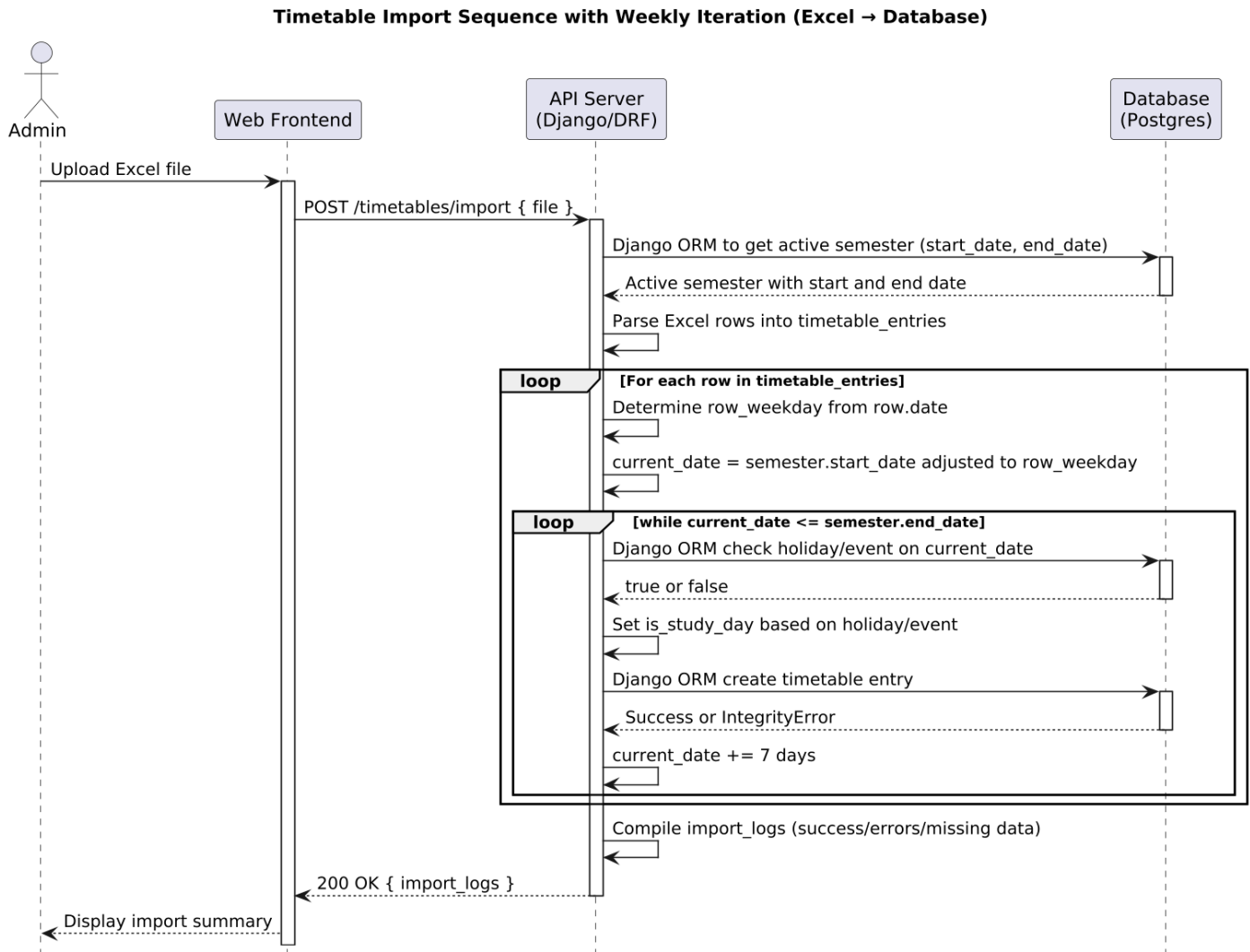


Figure 66: Import excel file timetable sequence diagram

can review the import summary.

Table 18 Analysis import Excel file timetable use case

Use Case: Manage timetables	
Title	Manage timetable
Actor	Admin
Objective	Allow admin to import timetable from excel file
Description	Enable admin to import excel file from timetable to add timetables to database of current semester.

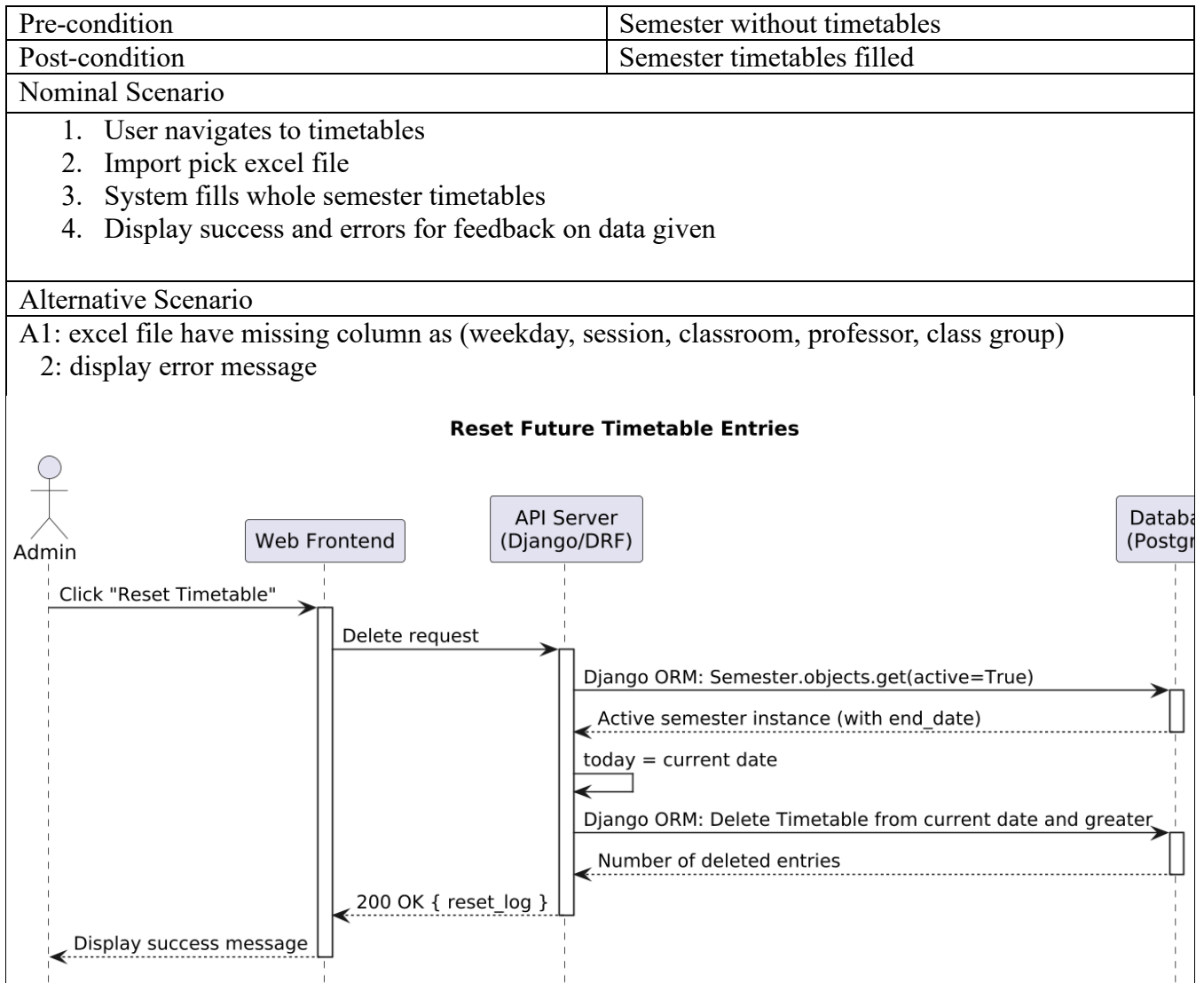


Figure 67: Reset timetables sequence diagram

Table 19 Analysis reset timetables use case

Use Case: Manage timetables	
Title	Manage timetable
Actor	Admin
Objective	Allow admin to reset timetable from excel file
Description	Enable admin to reset timetable starting from today and all future timetables
Pre-condition	Timetable filled
Post-condition	All future timetable deleted
Nominal Scenario	
1. User navigates to timetables 2. Click reset 3. Confirm reset 4. Display success	
Alternative Scenario	

3.2.2. Semester management user case

To add a new academic period, the administrator completes a semester form with a name, start date, and end date; the frontend sends this data to the API. The API retrieves the maximum end date of existing semesters via ORM and verifies that the new start date is strictly after that date to prevent overlap. If the check fails, the API returns a “400 Bad Request” error indicating the start date must follow the previous semester’s end date. Otherwise, the API creates a new semester record via ORM and returns a “201 Created” response with the new semester ID. The frontend then informs the administrator with a success message, confirming that the semester was added to the system.

Add Semester Sequence

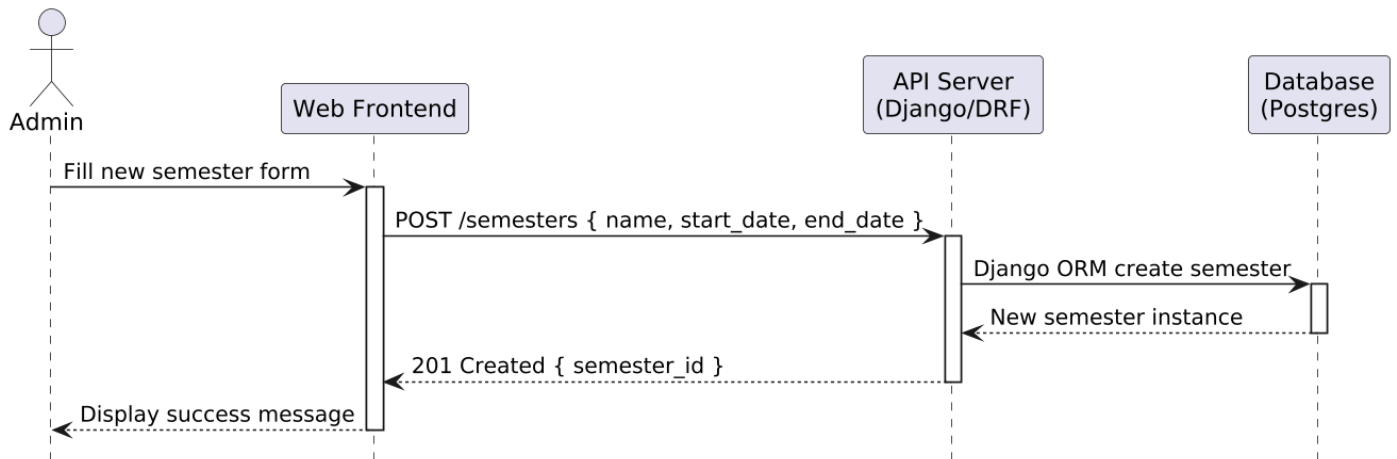


Figure 68: Create semester sequence diagram

Table 20 Analysis create semester use case

Use Case: Manage semester	
Title	Create semester
Actor	Admin
Objective	Allow admin to create semester
Description	Enable admin to create semester to keep organized timetable per semester
Pre-condition	No semester
Post-condition	Semester created
Nominal Scenario	
1. User navigates to semester 2. Click add 3. Fill add semester form 4. Click add	
Alternative Scenario	
A1: start date is earlier than the greatest date in the database 2: Error will be returned 3: error message will display for user	

3.2.3. Assign attendance

The administrator agent opens the mobile app and requests the list of current-session lessons, which the backend retrieves from the database and returns. After selecting a specific lesson, the agent views all attendance records for that lesson and marks each student’s status and any observations. The

mobile app then sends a PUT /attendances/{lesson_id}/assign request with the updated statuses to the API. The backend updates each record via Django’s ORM and confirms completion. Finally, the mobile app refreshes the UI, move that lesson from the unassigned list to assigned list to show it has been processed.

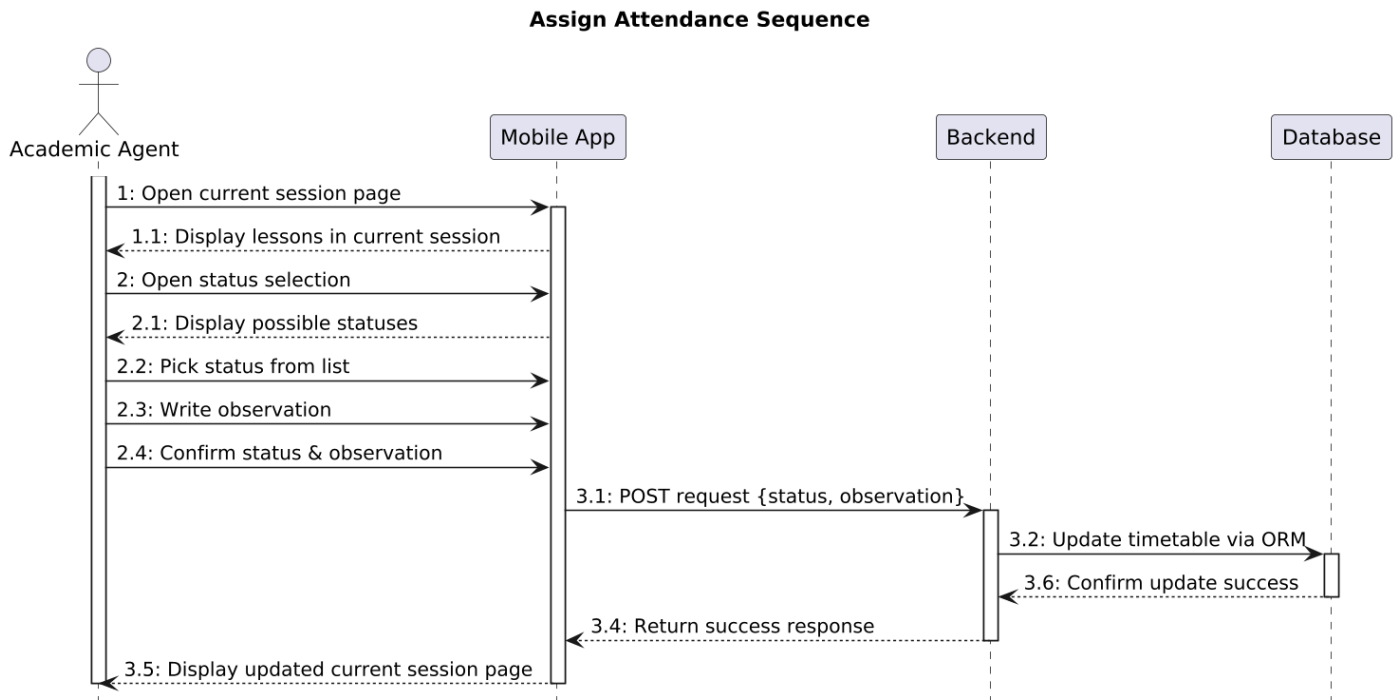


Figure 69: Assign attendance sequence diagram

Table 21 Analysis assign attendance use case

Use Case: Assign attendance	
Title	Assign attendance
Actor	Administrator agent
Objective	Allow administrator agent to assign attendance
Description	Enable administrator agent to assign attendance to each lesson in current session.
Pre-condition	Lessons without attendance
Post-condition	Lessons with assigned attendance
Nominal Scenario	
1. Administrator agent opens mobile app 2. Opens current session page 3. Pick lesson 4. Pick attendance form it 5. Add observation 6. confirm	
Alternative Scenario	

3.2.3. Notifications

Every 15 minutes, Celery Beat enqueues a notify_sessions task into Redis, which is then picked up by a Celery Worker. The worker calls the API endpoint /api/timetables/notify-sessions, prompting the

backend to query the database for any sessions about to start or end with pending attendance. If such sessions are found, the API sends push notifications via FCM to the administrator agent

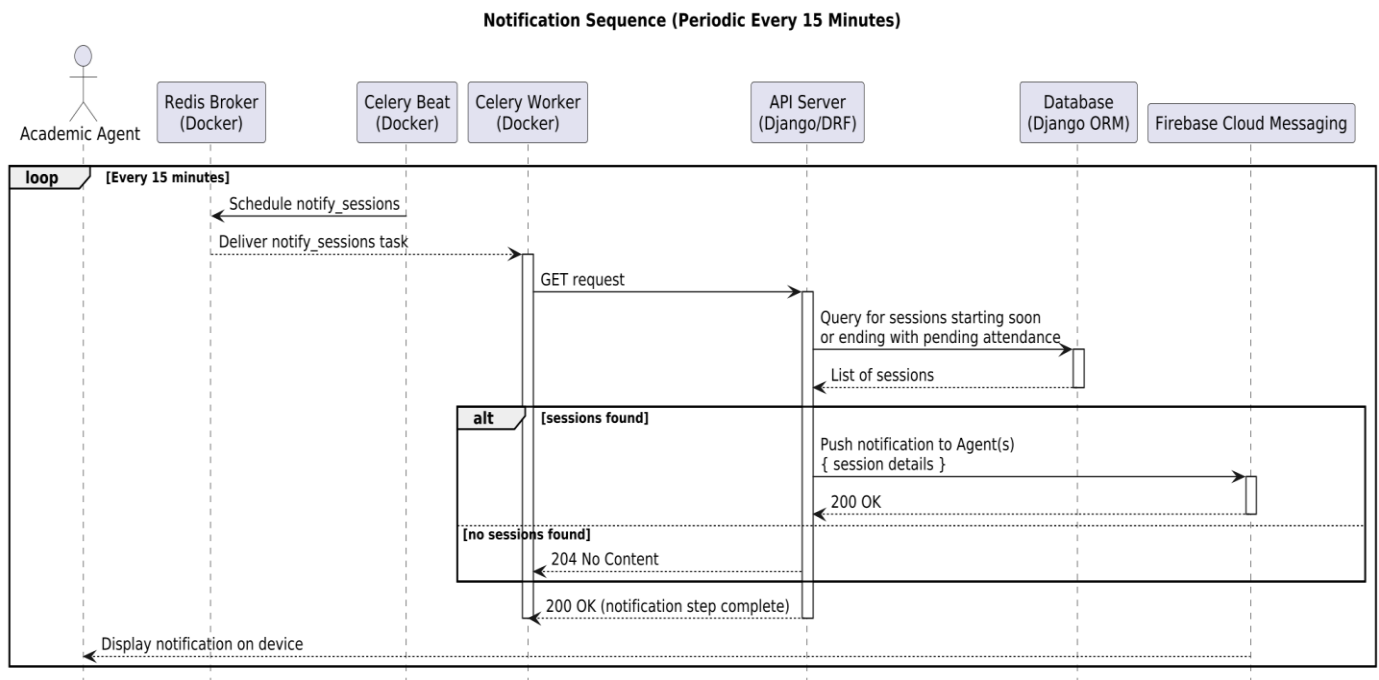


Table 22 Analysis use case

Figure 70: Notification sequence diagram

Use Case: Receive notification	
Title	Receive notification
Actor	Administrator agent
Objective	Allow administrator agent to receive notifications
Description	Enable administrator agent to receive notifications before session starts and when session is about to end and on or more lesson is still unassigned
Pre-condition	No notifications
Post-condition	Administrator agent alerted
Nominal Scenario	
1. Every 15 minutes celery beat tell Redis to give task to celery worker 2. Celery worker send request 3. Api checks current session or next session 4. If session is starting soon or session is about to end 5. Firebase FCM send notification to device 6. Administrator agent receives notification on his device	
Alternative Scenario	

4. Implementation

University

Timetable

Semeseter

User Managemets >

Class Managemets >

Course Managemets >

Classroom Managemets >

Sessions

History

Timetables Management

Select Semester

All Departments

☒ Class Group
 ☐ Professor

Select Class Group

Reset

Import

Import Completed:

Rows: 349

Created: 5933

Errors:

DAY / SESSION	S1 08:30 - 10:00	S2 10:10 - 11:40	S3 11:50 - 13:20	S4 13:30 - 15:00	S5 15:10 - 16:40	S6 16:40 - 18:20
Monday	-	-	-	-	-	-
Tuesday	-	-	-	-	-	-
Wednesday	-	-	-	-	-	-
Thursday	-	-	-	-	-	-

Figure 72: Result of import excel file

University

Timetable

Semeseter

User Managemets >

Class Managemets >

Course Managemets >

Classroom Managemets >

Sessions

History

Semester Manag

All Uni-Years

Add

Uni-year	Status	Actions
2024-2025	Completed	

Add Semester

University Year

2024-2025

Number

2

Start Date

03/02/2025

End Date

31/05/2025

Status

Active

Save

Cancel

Figure 71: Add new semester

Timetables Management

2024-2025 S2
Technologies de l'informatique
☒ Class Group ☐ Professor
DSI 22
Reset
Import

DAY / SESSION	S1 08:30 - 10:00	S2 10:10 - 11:40	S3 11:50 - 13:20	S4 13:30 - 15:00	S5 15:10 - 16:40	S6 16:40 - 18:20
Monday	SGBD Mohsen Othmani Info 08	Atelier SGBD Mohsen Othmani Info 08	Atelier SGBD Mohsen Othmani Info 08	-	Atelier SGBD Mohsen Othmani Info 08	Atelier SGBD Mohsen Othmani Info 08
Tuesday	-	-	-	Atelier virtualisation et cloud Hela Queslati Info 10	Atelier virtualisation et cloud Hela Queslati Info 10	-
Wednesday	Introduction virtualisation et cloud Hela Queslati Info 01	Atelier programmation objet avancée Walid Bayounes Info 10	-	Etude et management de projets innovants Ramzi Mahjoubi A05	-	-
Thursday	-	-	-	Architecture logicielle Emna Ben chaaben A03	-	-
Friday	-	Préparation à la certification français Salsaby Elkrawa A01	Business english Rim Boubaker A01	-	-	-
Saturday	-	-	-	-	-	-

Figure 74: Viewing timetable after filtering

The screenshot shows the Docker Desktop application window. The top bar includes the Docker logo, the word 'docker', and 'desktop PERSONAL'. A search bar with 'Search' and a 'Ctrl+K' button is present. The left sidebar contains navigation icons and labels: Ask Gordon (BETA), Containers, Images, Volumes, Builds, Models (BETA), Docker Hub, Docker Scout, and Extensions. The main area is titled 'backend' with a path 'C:\Users\MAMOU\Desktop\PF FINAL VERSION\university\backend'. Below this is a list of containers: 'worker' (backend-worker), 'celery-beat' (backend-celery-beat), 'web' (backend-web 7777:8000), 'db' (postgres:14 5432:5432), and 'redis' (redis:7.2.4 6379:6379). Each container has a status icon, a port mapping, and a delete button. To the right of the container list is a 'View configurations' button. The rightmost panel shows the configuration for the 'celery' key, including a list of tasks and a log output. The log output shows messages from the MainProcess and ForkPoolWorker-8, indicating successful task execution and notification checks.

Figure 73: docker container

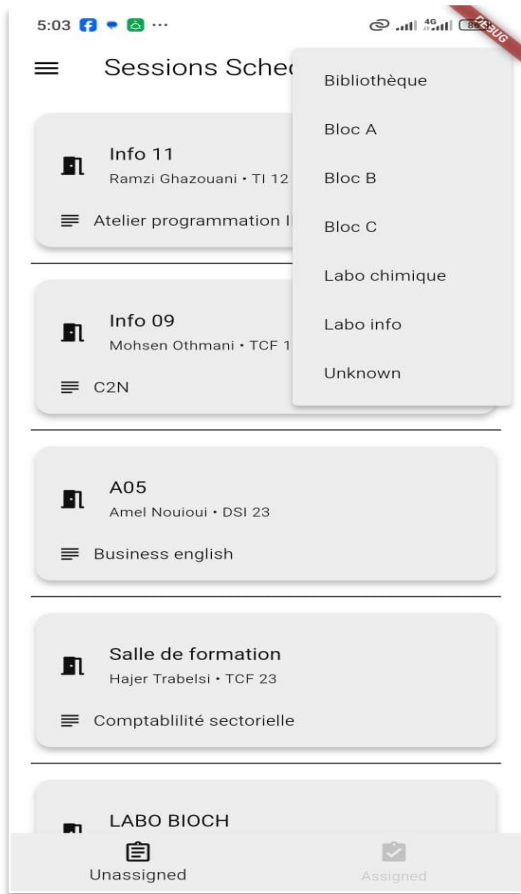


Figure 75: Current session screen

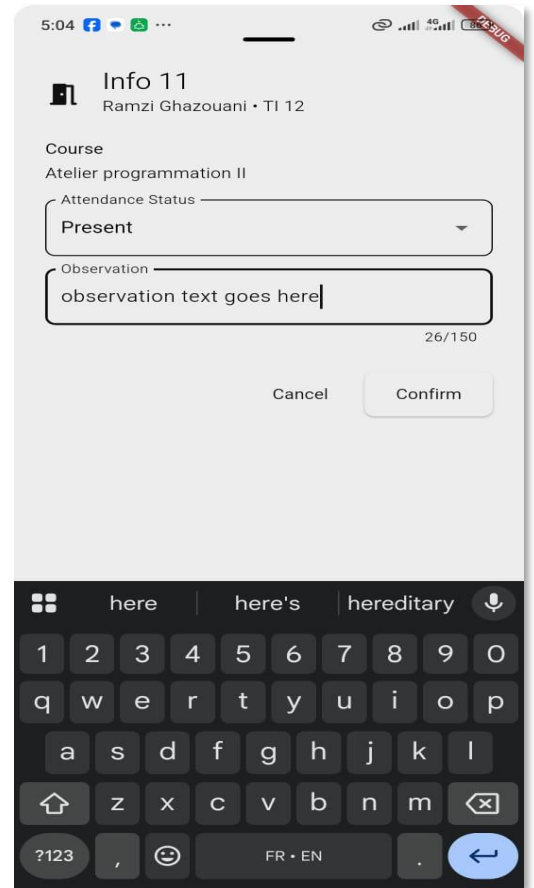


Figure 78: assigning attendance

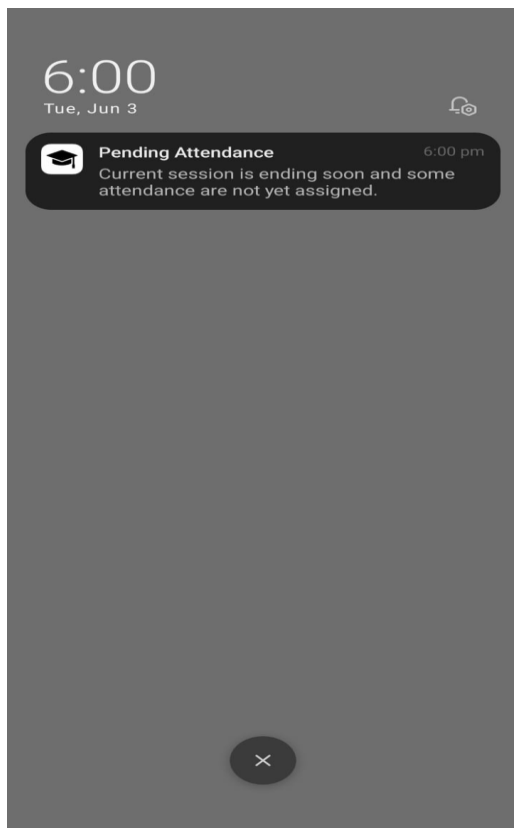


Figure 77: Notification

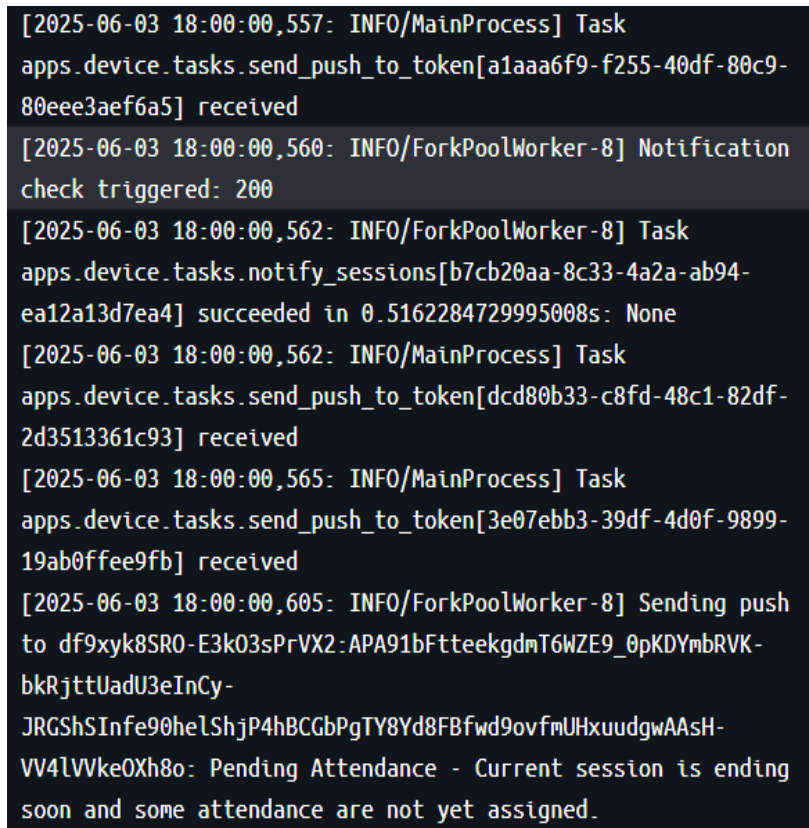


Figure 76: Docker console notification success

5. Conclusion

Sprint 2 successfully implements core data flows semester creation, timetable import, and mobile attendance assignment laying the groundwork for real-time tracking and reporting.

Chapter 6: Sprint 3 Reporting & analytics: dashboards

1. Introduction

In the previous chapter, we introduced the core concepts of Business Intelligence (BI), presented the main data flow, and outlined the objectives of integrating Power BI into our project. This chapter focuses on the practical implementation of sprint 3 where we begin by the creation of a DataMart in PostgreSQL using a snowflake schema. We then describe the ETL process used to populate the DataMart from the operational database, and finally, we set up Power BI dashboards to visualize and analyse the data effectively.

2. Specification of Requirements

This section outlines the Sprint 4 backlog, alongside the relevant use case diagram and selected user story descriptions

2.1. Sprint Backlog

The Sprint 3 backlog includes user stories related to Reporting and creating the analytics dashboards to monitor different aspect of the academic Each user story generally follows these four steps:

1. Connect to database via power bi
2. Transform data
3. Create visualisations and dashboards
4. Publish them

Table 23 Sprint 3 backlog

ID	User Story	Priority	Complexity
US9	As a supervisor, I want to view statistics on the university's courses progress status.	High	3
US10	As a head of department, I want to view statistics on my department's progress.	Medium	2
US11	As a professor, I want to view my personal progress statistics.	Medium	1
US13	As a supervisor, I want to view historical statistics from previous semesters.	Low	8
US14	As a supervisor, I want to compare the progress of two professors' courses so that I can maintain harmony before exams and ensure alignment between classes.	Medium	2
US14.2	As a supervisor, I want to compare statistics between semesters so that I can evaluate whether the current semester has improved or declined compared to previous ones.	Medium	3
US15	As a head of department, I want to compare the progress of two professors in my department so that I can assess performance and alignment.	Medium	2
US16	As a professor, I want to compare my personal progress with that of my classes and courses so that I can stay aligned with my colleagues.	Medium	2
US20	As a supervisor, I want to view statistics about classroom usage and overuse so that I can make informed decisions about classroom allocation.	Medium	2

2.2. Use cases

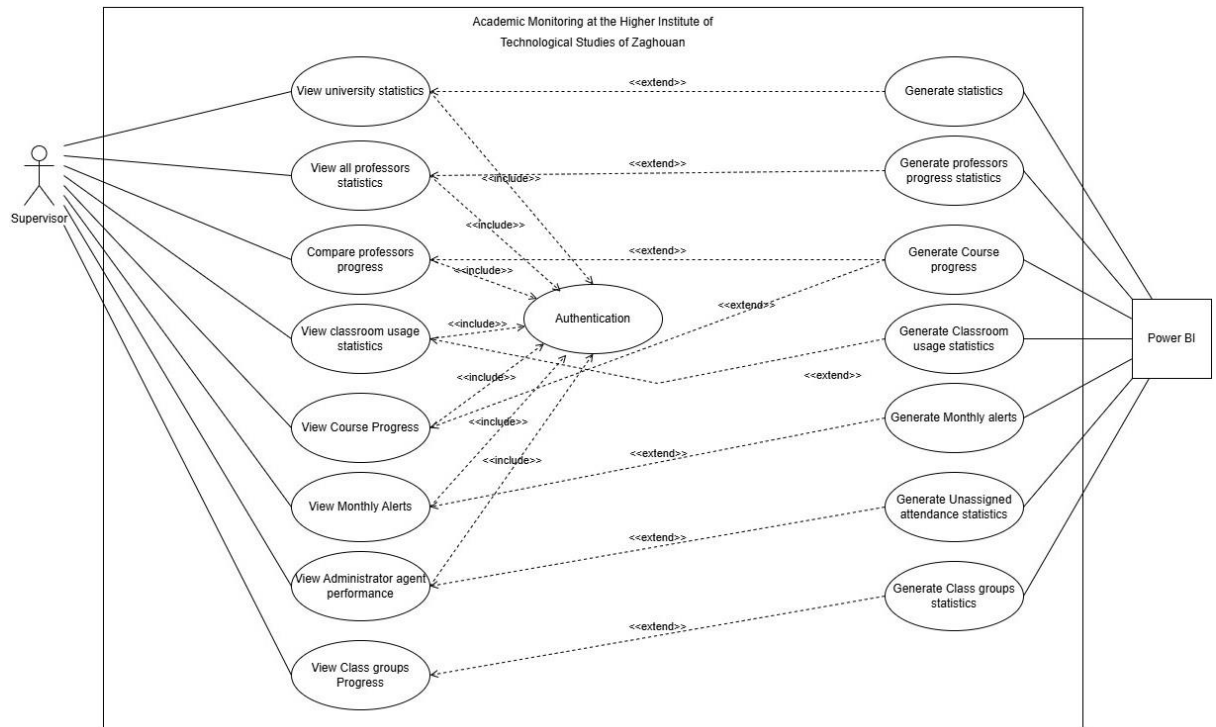


Figure 79: Supervisor use case

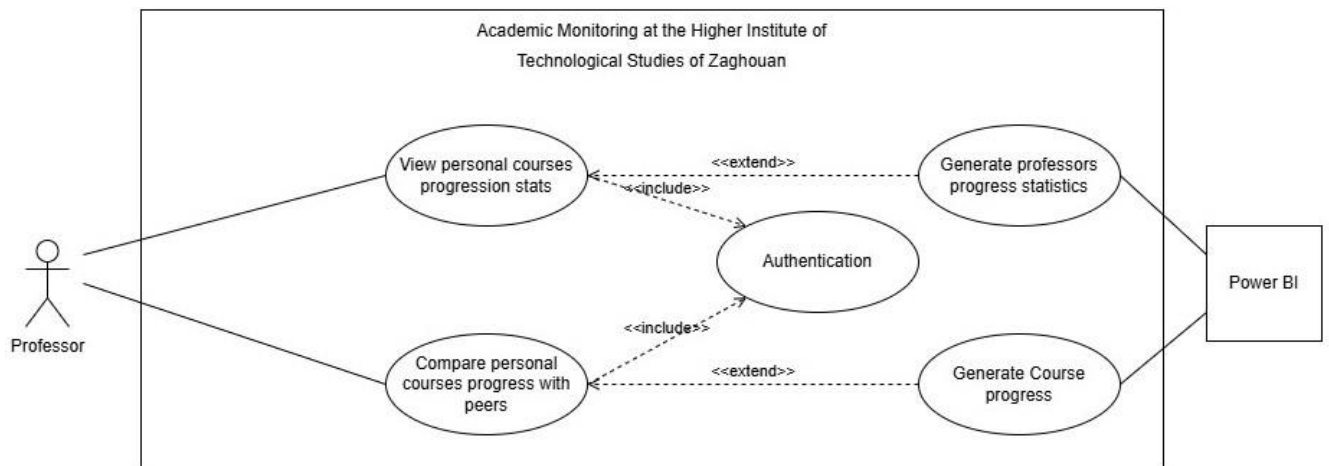


Figure 81: Professor use case

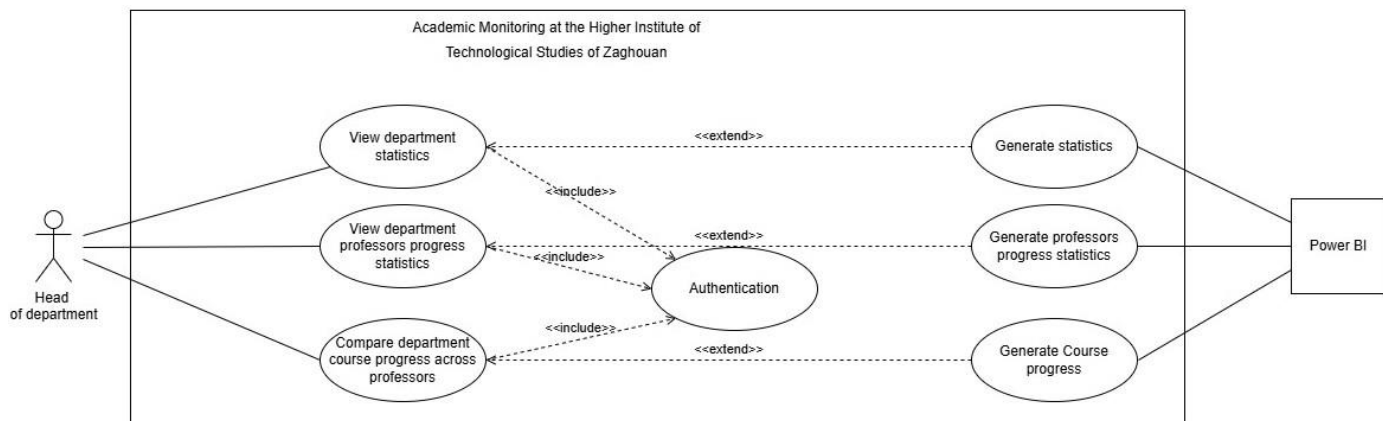


Figure 80: Head of department use case

3. Connecting to the Data Source

Power BI supports a wide range of data sources, including Excel workbooks, CSV/Text files, multiple databases, web URLs, Azure services, and more, and these connectors are regularly updated as new sources become available, allowing you to bring together disparate data into a single analytical environment. For our implementation, we will connect directly to our PostgreSQL operational database. In the “Get Data” dialog, we select the “PostgreSQL database” connector, then supply the required **Server** (in the format hostname:port) and **Database** name along with our authentication credentials. This direct database connection ensures that we can extract the tables and other relevant tables without intermediate exports or manual imports.

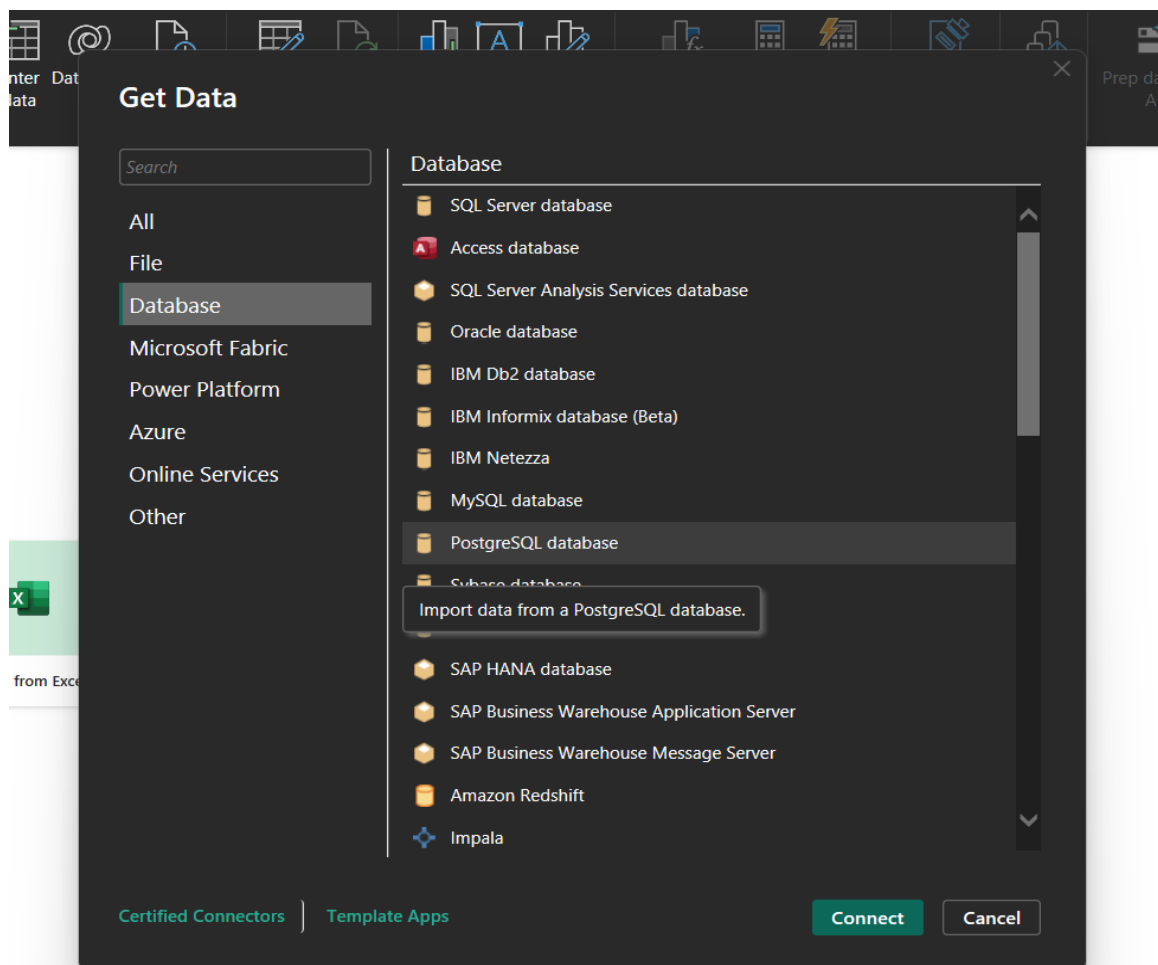


Figure 82: Choosing source PostgreSQL

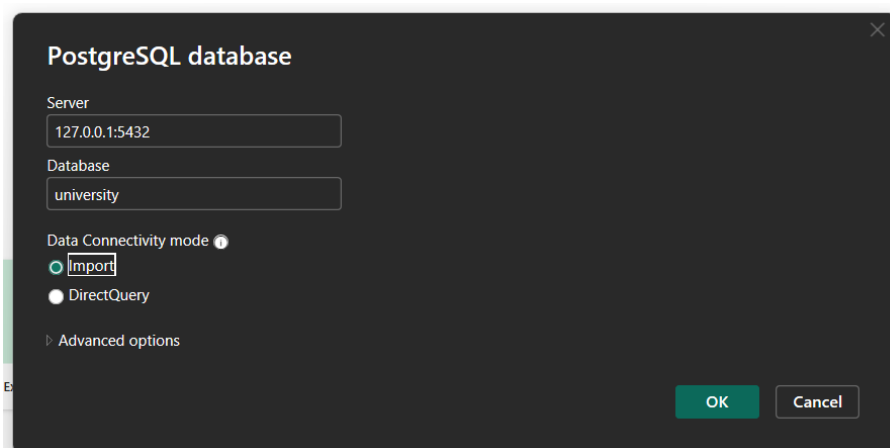


Figure 83: Connecting to the database

4. Data Preparation (Power Query)

Data Preparation involves cleaning, transforming, and structuring raw data so it's ready for analysis. This step ensures consistency, accuracy, and usability by handling tasks such as removing duplicates, filtering out irrelevant records, standardizing formats (e.g., dates and text), and merging related datasets. Effective data preparation lays the foundation for reliable insights and high-performance reporting.

In Power BI, Power Query is the built-in ETL tool used for data preparation. Accessible via the Transform Data button, Power Query provides a user-friendly interface for applying step-by-step transformations such as renaming columns, changing data types, splitting or merging tables, and writing custom M-language expressions. All transformation steps are recorded in a query script, making it easy to refresh and reproduce your data preparation logic whenever the underlying data changes.

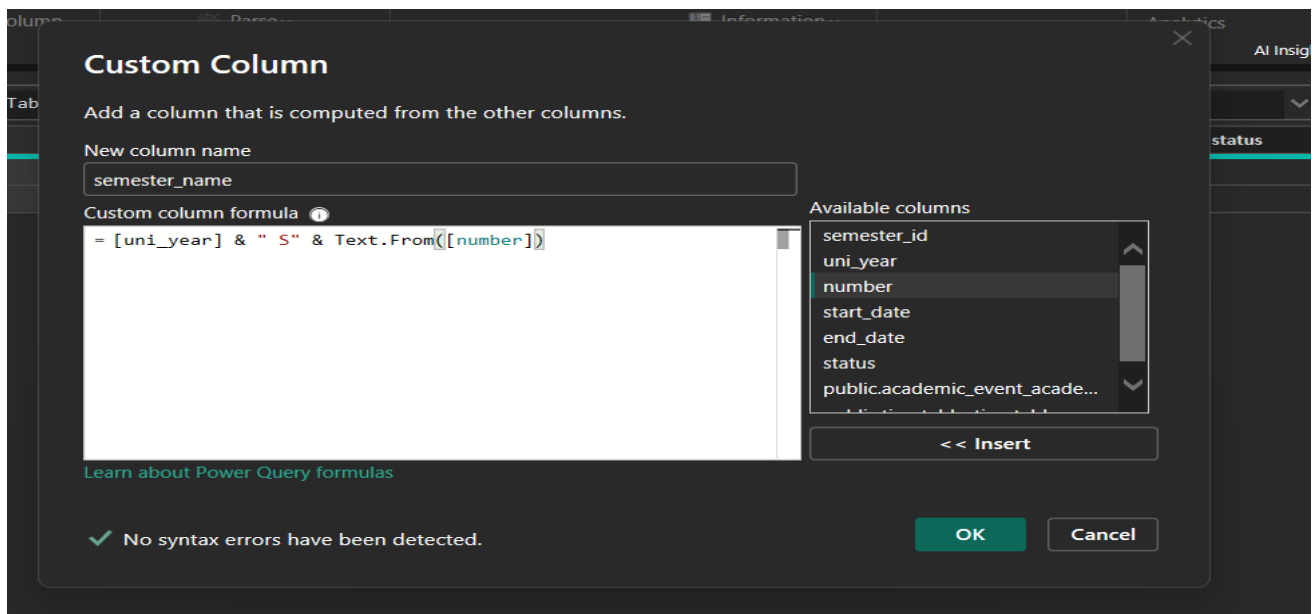


Figure 84: Adding column

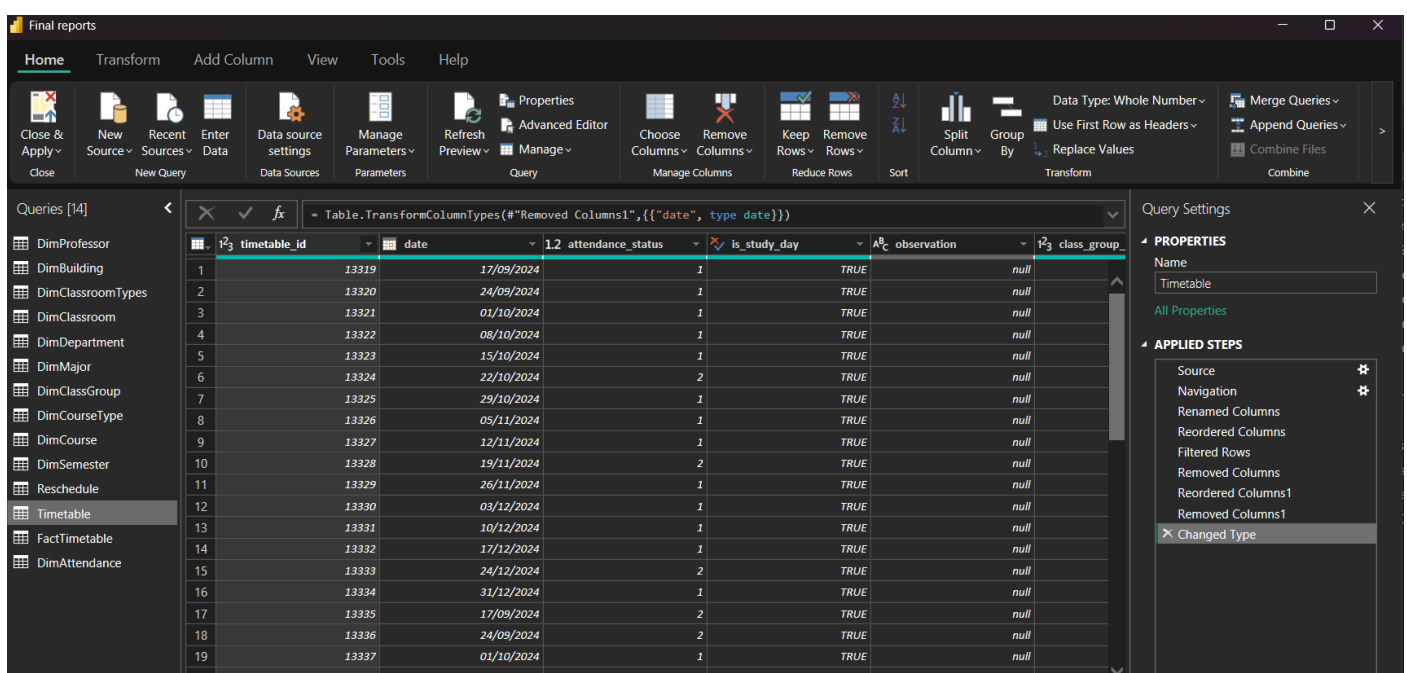


Figure 85: Power query power bi screen

5. Data Modelling

Data modelling in Power BI involves defining relationships between tables to enable precise and performant analysis. After loading and cleaning our data, we organize it around a central FactTimetable table, surrounded by dimension tables: DimDate, DimAttendance, DimBuilding, DimClassGroup, DimClassroom, DimCourse, DimDepartment, DimMajor, DimProfessor, and DimSemester. We also include sub-dimensions DimMajor links DimDepartment to DimClassGroup, while DimCourseTypes and DimClassroomTypes extend DimCourse and DimClassroom, respectively this schema shown in Figure 86.

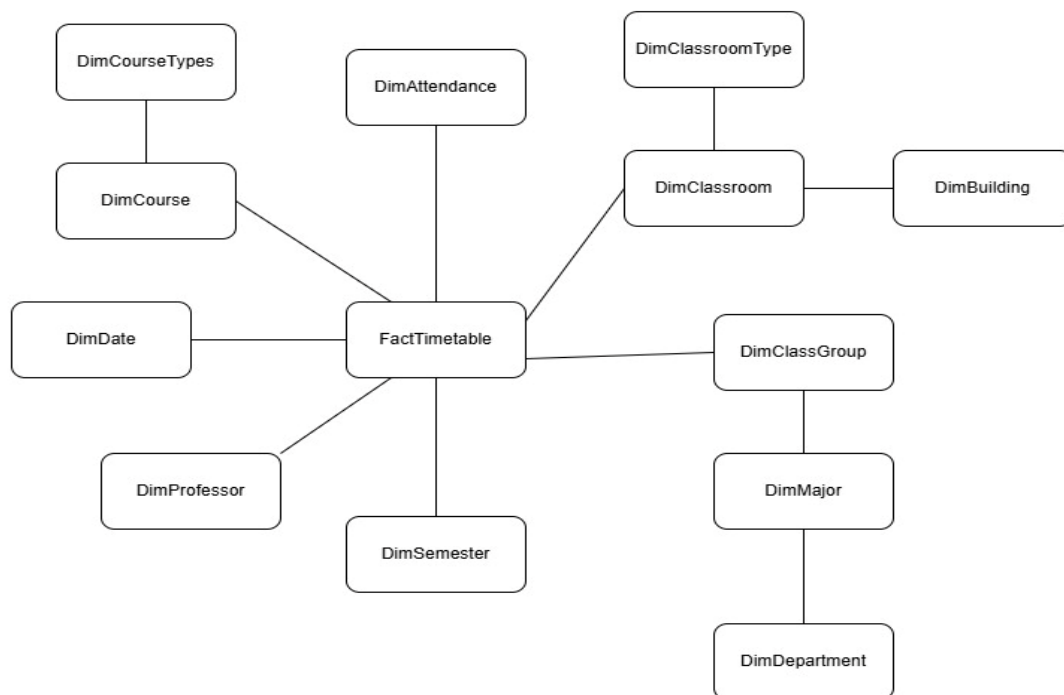


Figure 86: Our snowflake schema

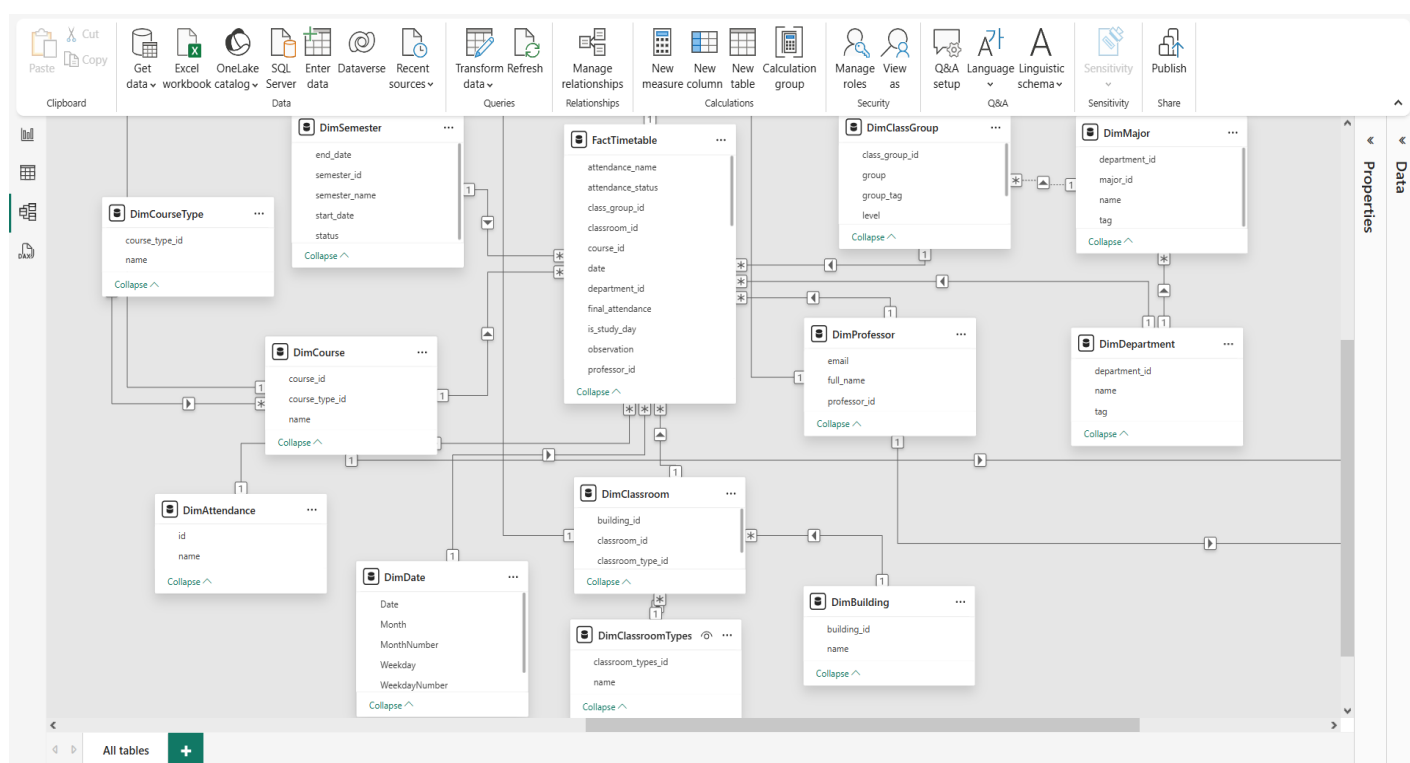


Figure 87: Modelling and relationship from Power BI

6. Metrics

Metrics in Power BI are dynamic calculations using DAX which refers to Data Analysis Expressions (DAX) it's a library of functions and operators that can be combined to build formulas and expressions in Power BI, Analysis Services, and Power Pivot in Excel data models. That summarize data into meaningful values such as attendance rates or total sessions. They are essential for tracking KPIs and deriving insights from large datasets. Unlike static data, metrics respond to filters and slicers, updating in real-time within visuals. This enables users to monitor performance and trends directly from interactive dashboards.

This Power BI report includes a curated set of 24 key measures designed to monitor academic attendance, courses progress, professor and group performance, and classroom utilization. Metrics like `AttendanceRate %`, `Professor_AttendanceRate %`, `GroupAbsentRate %`, and `Course_AttendanceRate %` provide a multi-level view of presence and absenteeism trends. Supporting counts such as `PresentCount`, `ProfessorAbsentCount`, `GroupAbsentCount`, and `UnassignedCount` give tangible context to these percentages. To enhance data clarity, dynamic text measures like `ProfessorReportTitle`, `GroupReportTitle`, and `CourseReportTitle` adjust based on filters, personalizing each view.

From a scheduling and efficiency perspective, `TotalScheduledHours`, `TotalAvailableHours`, `UnassignedHours`, and `RoomHours` quantify how classroom time is allocated and used. Utilization performance is assessed through `UtilizationRate %`, `RoomUtilization %`, and `AvgDailyUtilization %`, allowing stakeholders to evaluate how well space and time are being leveraged. Finally, `TotalSessionsCount`, `UnassignedRate %`, and `DailyUnassignedCount` help track potential scheduling gaps or delays in academic planning, ensuring accountability and continuous improvement in timetable management.

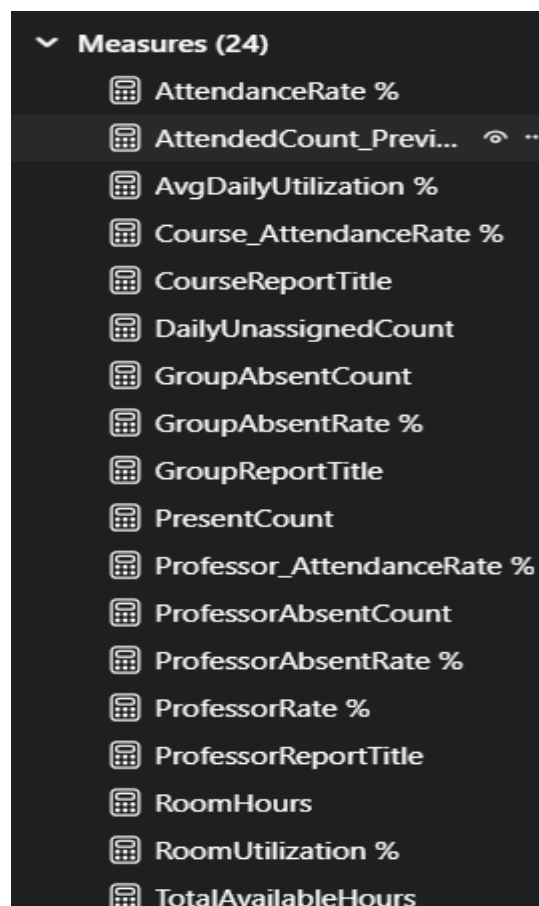


Figure 88: Metrics from table FactTimetable

```

1 PresentCount =
2 COUNTROWS(
3   FILTER(
4     FactTimetable,
5     FactTimetable[final_attendance] = 1
6   )
7 )

```

Figure 89: Metrics Present count using DAX

```

AttendanceRate % =
DIVIDE(
  [PresentCount],
  [TotalSessionsCount],
  0
)

```

Figure 90: Metrics Attendance Rate % using DAX

7. Report Design & Visualization

In this section, we present a set of dashboard interfaces generated as part of our solution. Each dashboard element is designed to meet a specific need related to the monitoring and management of academic activities. Figures 82, 82, 84, 85, 86, 87 and 88 showcase the final dashboards, highlighting attendance tracking, course progress, and classroom utilization within our university system.

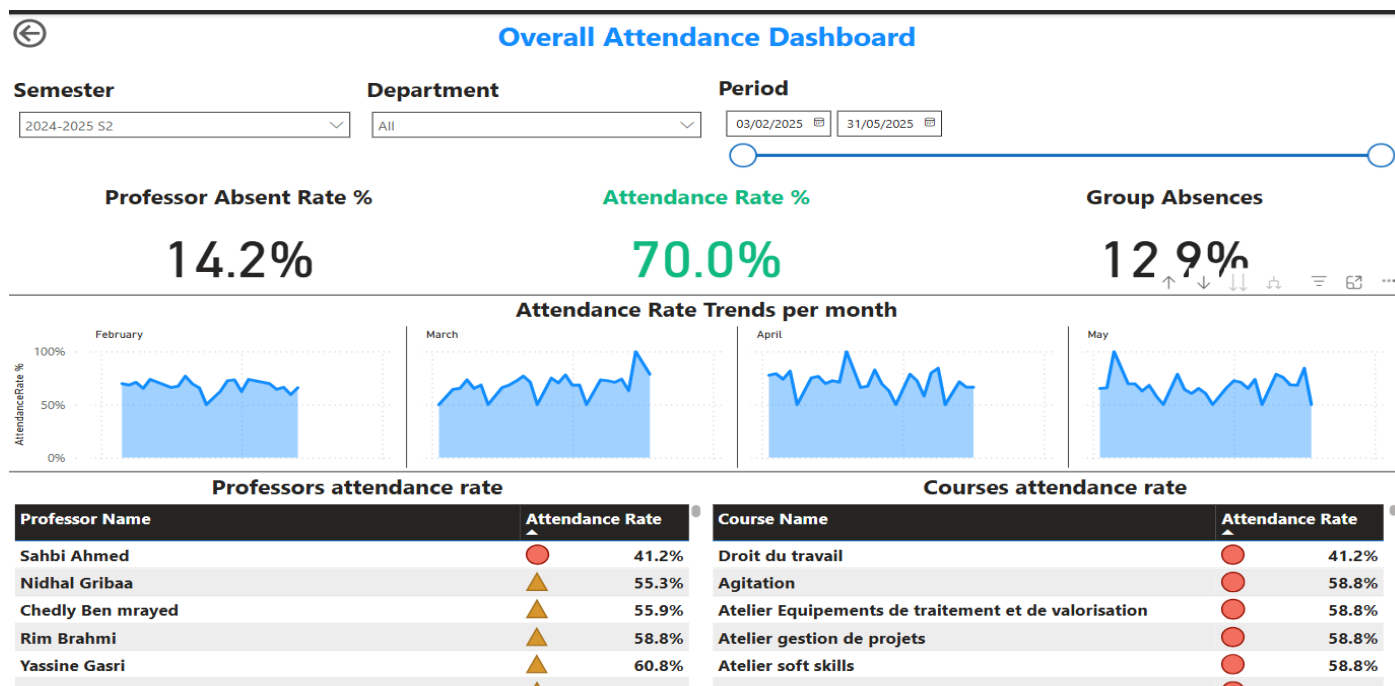


Figure 91: First report "Overall Attendance"



Course Report: C2N

Courses

C2N

85

TotalSessionsCount

76.5%

AttendanceRate %

9.4%

ProfessorAbsentRate %

10.6%

GroupAbsentRate %

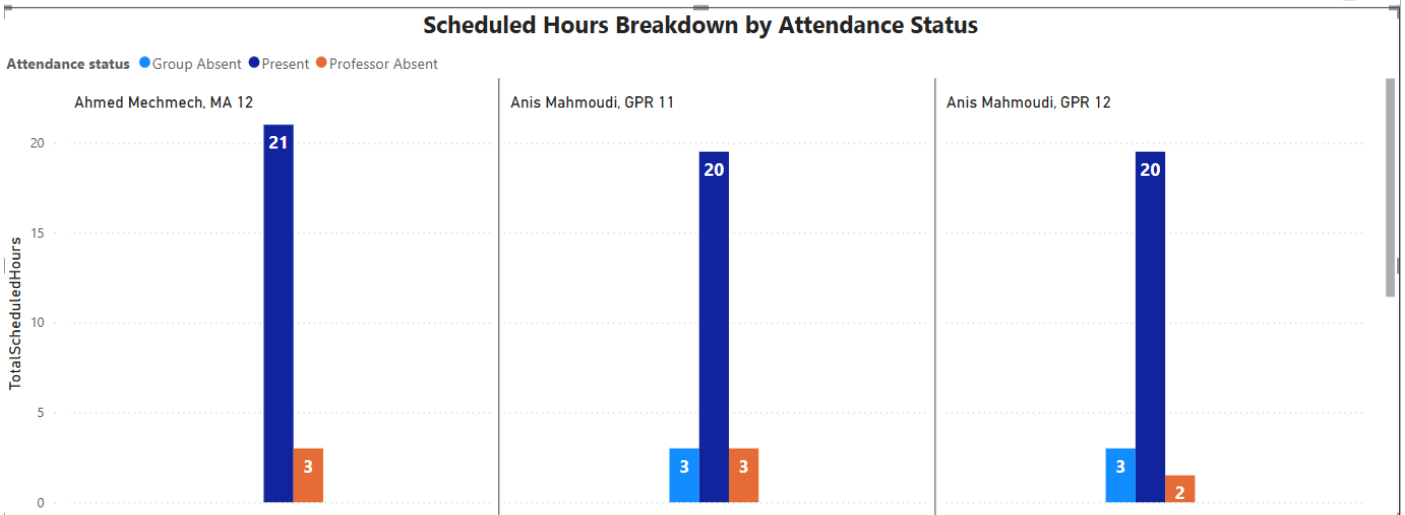


Figure 93: Second report "Course Report"

Professor Report: Mohsen Othmani

Professors

Mohsen Othmani

102

TotalSessionsCount

75.5%

AttendanceRate %

153

TotalScheduledHours

8

ProfessorAbsentCount

77

PresentCount

16

GroupAbsentCount

All attendance

Date	Course Name	Class	Attendance
03 February 2025	Atelier SGBD	DSI 22	Group Absent
03 February 2025	Atelier SGBD	DSI 22	Present
03 February 2025	SGBD	DSI 22	Present
04 February 2025	C2N	TCF 12	Present
10 February 2025	Atelier SGBD	DSI 22	Present
10 February 2025	SGBD	DSI 22	Present
11 February 2025	C2N	TCF 12	Present
17 February 2025	Atelier SGBD	DSI 22	Group Absent
17 February 2025	Atelier SGBD	DSI 22	Present

Attendance rate trends per date

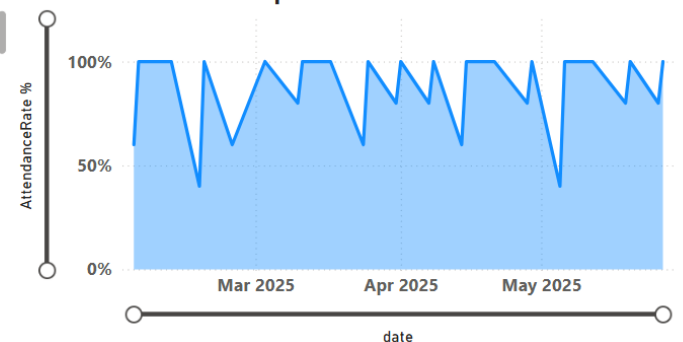


Figure 92: Third report "Professor Report"

Group Report: GPR 11

Class groups

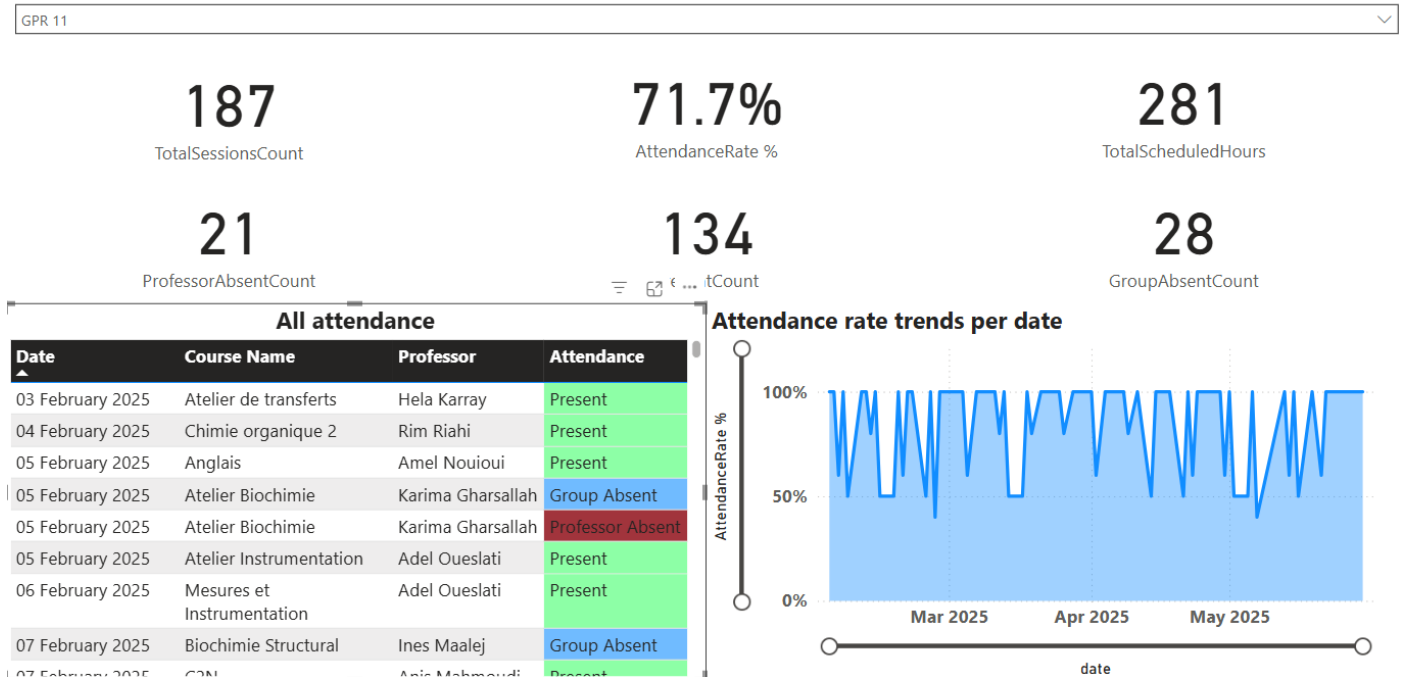


Figure 94: Fourth report "Group Report"

Monthly Alerts

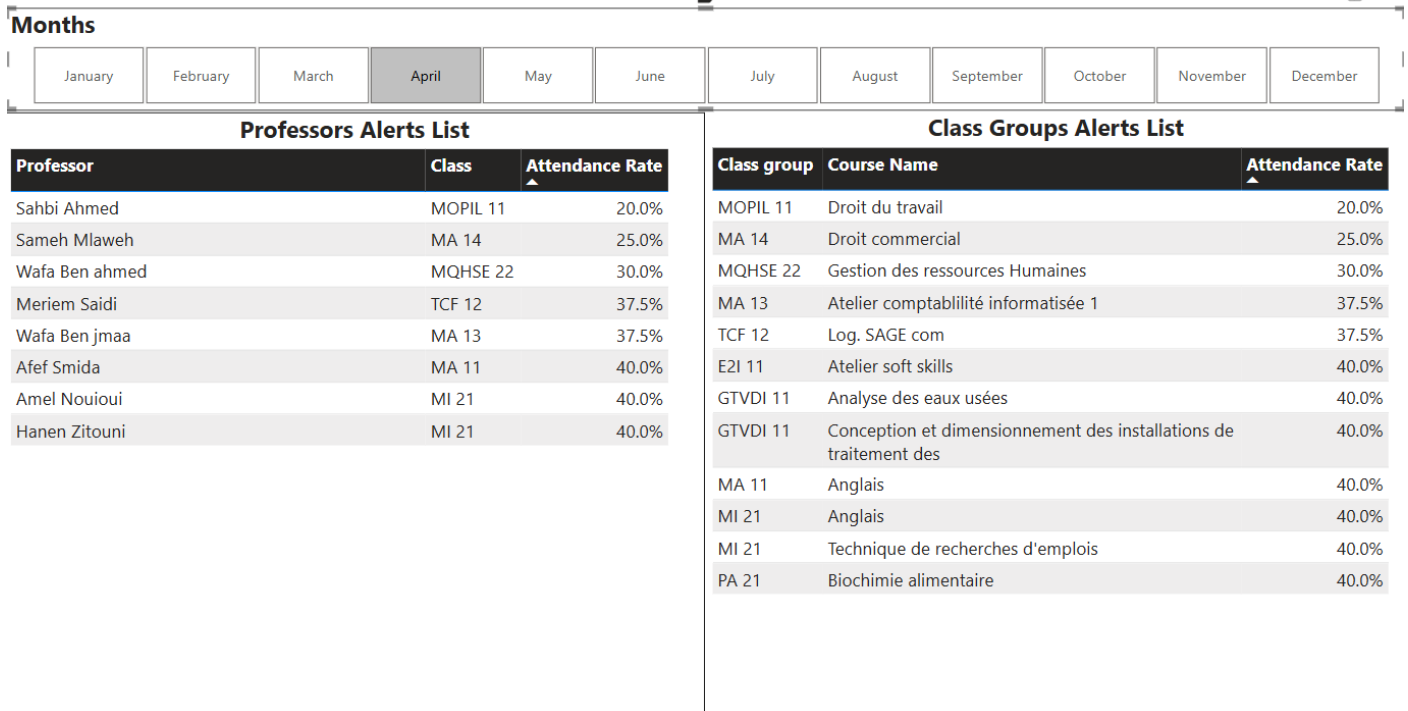


Figure 95: Fifth report: Monthly Alerts

Classrooms Report

Building

Bloc A

Classroom

A06

2331

TotalAvailableHours

7.7%

UtilizationRate %

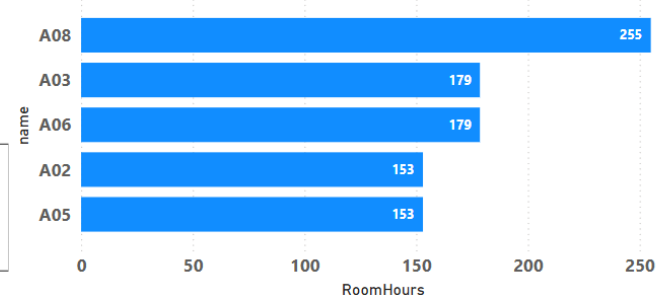
179

TotalScheduledHours

All classrooms usage %

Classroom	RoomHours	RoomUtilization %
Info 09	433.50	18.6%
Info 13	408.00	17.5%
Info 17	357.00	15.3%
C04	331.50	14.2%
C03	280.50	12.0%
C06	280.50	12.0%
Info 07	280.50	12.0%
Info 08	280.50	12.0%
Info 12	280.50	12.0%
A06	255.00	10.9%

Top 5 used classrooms



Bottom 5 used classrooms

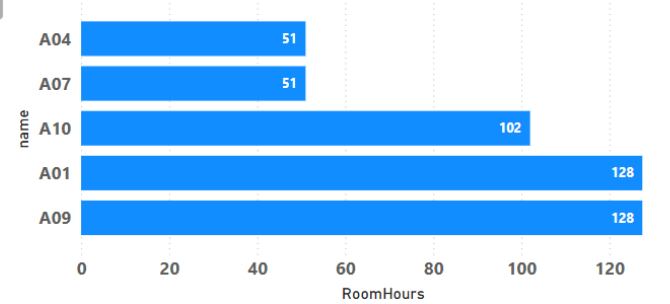


Figure 97: Sixth report: Classrooms Report

Unassigned Report

174

UnassignedCount

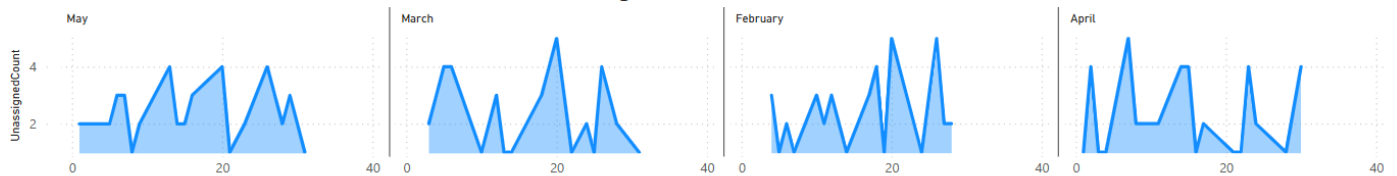
261.00

UnassignedHours

2.9%

UnassignedRate %

Unassigned Trend Per month



All Unassinged Timetables

Date	Classroom	Course Name	Class	Professor	Attendance
04 February 2025	A01	RO II	MOPIL 11	Wafa Mansouri	Unassigned
04 February 2025	Info 12	Atelier comptabilité informatisée 1	TCF 14	Hajer Trabelsi	Unassigned
04 February 2025	Info 16	Atelier comptabilité informatisée 1	TCF 12	Ali Slimene	Unassigned
05 February 2025	Info 01	Atelier virtualisation et cloud	DSI 21	Hela Oueslati	Unassigned
06 February 2025	Info 04	Atelier comptabilité informatisée 1	MA 13	Wafa Ben jmaa	Unassigned
06 February 2025	Salle Bibliothèque	Droit commercial	MA 14	Sameh Mlaweh	Unassigned
07 February 2025	Info 09	Introduction virtualisation et cloud	RSI 21	Malek Bouzayen	Unassigned
10 February 2025	B05	Entrepreneuriat social	PC 21	Faycel Touni	Unassigned
10 February 2025	B08	Marketing de base	MA 15	Mouna Kharoubi	Unassigned

Figure 96: Seventh report: Unassigned Report

8. Security & Role Configuration

In our project, we implemented RLS by creating roles within Power BI Desktop. For each role (such as *Professor* or *Head of Department*), we defined DAX filters that restrict access to data such as showing only the professor's own sessions or a department's internal statistics. These roles were then published along with the report to the Power BI Service, where we assigned users or groups to the appropriate roles. This setup guarantees secure, personalized insights across the university's academic staff.

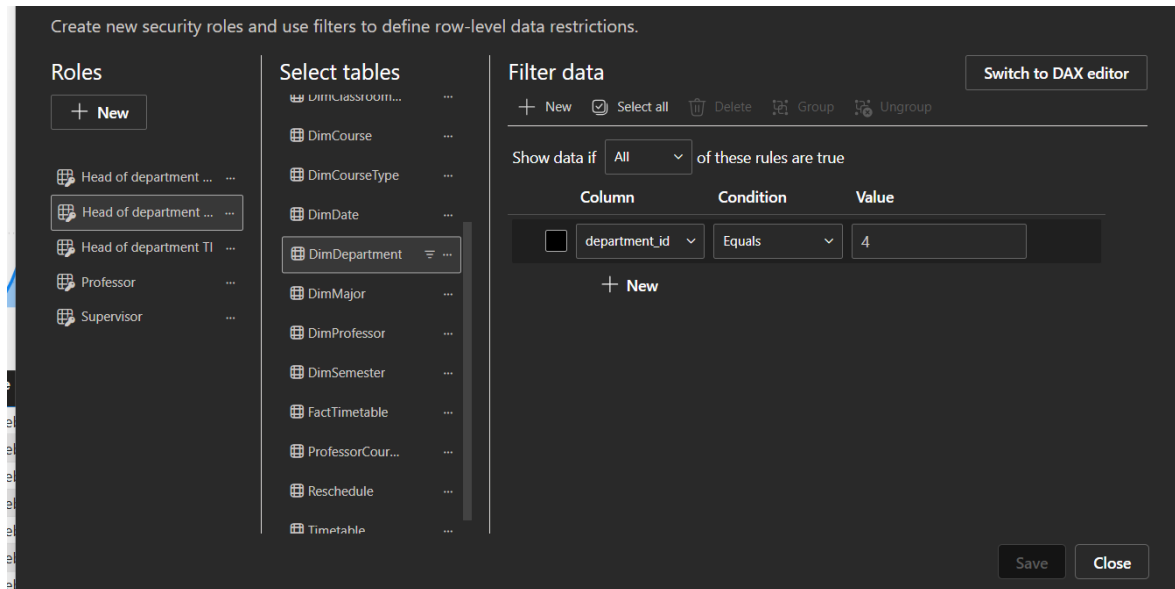


Figure 98: Power BI manage roles

9. Publishing & Sharing

In our project, we published the final reports to a shared workspace on the Power BI Service. Used the sharing features to provide secure access to professors, supervisors, and department heads. By assigning appropriate roles and permissions, each user sees only the data relevant to them, ensuring both accessibility and confidentiality.

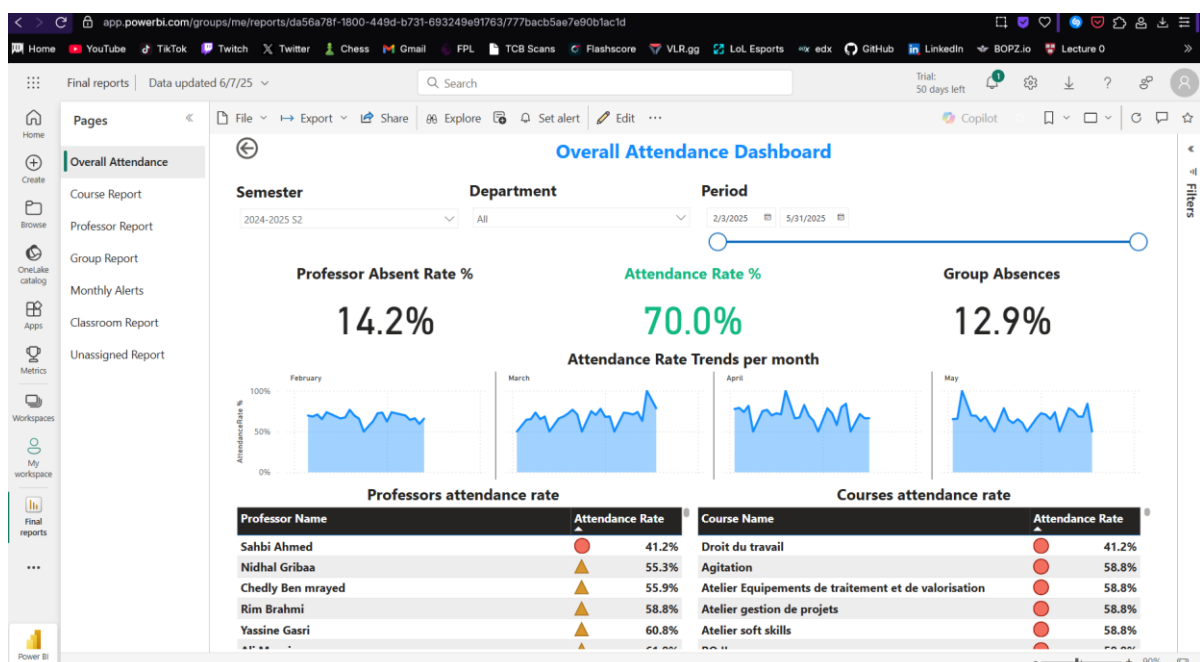


Figure 99: Power BI service

10. Conclusion

This sprint delivered a fully functional Power BI solution connecting live data, providing interactive analytics, and securing access for different user roles thereby completing the pipeline from operational database to strategic insights.

Chapter 7: Sprint 4 Pdf Reports, admin settings management

1. Introduction

This chapter details the final core features established during Sprint 5, focuses on finalizing key administrative and reporting functionalities. In this sprint, we implemented PDF report generation, provided administrator with controls to manage system-wide settings and restrictions.

2. Specification of Requirements

This section outlines the Sprint 4 backlog, alongside the relevant use case diagram and selected user story descriptions

2.1. Sprint Backlog

The Sprint 5 backlog includes user stories related to admin management settings and pdf reporting for attendance in mobile. Each user story generally follows these four steps:

1. Design the interface (mobile or web).
2. Implement backend logic and models.
3. Integrate frontend with backend services.
4. Test and validate the feature.

Table 24 Sprint 4 backlog

ID	User Story	Priority	Complexity
US7	As an administrator agent, I want to generate attendance reports based on a specific date or period so that I can provide reports later via mobile app.	High	8
US17	As an admin, I want to manage departments (view, add, delete, update) so that department information remains accurate.	Low	2
US18	As an admin, I want to manage classes (view, add, delete, update).	Low	2
US19	As an admin, I want to manage classrooms (view, add, delete, update) so that classroom data remains accurate.	Low	2
US20	As a supervisor, I want to view statistics about classroom usage and overuse so that I can make informed decisions about classroom allocation.	Medium	3
US21	As an admin, I want to manage courses (view, add, delete, update).	Low	2
US22	As an admin, I want to manage holiday periods and dates (view, add, update, delete).	Medium	8
US23	As an admin, I want to manage academic events (view, add, update, delete) and set their dates.	High	8
US27	As an admin, I want to manage session (view, add, delete, update) so that sessions data remains accurate.	Low	2

2.2. Use cases

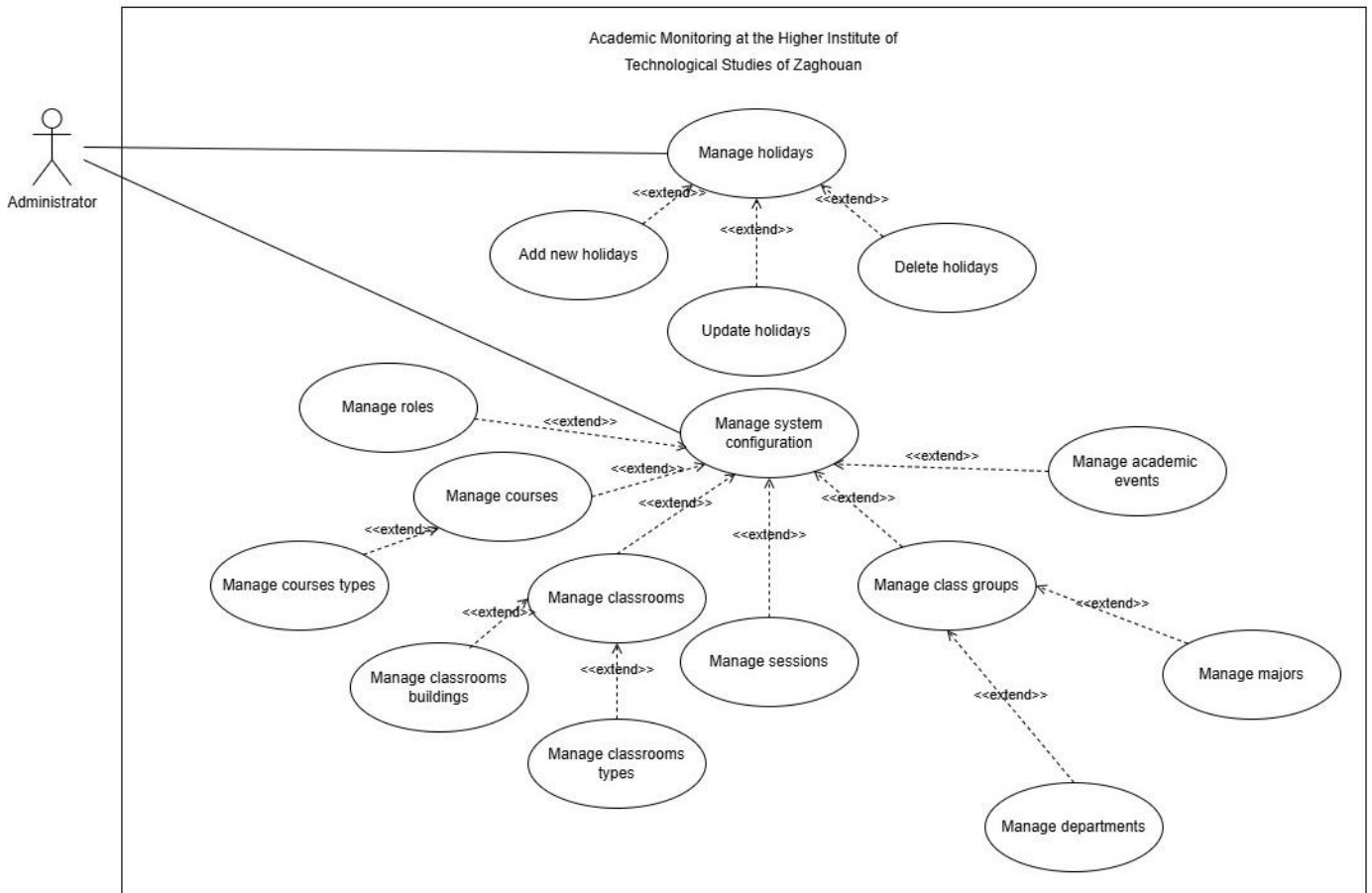


Figure 101: Administrator sprint 4 use case

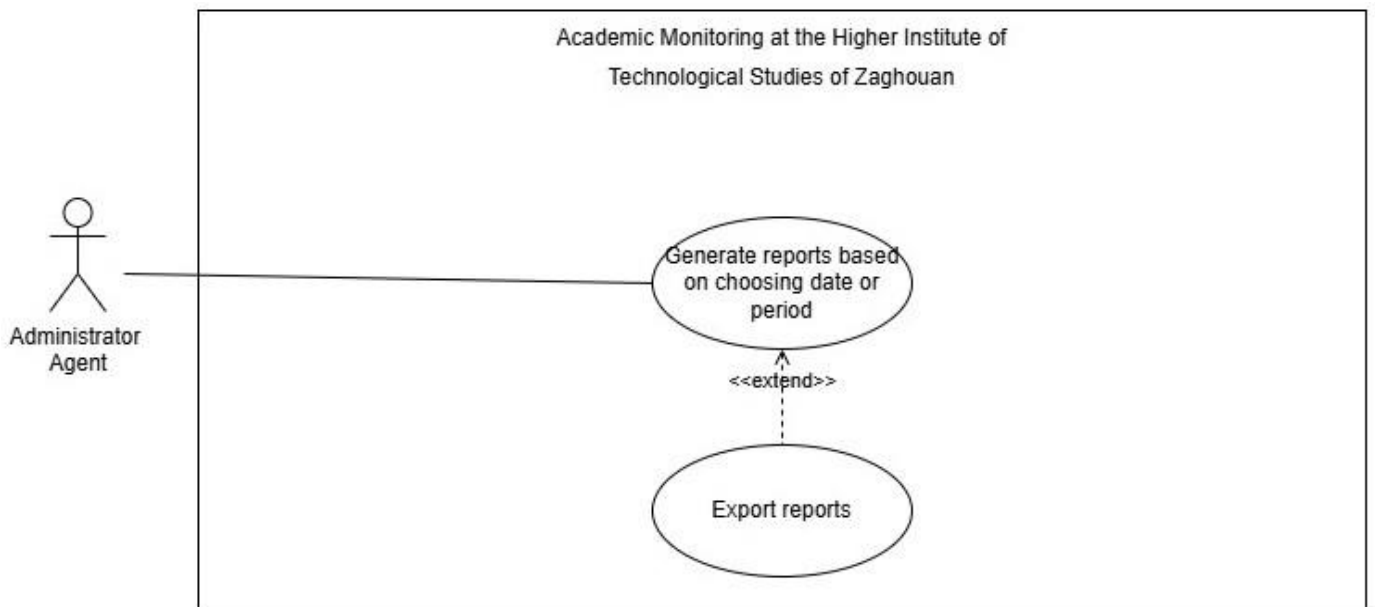


Figure 100: Administrator Agent use case sprint 4

3. Sequence diagrams and textual description of use cases

3.1. Generate and export report pdf use case

In this system, an administrator uses the mobile frontend to generate and download PDF reports. From the report screen, the administrator selects a specific date or period and initiates the export process.

The mobile frontend sends a request to the API server, built with Django REST Framework, including the selected date range. The API server queries the PostgreSQL database to retrieve the relevant data and then generates a PDF report using the fetched information. Once the report is created, the server sends the PDF file back to the mobile frontend, which automatically downloads it to the administrator's device. This seamless process allows administrators to access and review reports directly from their mobile devices, ensuring quick and convenient access to important data.

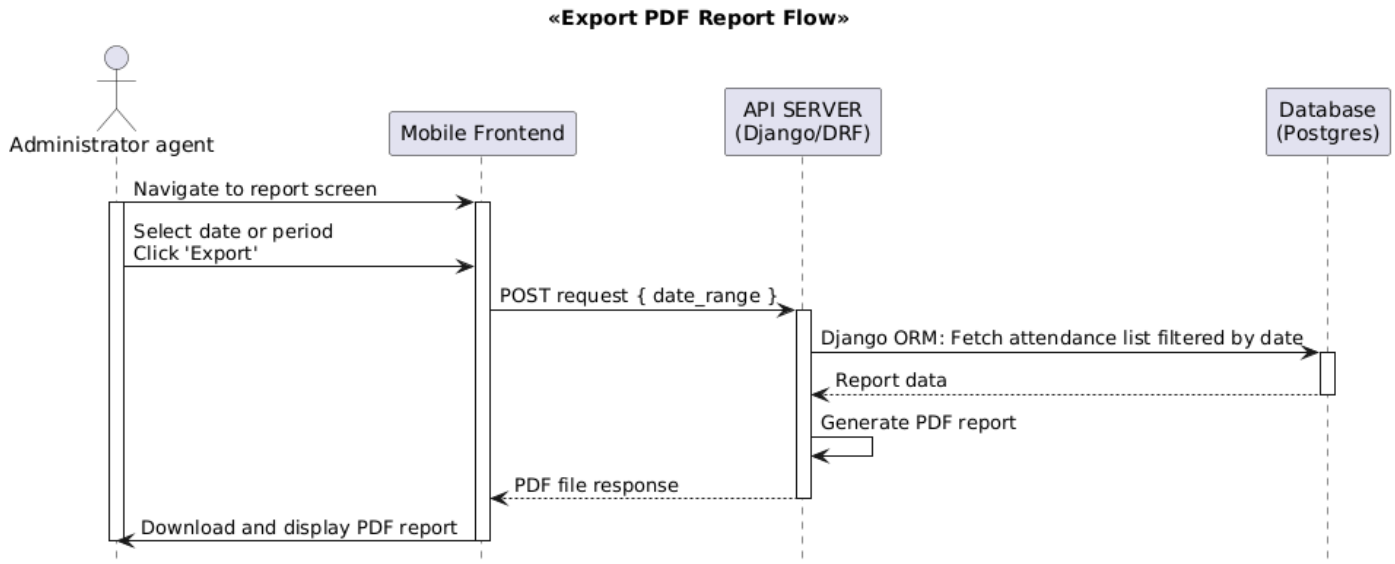


Figure 102: Export PDF sequence diagram

Use Case: Generate reports Pdf	
Title	Generate reports based on choosing date or period
Actor	Administrator agent
Objective	Allow administrator agent to export pdf reports
Description	Enable administrator agent to generate and export pdf reports of attendances based on date or period.
Pre-condition	Needing a report.
Post-condition	Pdf file of the attendance list saved on the device
Nominal Scenario	
1. Navigate to report screen 2. Choose date or period 3. Click export pdf 4. Message displays success message of file downloaded 5. Having pdf file on device	
Alternative Scenario	

3.2. Class group create use case

The "Create Class Group" process allows an administrator to add a new class group through the WEB frontend. The administrator fills out a form with the necessary details, such as the group name, associated major, and semester. Once submitted, the frontend sends the data to the Django REST API. The server validates the input and, if valid, inserts the new class group into the PostgreSQL database.

Upon successful creation, the API responds with confirmation and the newly created group’s details. The frontend then displays a success message or redirects the administrator, completing a smooth and structured creation workflow.

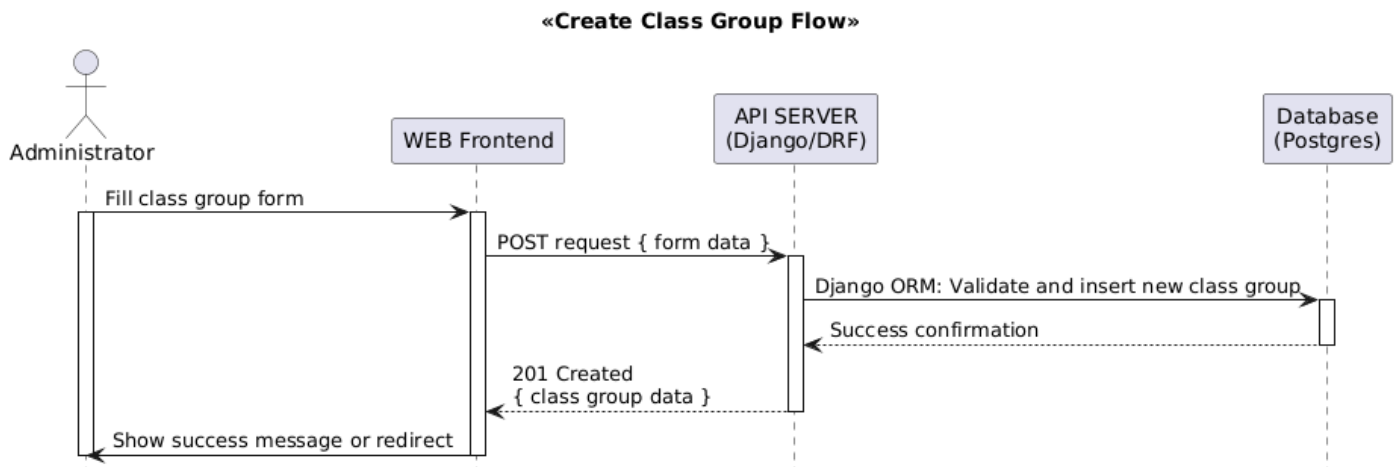


Figure 103: Create class group sequence diagram

Table 25 Analysis creates class group use case

Use Case: manage class group	
Title	Create class group
Actor	Admin
Objective	Allow Admin to create group
Description	Enable Admin to create class group to allow admin to create class groups
Pre-condition	No class group
Post-condition	Class group created
Nominal Scenario	
1. Navigate to Class group management 2. Click Add 3. Fill form with class group details 4. Click add 5. Display success message	
Alternative Scenario	
1: Class group already exist 2: Display message classroom already exists	

3.3. Class group update use case

Updating a class group allows the administrator to modify its details, such as the name, associated major, or semester. Through the mobile frontend, the administrator accesses the group’s current data, makes changes, and submits the update form. The frontend sends a PUT or PATCH request to the Django REST API containing the modified fields. The API performs validation and updates the corresponding class group record in the PostgreSQL database. Once updated, the server responds with the updated data, and the frontend displays a confirmation, ensuring that changes are accurately reflected across the system.

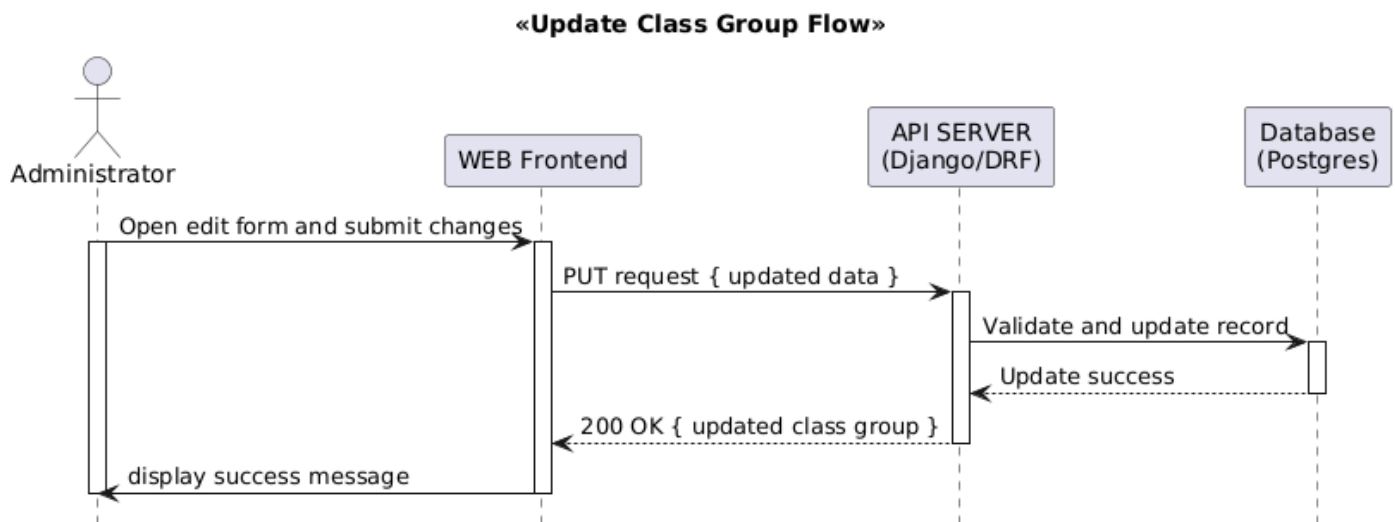


Figure 104: update class group sequence diagram

Table 26 Analysis update class group use case

Use Case: manage class group	
Title	Update class group
Actor	Admin
Objective	Allow Admin to update group
Description	Enable Admin to update class group to allow admin to update class groups
Pre-condition	No class group
Post-condition	Class group update
Nominal Scenario	
1. Navigate to Class group management 2. Click edit 3. Fill form with class group details 4. Click edit 5. Display success message	
Alternative Scenario	
1: Class group already exist 2: Display message classroom already exists	

3.4. Class group delete use case

The delete operation allows an administrator to remove a class group from the system using the mobile frontend. When the administrator chooses to delete a class group, the frontend sends a DELETE request to the Django REST API with the group's identifier. Before deletion, the server verifies that the class group is not referenced elsewhere in the database (e.g., in timetables or attendance records) to avoid breaking data integrity. If the group is safe to delete, it is removed from the PostgreSQL database, and a success response is returned to the frontend. The administrator is then notified that the deletion was successful.

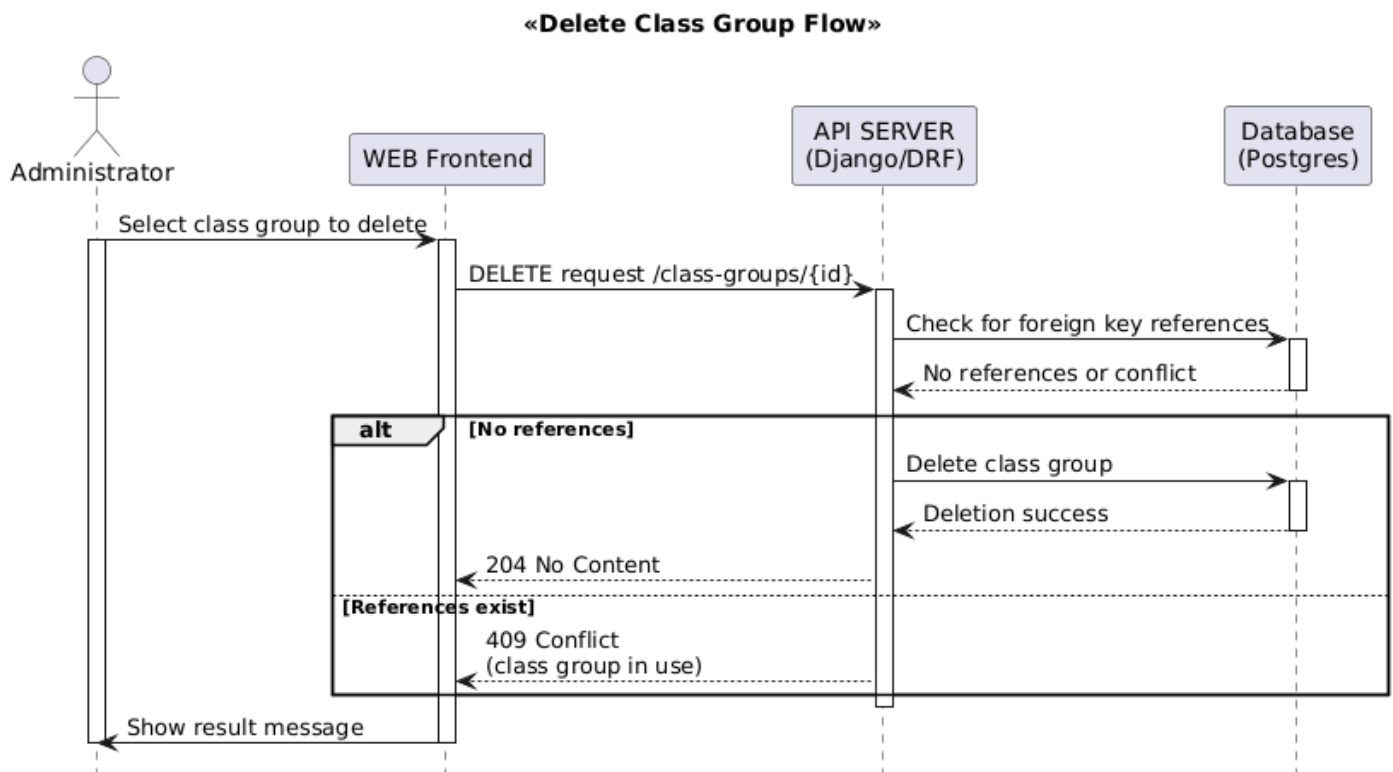


Figure 105: Delete class group sequence diagram

Table 27 Analysis deletes class group use case

Use Case: manage class group	
Title	Update class group
Actor	Admin
Objective	Allow Admin to update group
Description	Enable Admin to update class group to allow admin to update class groups
Pre-condition	No class group
Post-condition	Class group update
Nominal Scenario	
1. Navigate to Class group management 2. Click edit 3. Fill form with class group details 4. Click edit 5. Display success message	
Alternative Scenario	
1: Class group already exist 2: Display message classroom already exists	

The remaining management settings follow the same CRUD structure, with each respecting its own integrity constraints and business rules to prevent conflicts. This approach ensures data accuracy and reduces the likelihood of administrative errors.

4. Implementation

The screenshot shows the 'Department management' interface. On the left is a dark sidebar with a 'University' header and a list of navigation items: Timetable, Semeseter, User Managemets (with a dropdown arrow), Users, Roles, Class Managemets (with a dropdown arrow), Departments (highlighted), Majors, Classes, Course Managemets (with a right arrow), Classroom Managemets (with a right arrow), Sessions, and History. The main content area has a title 'Department management' and a 'Filter...' input field with an 'Add' button. Below this is a table with three columns: 'Tag', 'Name', and 'Actions'. The table contains three rows: GPR (Genie Des Procede), SEG (Sciences Economiques et Sciences de Gestion), and TI (Technologies de l'informatique). Each row has two action icons: a green pencil and a red trash can. At the bottom right of the table area, there are navigation controls: a left arrow, 'Page 1 of 1', and a right arrow.

Tag	Name	Actions
GPR	Genie Des Procede	
SEG	Sciences Economiques et Sciences de Gestion	
TI	Technologies de l'informatique	

Figure 106: Department management screen

The screenshot shows the 'Class Group Management' interface. The sidebar is identical to the previous screen, with 'Class Managemets' highlighted. The main content area has a title 'Class Group Management' and two dropdown filters: 'All Departments' and 'All Majors', followed by an 'Add' button. Below this is a table with two columns: 'Class' and 'Actions'. The table contains ten rows of class codes: DSI 21, DSI 22, DSI 23, E2I 11, GTVDI 11, ISID 11, MI 21, MQHSE 21, MQHSE 22, and MQHSE 23. Each row has two action icons: a green pencil and a red trash can. At the bottom right of the table area, there are navigation controls: a left arrow, 'Page 1 of 4', and a right arrow.

Class	Actions
DSI 21	
DSI 22	
DSI 23	
E2I 11	
GTVDI 11	
ISID 11	
MI 21	
MQHSE 21	
MQHSE 22	
MQHSE 23	

Figure 107: Class group management screen

Semeseter

User Managemets

Users

Roles

Class Managemets

Departments

Majors

Classes

Course Managemets

Course types

Courses

Classroom Managemets

Buildings

Classrom Types

Classrooms

Sessions

History

Sessions Management

All Session Types Add

Session Type	Number	Start Time	End Time	Actions
Study	1	08:30:00	10:00:00	
Study	2	10:10:00	11:40:00	
Study	3	11:50:00	13:20:00	
Study	4	13:30:00	15:00:00	
Study	5	15:10:00	16:40:00	
Study	6	16:40:00	18:10:00	

Page 1 of 1

Figure 108: Session management screen

Semeseter

User Managemets

Users

Roles

Class Managemets

Departments

Majors

Classes

Course Managemets

Course types

Courses

Classroom Managemets

Buildings

Classrom Types

Classrooms

Sessions

History

Classrooms Management

All Buildings Filter by name... Add

Building	Name	Classroom Type	Actions
Bibliothèque	Amphithéâtre	Cours	
Bibliothèque	Salle Bibliothèque	Cours	
Bibliothèque	Salle de formation	Cours	
Bloc A	A01	Cours	
Bloc A	A02	Cours	
Bloc A	A03	Cours	
Bloc A	A04	Cours	
Bloc A	A05	Cours	
Bloc A	A06	Cours	

Page 1 of 8

Figure 109: Classroom management screen

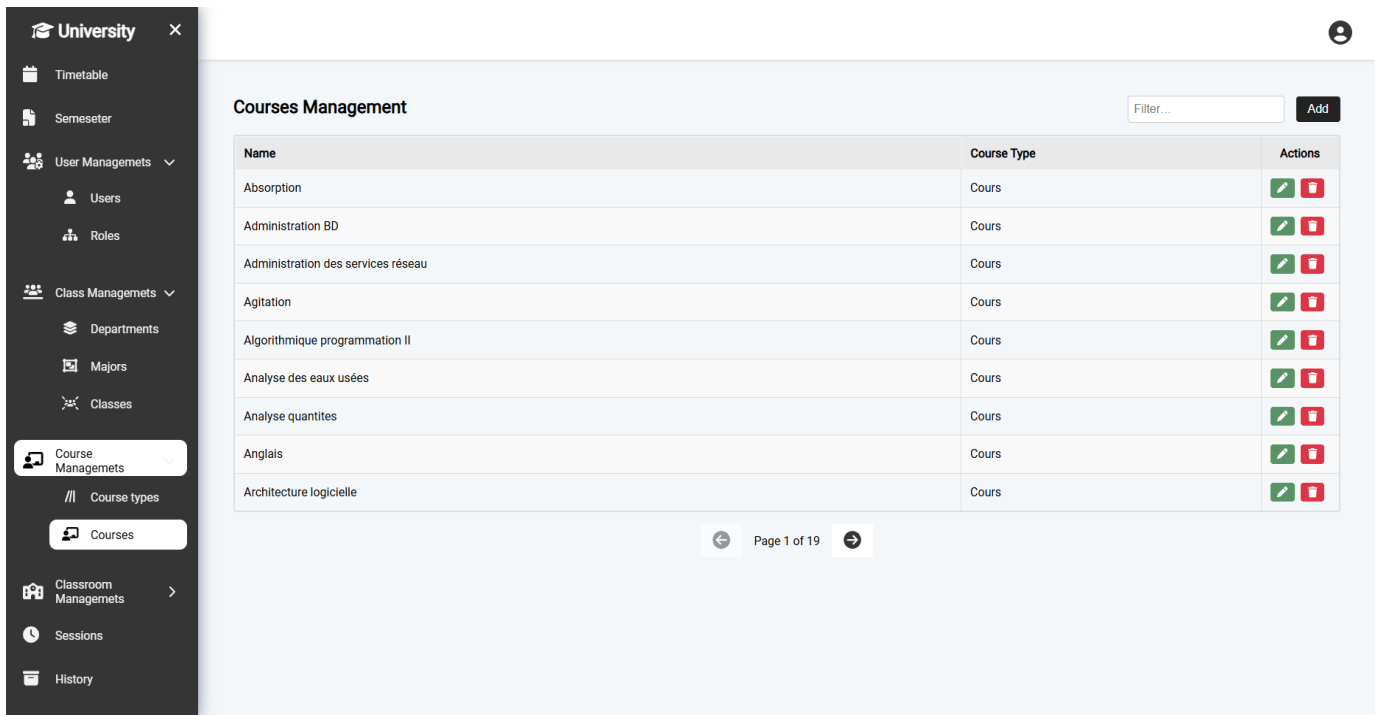


Figure 110; Course management screen

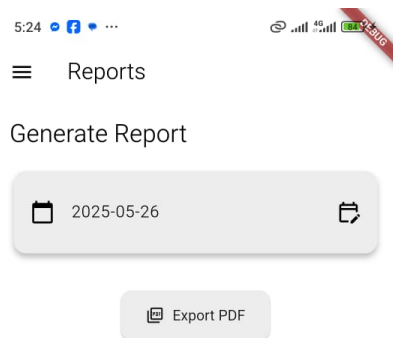


Figure 111: Report screen

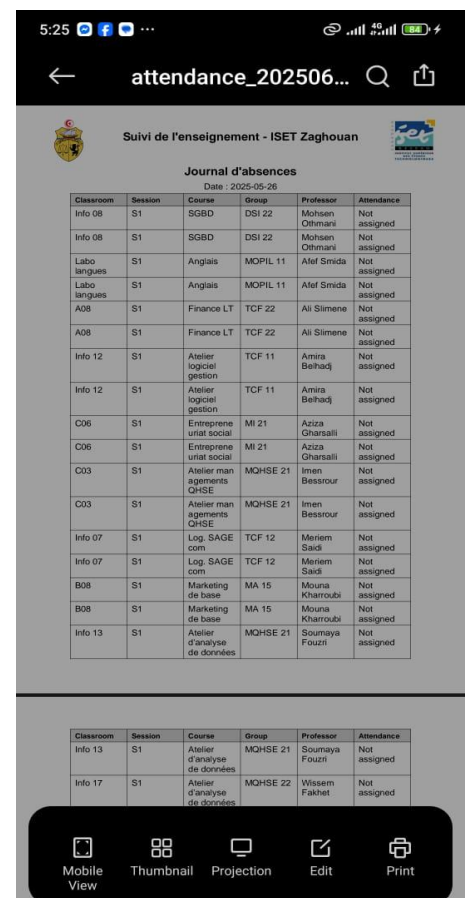


Figure 112: PDF report downloaded on device

5. Conclusion

Sprint 4 successfully delivered the final administrative and reporting functionalities critical for operational efficiency. The system now supports generating PDF attendance and summary reports, enabling supervisors and agents to export accurate documentation and manage all system settings through user friendly interface.

General Conclusion

This work marks the culmination of our final-year project, developed within ISET, with the goal of designing and implementing a comprehensive academic monitoring system to track teaching progress, classroom utilization, and attendance. This final internship phase proved pivotal in bridging theoretical learning with real-world application and would have been far more challenging without the invaluable support and guidance of my supervisor.

We began by clearly identifying the core challenges in academic attendance monitoring, then evaluated existing tools to fully understand their features. Planning was structured using SCRUM methodology, ensuring iterative development and stakeholder feedback. During implementation, we harnessed robust technologies: PostgreSQL for data storage, Angular for the web frontend, Flutter for the mobile application, Django REST Framework for building the API, and Power BI for data visualization and analytics.

Importantly, this project establishes a strong foundation while opening doors to future enhancements, where we already setup basic database structure for reschedule and exams and ds managements some extra ideas of enhancements and future features including:

- **Exam Scheduling Module:** Enable exam planning with automated conflict checks and notifications.
- **Interactive Rescheduling UI:** Introduce drag-and-drop functionality in the web app for intuitive timetable adjustments.
- **Enhanced Dashboards for All Roles:** Develop role-tailored web dashboards mirroring Power BI reports for professors, department heads, and supervisors.
- **Predictive Analytics:** Integrate forecasting models in Power BI for attendance trends and early risk identification.
- **Mobile Offline Support:** Extend the mobile app to capture attendance offline and sync data when online.
- **User Assistance Features:** Embed an AI-powered help chatbot and user guides to assist with in-app tasks and queries.

In conclusion, we believe that the system we developed centered on streamlined attendance assignment, comprehensive statistics, and robust controls will benefit the university significantly and provide a scalable base for future enhancements and innovations in academic monitoring.

Webliography

- [1] ISETZG https://fr.wikipedia.org/wiki/Institut_sup%C3%A9rieur_des_%C3%A9tudes_technologiques
Accessed: 28/05/2025.
- [2] SeatsOne <https://seatssoftware.com> Accessed: 28/05/2025
- [3] oneTap <https://www.onetapcheckin.com> Accessed: 28/05/2025
- [4] Attendance Radar <https://attendanceradar.com> Accessed: 28/05/2025
- [5] Agil <https://www.devx.com/terms/agile-software-development> Accessed: 28/05/2025
- [6] Scrum <https://scrumguides.org> Accessed: 28/05/2025
- [7] Angular <https://angular.dev> Accessed: 28/05/2025
- [8] Flutter <https://flutter.dev> Accessed: 28/05/2025
- [9] Django DRF <https://www.django-rest-framework.org> Accessed: 28/05/2025
- [10] Postgress <https://www.postgresql.org/docs/> Accessed 28/05/2025
- [11] Docker <https://docs.docker.com> Accessed 28/05/2025
- [12] Redis <https://redis.io/docs/latest/> Accessed 28/05/2025
- [13] Celery <https://docs.celeryq.dev/en/latest/getting-started/introduction.html>
Accessed 28/05/2025
- [14] Firebase FCM <https://firebase.google.com/docs/cloud-messaging/concept-options>
Accessed 28/05/2025
- [15] Power BI <https://www.microsoft.com/en-us/power-platform/products/power-bi>
Accessed 28/05/2025
- [16] VS Code <https://code.visualstudio.com> Accessed 28/05/2025
- [17] Android studio <https://developer.android.com/studio> Accessed 28/05/2025
- [18] Postman <https://www.postman.com> Accessed 28/05/2025
- [19] pgAdmin <https://www.pgadmin.org> Accessed 28/05/2025
- [20] draw.io <https://www.drawio.com> Accessed 28/05/2025
- [21] docker desktop <https://docs.docker.com/desktop/> Accessed 28/05/2025
- [22] Trello <https://trello.com> Accessed 28/05/2025
- [23] GITHUB <https://github.com> Accessed 28/05/2025

[24] Fibonacci complexity scrum <https://www.agile-academy.com/en/agile-dictionary/fibonacci-sequence/> Accessed 02/06/2025

[25] BI <https://www.ibm.com/think/topics/business-intelligence> Accessed 06/06/2025

[26] Data Mart <https://blog.naveenchandra.co.in/all-articles/data-mart/> Accessed 06/06/2025